

Dobývání znalostí

Učební text k okruhu státní závěrečné zkoušky

SZZ Umělá inteligence, MFF UK

srpen 2026

Text pokrývá okruh *Dobývání znalostí* magisterské SZZ oboru Umělá inteligence. Kapitoly odpovídají **jedna ku jedné** větám oficiálního znění okruhu (viz mapovací tabulka níže). Vychází ze slidů přednášek **NDBI023 Dobývání znalostí** a **NAIL116 Sociální sítě a jejich analýza** (I. Mrázová, MFF UK) a je doplněn o standardní látku z učebnic Han, Kamber, Pei: *Data Mining — Concepts and Techniques*, Tan, Steinbach, Kumar: *Introduction to Data Mining*, Aggarwal: *Data Mining — The Textbook* a pro sítě Easley, Kleinberg: *Networks, Crowds and Markets*, Newman: *Networks* a Zafarani, Abbasi, Liu: *Social Media Mining*. Obsahuje definice, pseudokódy, odvození, číselné příklady a srovnávací tabulky; každá kapitola končí shrnutím pro ústní zkoušku.

Rozsah textu a značení

Rozsah kapitol odpovídá váze látky: nejvíce prostoru dostávají metody dobývání znalostí (kap. 3) a analýza sociálních sítí (kap. 6) — tedy to, čemu se obě přednášky věnují nejpodrobněji a co okruh jmenuje nejkonkrétněji.

Značkou [♦ nad rámeček slidů] je označena látka, která **není ve slidech** ani jednoho z obou kurzů (slidy jsou k dispozici kompletní), ale je v textu ponechána — buď proto, že ji *jmenuje oficiální znění okruhu*, nebo proto, že bez ní nedává okolní výklad smysl. Slouží jako nabídka ke škrtnutí: co označeno není, je ve slidech doloženo. Následuje-li za značkou dvojtečka, upřesňuje, čeho přesně se výhrada týká — typicky že souhrnná tabulka nebo rámeček jsou moje, ale jednotlivé položky v nich ze slidů pocházejí.

Slidy pokrývají: analýzu dat a shlukování, asociační pravidla, rozhodovací stromy a ensembly, SVM s vyhodnocováním klasifikátorů, bayesovské modely a ELM (NDBI023); základní pojmy, modelování vlivu, homofilii a afiliaci, analýzu vazeb a komplexní sítě s detekcí komunit a predikcí vazeb (NAIL116). Nejnápadnější mezera je **proces KDD a metodologie** (kap. 1) a **charakteristická a diskriminační pravidla** (kap. 4): okruh je jmenuje výslovně, ale *ani jeden z kurzů je neprobírá* — jsou proto zpracovány podle Han–Kamber.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Základní paradigmaty dobývání znalostí.	1 Základní paradigmaty dobývání znalostí
Příprava dat; výběr atributů a metody pro analýzu jejich relevance.	2 Příprava dat, výběr atributů a analýza jejich relevance
Metody pro dobývání znalostí; asociační pravidla, přístupy založené na principu učení s učitelem a klastrová analýza.	3 Metody pro dobývání znalostí (3.1 asociační pravidla, 3.2–3.3 učení s učitelem, 3.4 klastrová analýza)
Metody pro extrakci charakteristických diskriminačních pravidel a měření jejich zajímavosti.	4 Charakteristická a diskriminační pravidla a jejich zajímavost
Reprezentace, vyhodnocování a vizualizace získaných znalostí.	5 Reprezentace, vyhodnocování a vizualizace znalostí
Modely pro analýzu sociálních sítí; míry centrality, detekce komunit.	6 Modely pro analýzu sociálních sítí, centralita a detekce komunit
Praktické využití technik pro dobývání znalostí a analýzu sociálních sítí.	7 Praktické využití technik

Poznámka k terminologii. Text používá české termíny; anglický ekvivalent je uveden kurzívou v závorce při prvním výskytu, protože obě přednášky i celá literatura jsou anglicky.

Obsah

Mapování okruhu na kapitoly	1
1 Základní paradigmaty dobývání znalostí	3
1.1 Motivace a vymezení oboru	3
1.2 Proces KDD	3
1.3 Typy úloh dobývání znalostí	4
1.4 Typy dat	5
1.5 Metodologie dobývání znalostí	5
1.6 Shrnutí ke zkoušce	6
2 Příprava dat, výběr atributů a analýza jejich relevance	7
2.1 Proč je příprava dat klíčová	7
2.2 Čištění dat: chybějící hodnoty	7
2.3 Čištění dat: šum a odlehlé hodnoty	7
2.4 Integrace dat	8
2.5 Transformace a standardizace atributů	8
2.6 Diskretizace numerických atributů	9
2.6.1 Fuzzy diskretizace	9
2.6.2 Seskupování hodnot kategoriálních atributů (KEX)	10
2.7 Redukce dat	10
2.7.1 Příliš mnoho objektů	10
2.7.2 Příliš mnoho atributů	10
2.8 Analýza relevance atributů: filtry a wrappery	10
2.8.1 Kritéria na kontingenčních tabulkách	11
2.8.2 Kritéria založená na entropii	12
2.9 Vztahy mezi atributy: regrese a korelace	12
2.10 Shrnutí ke zkoušce	13
3 Metody pro dobývání znalostí	15
3.1 Asociační pravidla a frekventované vzory	15
3.1.1 Analýza nákupního košíku	15
3.1.2 Support, confidence a lift	15
3.1.3 Volba položek, taxonomie a virtuální položky	16
3.1.4 Algoritmus Apriori	17
3.1.5 Vícenásobný minimální support: MS-Apriori	18
3.1.6 Třídní asociační pravidla (CAR)	19
3.1.7 Dolování sekvenčních vzorů	20
3.1.8 FP-Growth a FP-strom	21
3.2 Přístupy založené na principu učení s učitelem	22
3.2.1 Formulace úlohy	22
3.2.2 Rozhodovací stromy	22
3.2.3 Bayesovská klasifikace	27
3.2.4 Support Vector Machines	27
3.2.5 Logistická regrese	29
3.2.6 Diskriminační analýza	30
3.2.7 Neuronové sítě a extrémní učicí stroje (ELM)	30
3.2.8 Ensemble metody	31
3.2.9 Srovnání klasifikačních metod	33

3.3	Klastrová analýza	33
3.3.1	Formulace úlohy a míry vzdálenosti	33
3.3.2	Metoda k -means	34
3.3.3	Metoda k -medoids, CLARA a CLARANS	35
3.3.4	Hierarchické shlukování	36
3.3.5	Vektorová kvantizace: algoritmus LVQ	36
3.3.6	Mřížkové a hustotní metody	37
3.4	Shrnutí ke zkoušce	37
4	Charakteristická a diskriminační pravidla a jejich zajímavost	40
4.1	Charakterizace vs. diskriminace	40
4.2	Koncepční hierarchie	40
4.3	Indukce orientovaná na atributy (AOI)	40
4.4	Míry t -weight a d -weight	41
4.5	Měření zajímavosti pravidel	42
4.6	Shrnutí ke zkoušce	43
5	Reprezentace, vyhodnocování a vizualizace znalostí	44
5.1	Formy reprezentace znalostí	44
5.2	Vyhodnocování modelů	44
5.2.1	Kritéria hodnocení klasifikačních metod	44
5.2.2	Metody odhadu chyby	45
5.2.3	Matice záměn a odvozené míry	45
5.2.4	Skórování, řazení a lift analýza	46
5.2.5	ROC křivka a AUC	46
5.3	Vizualizační techniky	47
5.4	Vyhodnocení znalostí z pohledu uživatele	48
5.5	Shrnutí ke zkoušce	48
6	Modely pro analýzu sociálních sítí, centralita a detekce komunit	49
6.1	Vymezení oboru a motivace	49
6.2	Reprezentace sítí	49
6.3	Síla vazeb	50
6.4	Klíčoví hráči: míry centrality	51
6.5	Koheze: míry na úrovni sítě	52
6.6	Model reálných komplexních sítí a jejich vlastnosti	53
6.7	Modelování vlivu	54
6.8	Vliv versus korelace	55
6.9	Homofilie, výběr a sociální vliv	56
6.10	Afiliační sítě a uzávěrové procesy	57
6.11	Schellingův model segregace	58
6.12	Pozitivní a negativní vztahy: strukturní balance	58
6.13	Analýza vazeb: huby, autority a algoritmus HITS	59
6.14	PageRank	61
6.15	Tematické komunity a jejich objevování	63
6.16	Detekce komunit	64
6.16.1	Taxonomie metod detekce komunit	67
6.17	Predikce vazeb	69
6.18	Shrnutí ke zkoušce	71
7	Praktické využití technik	73
7.1	Dobývání znalostí	73
7.2	Analýza sociálních sítí	74

7.3	Provozní a etické aspekty nasazení	75
7.4	Shrnutí ke zkoušce	75

1 Základní paradigmatata dobývání znalostí

Kapitola vymezuje obor, jeho proces a základní dělení úloh a dat. Je to rámec, do něhož se zasazují všechny ostatní kapitoly: bez něj nelze vysvětlit, proč je příprava dat samostatná kapitola a proč se metody dělí právě na asociační pravidla, učení s učitelem a shlukování.

1.1 Motivace a vymezení oboru

Organizace shromažďují data v objemech, které daleko přesahují kapacitu člověka je prohlédnout. Klasické dotazování (SQL, OLAP) umí odpovědět na otázku, kterou už umíme položit; *dobývání znalostí* odpovídá na otázky, které jsme položili neuměli — hledá v datech struktury, o jejichž existenci jsme předem nevěděli.

Ilustrativní příklad: systém prošel 20 milionů výkonů účtovaných zdravotní pojišťovně, 214 tisíc z nich označil za podivné a 14 tisíc nejpodzřelejších předal k ruční kontrole. Revizní lékaři, kteří jsou schopni prohlédnout řádově stovky případů, z předložených případů okamžitě označili 8 tisíc za podvodné. Stroj tu nenahradil experta — *zaostřil* jeho pozornost.

Dobývání znalostí z databází

Dobývání znalostí z databází (*Knowledge Discovery in Databases*, KDD) je netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat.

Klíčová jsou všechna čtyři slova:

- **netriviální** — nejde o prostý dotaz ani agregaci;
- **implicitní** — znalost v datech není explicitně zapsána;
- **dosud neznámá** — výsledek, který každý zná, je bezcenný;
- **potenciálně užitečná** — musí vést k akci nebo rozhodnutí.

Termín *dobývání dat* (*data mining*, DM) se v úzkém pojetí vztahuje pouze k jednomu kroku procesu KDD (vlastní aplikaci analytických algoritmů), v běžné praxi se však oba pojmy používají zaměnitelně. Obor se konstituoval v 90. letech 20. století na průsečíku tří disciplín:

- **umělá inteligence** — zejména metody strojového učení (indukce pravidel, stromy, neuronové sítě);
- **databázové systémy** — ukládání a efektivní přístup k velkým datovým souborům, datové sklady, vyhledávání informací;
- **statistika** — modelování a analýza závislostí nalezených v datech, testování hypotéz.

Čtvrtou — a v praxi rozhodující — složkou je nutnost využít zpracovaná data k podpoře (strategického) rozhodování v podniku. Bez ní se dobývání znalostí redukuje na akademické cvičení.

Vztah k datovým skladům a OLAP. [♦ nad rámec slidů] Typickým zdrojem dat pro KDD je *datový sklad* (*data warehouse*) — integrovaná, subjektivě orientovaná, časově variantní a stálá databáze konsolidovaná z provozních systémů. Data se v něm organizují do *datové kostky*: *dimenze* (čas, produkt, region, ...) $\times$ *míry* (tržby, počty kusů); v relační implementaci jde o *hvězdicové schéma* (tabulka faktů obklopená tabulkami dimenzí), fyzicky MOLAP (multidimenzionální uložení) nebo ROLAP (relační). **OLAP** (*On-Line Analytical Processing*) nad kostkou umožňuje agregační operace *roll-up* / *drill-down* a řezy *slice* & *dice* — odpovídá ale jen na otázky, které si analytik umí předem položit. Dobývání znalostí jde o krok dál: hledá vzory, o nichž předem nevíme, že v datech jsou. Souvislost generalizace v OLAP s indukci orientovanou na atributy viz kap. 4.

1.2 Proces KDD

[♦ nad rámec slidů: pětifázový rámec jako celek; náplň jednotlivých fází ve slidech je] Dobývání znalostí je **interaktivní a iterativní** proces: výsledky jednoho kroku běžně vedou k návratu ke krokům předchozím. Klasické schéma (Fayyad et al.) má pět fází:

Fáze	Vstup → výstup	Náplň
Výběr (<i>selection</i>)	původní data → vybraná data	výběr relevantních záznamů a atributů z datového skladu
Předzpracování (<i>pre-processing</i>)	vybraná data → předzpracovaná data	čištění, chybějící hodnoty, šum, odlehlé hodnoty, integrace zdrojů
Transformace (<i>transformation</i>)	předzpracovaná data → transformovaná data	normalizace, diskretizace, odvozené atributy, redukce dimenze
Vlastní dobývání (<i>data mining</i>)	transformovaná data → vzory	aplikace algoritmů (stromy, asociace, klastry, ...)
Interpretace (<i>interpretation</i>)	vzory → znalost	vyhodnocení z pohledu koncového uživatele, vizualizace, nasazení

Cílem prvních tří fází (souhrnně *příprava dat*, kap. 2) je z dat uložených ve složité struktuře vybudovat **jednu tabulku** obsahující relevantní údaje o zkoumaných objektech (klientech banky, zákaznících, ...). Tento krok typicky spotřebuje 60–80 % celkového času projektu.

Sedm kroků očima manažera. [♦ *nad rámec slidů*] Organizační pohled na tentýž proces (impulsem je „aktuální problém“, výstupem „znalost k jeho řešení“): (1) sestavit tým odborníků — experti na oblast, na data a na metody KDD, žádná z rolí není zastupitelná; (2) specifikovat problém — převést obchodní zadání do kontextu dobývání znalostí („zvýšit zisk“ → „predikovat pravděpodobnost odchodu zákazníka“); (3) získat všechna dostupná data včetně externích popisujících prostředí (roční období, reklamní kampaně, počasí) — může vést k přeformulování problému; (4) vybrat metodu či kombinaci metod plus metody vizualizační; (5) předzpracovat data; (6) dolovat — opakovaná aplikace, výsledky předchozích běhů ovlivňují parametry následujících; (7) interpretovat výsledky — část bývá nezájímavá nebo triviální, vhodné je shrnutí do analytické zprávy, výstupem může být i provedení konkrétní akce.

1.3 Typy úloh dobývání znalostí

[♦ *nad rámec slidů*] Podle vztahu nalezené znalosti k cílovému konceptu:

Typ úlohy	Cíl	Charakter výsledku
Klasifikace / predikce	zařadit nové případy, resp. odhadnout budoucí vývoj entity	preferujeme <i>přesnost pokrytí</i> před jednoduchostí; hodně znalosti, obtížně interpretovatelné
Popis (charakterizace, profil)	najít dominantní strukturu či vztahy v datech	požadujeme <i>srozumitelnost</i> při plném pokrytí konceptu; méně znalosti a méně přesné
Hledání „nugetů“	najít zajímavou (novou, překvapivou) znalost	znalost <i>nemusí</i> pokrývat celý koncept; hodnotí se překvapivost

Příklady predikce: předpověď počasí, ceny akcií, spotřeby elektřiny. Příklad popisu: profil bonitního klienta. Příklad nugetu: neobvyklá kombinace zboží v košíku, podezřelý vzorec transakcí.

Deskriptivní vs. prediktivní úlohy

Deskriptivní (*descriptive*) úlohy charakterizují obecné vlastnosti dat: charakterizace a diskriminace, asociační a korelační analýza, shlukování, detekce odlehlých hodnot, analýza vývoje.

Prediktivní (*predictive*) úlohy provádějí inferenci nad daty za účelem predikce: klasifikace (diskrétní cílový atribut), regrese (spojitý cílový atribut), predikce časových řad.

Ortogonální dělení podle dostupnosti cílové proměnné: *učení s učitelem* (*supervised*) — klasifikace, regrese; *učení bez učitele* (*unsupervised*) — shlukování, asociační pravidla, detekce odlehlých hodnot; *poloučené* (*semi-supervised*) — malá označovaná a velká neoznačovaná část.

Toto dělení je zároveň klíčem ke kap. 3: okruh v ní jmenuje právě jednoho reprezentanta učení bez učitele s transakčními daty (asociační pravidla), učení s učitelem a učení bez učitele s metrickými daty (klastrová analýza).

1.4 Typy dat

Podle typu atributu — určuje, jaké operace jsou smysluplné a jaké metody lze použít:

Škála	Operace	Příklad	Statistika
Nominální (<i>nominal</i>)	$=, \neq$	barva, druh ovoce	modus, četnosti
Ordinální (<i>ordinal</i>)	$=, <$	známka, {nízký, střední, vysoký}	medián, kvantily
Intervalová (<i>interval-scaled</i>)	$+, -$	teplota ve °C, datum	průměr, směr. odchylka
Poměrová (<i>ratio-scaled</i>)	$\times, /$	příjem, hmotnost, četnost	geometrický průměr

U intervalové škály je klíčové, že *intervals mají v celém rozsahu stejnou důležitost*: rozdíl věku 10 a 20 let je týž jako mezi 40 a 50 lety. U poměrové škály navíc existuje absolutní nula, takže má smysl poměr; poměrové atributy s exponenciálním chováním (např. počet mikroorganismů Ae^{Bt}) se před dalším zpracováním logaritmují a pak se s nimi zachází jako s intervalovými. *Nominální* atribut o v hodnotách se pro numerické metody převádí na v binárních atributů (hodnota 1 pro odpovídající kategorii, jinak 0); *ordinální* atribut se běžně zpracovává jako intervalový.

Podle struktury datového souboru:

- **Tabulková / relační data** — objekty $\times$ atributy; standardní vstup většiny metod.
- **Transakční data** — proměnná délka záznamu (nákupní košík); vstup asociačních pravidel.
- **Temporální data** (*temporal*) — např. časové řady kurzů akcií. Typickou úlohou je predikce budoucích hodnot; řada se transformuje do tabulky posuvným oknem: vstupy $y(t_0), y(t_1), y(t_2), y(t_3) \rightarrow$ výstup $y(t_4)$; $y(t_1), \dots, y(t_4) \rightarrow y(t_5)$ atd.
- **Prostorová data** (*spatial*) — geografické informační systémy; implicitní relace sousedství (hodnoty atributů sousedních objektů se od sebe příliš neliší), užitečné zejména pro interpretaci.
- **Strukturní data** (*structural*) — např. chemické sloučeniny zapsané řetězcí SMILES nebo soubory faktů v Prologu; úlohou je třeba klasifikace látek podle struktury.
- **Textová a webová data, multimédia, grafy a sítě** — vyžadují specifickou reprezentaci (vektorový model, grafové modely, viz kap. 6).

1.5 Metodologie dobývání znalostí

[♦ nad rámec slidů] Cílem metodologie je poskytnout jednotný rámec a vést aplikace bez ohledu na obor podnikání — umožnit sdílení a přenos zkušeností z úspěšných projektů. Metodologie vznikají buď u velkých výrobců BI nástrojů (SEMMA od SAS Institute, 5A od SPSS), nebo jako neutrální, softwarově nezávislé standardy (CRISP-DM).

SEMMA (SAS Enterprise Miner), pět fází: *Sample* — výběr dat pro modelování (vzorkování, imputace, rozdělení na trénovací, testovací a validační množinu); *Explore* — vizuální průzkum a redukce dat (odlehle hodnoty, vícerozměrné histogramy, shlukování, Kohonenovy mapy); *Modify* — příprava objektů, hodnot a proměnných (výběr, tvorba, transformace); *Model* — aplikace metod (stromy, regrese, neuronové sítě); *Assess* — vyhodnocení spolehlivosti a užitečnosti; *uživatel jim musí rozumět*.

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) — otevřený, oborově, nástrojově a aplikačně neutrální model procesu; vývoj financovala iniciativa ESPRIT, konsorciem tvořily ISL (později SPSS, dnes IBM), Teradata, NCR, Daimler AG a pojišťovna OHRA. Šest fází, jejichž **pořadí není striktní** — výsledky jedné fáze určují volbu dalších kroků a některé kroky je nutné opakovat (proces se zobrazuje jako cyklus s vnějším cyklem nasazení a monitoringu):

1. **Porozumění problému** (*Business Understanding*) — obchodní cíle a kritéria úspěchu; inventura zdrojů (lidé, informace, software, výpočetní kapacita), předpoklady a omezení, rizika, náklady a přínosy; cíle dobývání znalostí; plán projektu.
2. **Porozumění datům** (*Data Understanding*) — sběr počátečních dat; popis (množství, vlastnosti, formát); průzkum a vizualizace (rozdělení klíčových atributů, vztahy dvojic, subpopulace,

jednoduché statistiky); ověření kvality dat.

3. **Příprava dat** (*Data Preparation*) — výběr, čištění, konstrukce odvozených atributů, integrace, agregace, formátování; iterativně a v různém pořadí.
4. **Modelování** (*Modeling*) — volba techniky (s ohledem na její předpoklady, může vynutit další úpravy dat), návrh testu, sestavení modelu s různými parametry, posouzení modelu.
5. **Vyhodnocení** (*Evaluation*) — výsledky vůči *obchodním* kritériím úspěchu; schválení modelů; revize procesu; určení dalších kroků.
6. **Nasazení** (*Deployment*) — prezentace ve formě snadno interpretovatelné a nasaditelné uživatelem; plán nasazení, monitoring a údržba; závěrečná zpráva a revize projektu (dokumentace obtíží a slepých uliček).

ASUM-DM (IBM, 2015) reaguje na hlavní výtoku vůči CRISP-DM — malou podporu projektového řízení. Fáze *Analyze, Design, Configure & Build, Deploy, Operate & Optimize*, průřezově *Project Management*.

	SEMMA	CRISP-DM	ASUM-DM
Původ	SAS Institute	ESPRIT / konsorcium	IBM (2015)
Neutralita	vázáno na SAS EM	nástrojově neutrální	vázáno na IBM
Byznys fáze	chybí	ano (fáze 1 a 5)	ano
Projektové řízení	ne	slabé	explicitní, průřezové
Počet fází	5	6	5 + PM

1.6 Shrnutí ke zkoušce

- **KDD** = netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat; interaktivní a iterativní.
- **Pět fází KDD**: výběr → předzpracování → transformace → dobývání → interpretace. Příprava dat spotřebuje 60–80 % času.
- **Kořeny oboru**: umělá inteligence (strojové učení), databáze, statistika + potřeba podpory rozhodování. **OLAP** odpovídá na předem položené otázky, KDD hledá vzory, o nichž nevíme.
- **Tři typy úloh**: klasifikace/predikce (přesnost > jednoduchost), popis (srozumitelnost, plné pokrytí), hledání nugetů (překvapivost, nemusí pokrývat).
- **Deskriptivní vs. prediktivní úlohy; s učitelem vs. bez učitele vs. poloučené.**
- **Škály atributů**: nominální, ordinální, intervalová, poměrová — určují přípustné operace i metody. Nominální → v binárních atributů; poměrový s exponenciálním chováním → logaritmovat.
- **Metodologie**: SEMMA (Sample, Explore, Modify, Model, Assess), CRISP-DM (6 fází, pořadí není striktní, cyklické), ASUM-DM (CRISP-DM + projektové řízení).
- Nejčastější doplňující otázka: *proč pořadí fází CRISP-DM není striktní?* — protože výsledky fáze určují další kroky; např. zjištěná kvalita dat vede zpět k přeformulování cíle.

2 Příprava dat, výběr atributů a analýza jejich relevance

Věta okruhu spojuje dvě věci do jedné kapitoly: *přípravu dat* (co udělat, aby data byla vůbec zpracovatelná) a *výběr atributů s analýzou jejich relevance* (jak z mnoha atributů vybrat ty, které nesou informaci). Druhá část je vlastně speciální případ první — redukce dimenze — ale okruh ji jmenuje zvlášť, protože se u ní očekává znalost konkrétních kritérií a umění je spočítat.

2.1 Proč je příprava dat klíčová

Předzpracování má *klíčový význam pro úspěch aplikace*, je obtížné a časově náročné a výhodou je spolupráce s doménovým expertem. Cílem je:

- vybrat (nebo vytvořit) z dostupných dat ta, která jsou relevantní pro danou úlohu,
- reprezentovat je ve formě vhodné pro zvolený algoritmus,
- výsledným stavem je (*jedna*) datová tabulka obsahující hodnoty atributů objektů.

♦ *nad rámec slidů* Kvalita dat se posuzuje podle rozměrů: *přesnost* (*accuracy*), *úplnost* (*completeness*), *konzistence*, *aktuálnost* (*timeliness*), *důvěryhodnost*, *interpretovatelnost*. Zásada „garbage in, garbage out“ platí bez výjimky: sebelepší algoritmus na špatných datech vytvoří jen sebejistý nesmysl.

2.2 Čištění dat: chybějící hodnoty

Základní strategie (v pořadí od nejjednodušší k nejpřesnější):

1. **Ignorovat objekty** s libovolnou chybějící hodnotou. Jednoduché, ale při mnoha attributech ztratíme většinu dat; navíc zavádí zkreslení, pokud chybějící hodnoty nejsou náhodné.
2. **Nahradit novou hodnotou** „nevím“ (*I don't know*). Chybějící hodnota se stane plnohodnotnou kategorií — vhodné, když samotná absence nese informaci (nevyplněný příjem u žádosti o úvěr).
3. **Nahradit existující hodnotou atributu**: nejčastější hodnotou (modus, u numerických průměr či medián), úměrně poměru všech hodnot (náhodně podle empirického rozdělení), nebo libovolnou hodnotou.
4. **Doplnit hodnotu pomocí modelu** — naučit prediktivní model (regrese, rozhodovací strom, k -NN) předpovídat chybějící atribut z ostatních. Nejpřesnější, výpočetně nejnáročnější, riskuje zavlečení umělé závislosti mezi atributy.

♦ *nad rámec slidů* Terminologicky se rozlišuje MCAR (*missing completely at random*), MAR (*missing at random* — chybění závisí na pozorovaných attributech) a MNAR (*missing not at random* — závisí na nepozorované hodnotě samotné). Jen u MCAR je prosté vypuštění nezkreslené.

Některé metody umí chybějící hodnoty zpracovat samy: C4.5 je při konstrukci stromu ignoruje (kritérium dělení počítá jen ze známých hodnot) a při klasifikaci je odhaduje z ostatních vzorů; CART chybějící hodnoty do rozhodovacího kritéria nezapočítává; naivní Bayes umí klasifikovat i nekompletně popsané vzory (kap. 3).

2.3 Čištění dat: šum a odlehlé hodnoty

Šum (*noise*) je náhodná chyba nebo rozptyl v měřené veličině. Techniky vyhlazování: **binning** (hodnoty se rozdělí do košů a nahradí průměrem, mediánem nebo hranicí koše), **regrese** (hodnoty se nahradí predikcí regresní funkce), **shlukování** (hodnoty mimo shluky se označí za odlehlé) a **kombinace stroje a člověka** (podezřelé hodnoty se předloží ke kontrole).

Rozpětí, kvartily, mezikvartilové rozpětí, odlehlá hodnota

Rozpětí (*range*) množiny dat je rozdíl nejvyšší a nejnižší hodnoty.

Kvartily Q_1, Q_2, Q_3 dělí data na čtyři části: 25 % pozorování má hodnotu menší než dolní kvartil Q_1 , 50 % pod mediánem Q_2 a 75 % pod horním kvantilem Q_3 .

Mezikvartilové rozpětí (*interquartile range*): $IQR = Q_3 - Q_1$.

Odlehlá hodnota (outlier) je extrémní hodnota ležící mimo celkový vzorec dat. Běžné pravidlo: odlehlá je každá hodnota

$$x > Q_3 + 1,5 \cdot \text{IQR} \quad \text{nebo} \quad x < Q_1 - 1,5 \cdot \text{IQR}.$$

Výpočet kvartilů pro n vzestupně seřazených hodnot (diskrétní data): pro Q_1 dělíme n čtyřmi — je-li $n/4$ celé číslo, bereme střed mezi odpovídajícím a následujícím členem; není-li celé, zaokrouhlíme nahoru a vezmeme odpovídající člen. Pro Q_3 analogicky s $3n/4$. U spojitých dat se místo zaokrouhlení interpoluje.

Příklad — krabicový graf a odlehlé hodnoty

Hladina glukózy v krvi u 30 žen (mmol/l), stonkový diagram (klíč: 2 | 1 znamená 2,1):

2 | 2 2 3 3 5 7 (6)

3 | 1 2 6 7 7 7 8 8 8 8 9 9 9 (13)

4 | 0 0 0 0 4 5 6 7 8 (9)

5 | 1 5 (2)

$n = 30$, $30/4 = 7,5 \rightarrow$ zaokrouhlíme na 8, osmé číslo je 3,2, tedy $Q_1 = 3,2$. Medián $Q_2 = 3,8$.

Pro Q_3 : $3 \cdot 30/4 = 22,5 \rightarrow 23$, 23. číslo je 4,0, tedy $Q_3 = 4,0$.

$\text{IQR} = 4,0 - 3,2 = 0,8$; dolní mez $= 3,2 - 1,5 \cdot 0,8 = 2,0$; horní mez $= 4,0 + 1,5 \cdot 0,8 = 5,2$.

Odlehlá hodnota je 5,5. Nejvyšší hodnota, která odlehlá není, je 5,1 — v krabicovém grafu (*box plot*) k ní vede vous, odlehlá hodnota se vyznačí křížkem.

Násobek 1,5 je konvence; s násobkem 1 dostaneme přísnější kritérium. Pro želví samce s $Q_1 = 400$ g, $Q_3 = 580$ g je $\text{IQR} = 180$, meze $400 - 180 = 220$ a $580 + 180 = 760$ — žádná z hodnot 400, 260, 550, 640 g není odlehlá a největší hmotnost bez označení za outlier je 760 g. Pro samice s $Q_1 = 260$, $Q_3 = 340$, $\text{IQR} = 80$ jsou meze 180 a 420, takže odlehlé jsou hodnoty 170 g a 440 g.

2.4 Integrace dat

♦ **nad rámeček slidů** Při *integraci (data integration)* spojujeme více zdrojů do jedné tabulky. Hlavní problémy: **problém identity entity** (atribut `cust_id` v jedné databázi odpovídá `customer_number` v druhé; nutná metadata a párování záznamů); **redundance** (atribut odvoditelný z jiných — roční příjem $= 12 \times$ měsíční; detekuje se korelační analýzou u numerických a χ^2 testem u kategoriálních atributů, viz odd. 2.8); **konflikty hodnot** (různé jednotky, škály, kódování); **duplicita** (tentýž objekt vícekrát s drobně odlišnými hodnotami).

2.5 Transformace a standardizace atributů

V euklidovském prostoru je standardizace atributů doporučená, aby všechny atributy mohly mít *stejný vliv* na výpočet vzdáleností. Pro body $x_i = (0,1; 20)$ a $x_j = (0,9; 720)$ je

$$\text{dist}(x_i, x_j) = \sqrt{(0,9 - 0,1)^2 + (720 - 20)^2} = 700,000457,$$

tedy vzdálenost je zcela ovládnuta rozdílem $720 - 20 = 700$ ve druhém atributu; první atribut nemá prakticky žádný vliv. Standardizace nutí atributy do společného rozsahu hodnot.

Základní standardizační transformace

Nechť f je atribut a x_{if} jeho hodnota u i -tého objektu.

Desítkové škálování (decimal scaling) do $[-1, 1]$: dělíme hodnoty nejmenší mocninou 10, která udrží všechny transformované hodnoty v $[-1, 1]$.

Min-max normalizace (range standardization) do $[0, 1]$:

$$\text{range}(x_{if}) = \frac{x_{if} - \min(f)}{\max(f) - \min(f)},$$

do $[-1, 1]$ pak $\text{range}_{[-1,1]}(x_{if}) = 2 \cdot \text{range}(x_{if}) - 1$.

Standardizace podle průměrné absolutní odchylky — nulový průměr a jednotková střední absolutní odchylka:

$$m_f = \frac{1}{n}(x_{1f} + \dots + x_{nf}), \quad s_f = \frac{1}{n}(|x_{1f} - m_f| + \dots + |x_{nf} - m_f|), \quad z(x_{if}) = \frac{x_{if} - m_f}{s_f}.$$

Standardizace podle směrodatné odchylky (*std-score*) — nulový průměr a jednotková korigovaná výběrová směrodatná odchylka:

$$\text{cors}_f = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{if} - m_f)^2}, \quad \text{std}(x_{if}) = \frac{x_{if} - m_f}{\text{cors}_f}.$$

Volba mezi oběma z-skóry: střední absolutní odchylka je *robustnější* k odlehlým hodnotám než směrodatná (odchylky se neumocňují na druhou), takže odlehlé hodnoty stlačí ostatní data méně.

2.6 Diskretizace numerických atributů

Řada metod (asociační pravidla, ID3, CHAID, bayesovská klasifikace s diskrétními atributy) vyžaduje kategoriální vstup. *Diskretizace* rozdělí rozsah numerického atributu na intervaly. Metody se liší ve čtyřech ohledech: *strategií* (dělení intervalů shora, nebo slučování zdola), *počtem* intervalů, *typem* intervalů a *kritériem* kvality (minimální klasifikační chyba, entropie, χ^2 test).

- **Ekvidistantní** (*equi-width*) — rozsah se rozdělí na intervaly stejné délky. Nejjednodušší, ale citlivé na odlehlé hodnoty a nerovnoměrné rozdělení.
- **Ekvifrekvenční** (*equi-depth*) [♦ nad rámec slidů] — intervaly se stejným počtem objektů.
- **S využitím informace o třídě** — hranice se volí tak, aby intervaly byly co nejčistší; kritériem je entropie (rekurzivní dělení, jako v C4.5) nebo χ^2 (slučování zdola, jako v CHAID).

2.6.1 Fuzzy diskretizace

Hranice intervalů jsou u běžné diskretizace ostré, takže dva blízké objekty mohou skončit v různých kategoriích. *Fuzzy diskretizace* hranice „rozostří“: jedné numerické hodnotě mohou odpovídat *dvě* diskretizované hodnoty, přičemž součet příslušných charakteristických funkcí je 1. Vznikají tak „fuzzy“-objekty s vahou < 1 ; váha fuzzy-objektu je dána *součinem* hodnot charakteristických funkcí všech atributů:

$$w = \prod_j \mu_j.$$

Rizikem je ztráta informace a *kontradikce* — objekty se stejnými hodnotami diskretizovaných atributů, které patří do různých tříd.

Procedura *Interval* (fuzzy diskretizace)

1. Má-li posloupnost hodnot stejnou třídu, vytvoř z ní interval s charakteristickou funkcí rovnou 1 na celém intervalu.
2. Pro každý interval Int_i patřící do třídy „?“:
 - patří-li sousední intervaly Int_{i-1} a Int_{i+1} do *téže* třídy, spoj je do jednoho intervalu $Int_{i-1} \cup Int_i \cup Int_{i+1}$ (charakteristická funkce = 1 na celém intervalu);
 - jinak vytvoř *dva fuzzy-intervaly*: spojením $Int_{i-1} \cup Int_i$ s charakteristickou funkcí rovnou 1 na $[L_{i-1}, U_{i-1}]$ a lineárně klesající $\frac{U_{i+1}-x}{U_{i+1}-L_{i-1}}$ na přechodové oblasti, a analogicky spojením $Int_i \cup Int_{i+1}$ s funkcí lineárně rostoucí $\frac{x-L_{i-1}}{U_{i+1}-L_{i-1}}$.
3. Zajisti spojitě pokrytí celého rozsahu (mezery mezi intervaly se chápou jako intervaly třídy „?“).

2.6.2 Seskupování hodnot kategoriálních atributů (KEX)

Má-li kategoriální atribut příliš mnoho hodnot, je užitečné je seskupit. **Cíl:** vytvořit takové grupy $A(Grp)$, aby $P(Class | A(Grp))$ významně lišilo od apriorního $P(Class)$ — a vytvořit jich tolik, kolik je tříd, plus jednu navíc („?“).

Algoritmus seskupování hodnot (KEX)

Hlavní cyklus:

1. vytvoř uspořádaný seznam hodnot uvažovaného atributu;
2. pro každou hodnotu: (a) spočítej četnosti výskytu objektů s touto hodnotou v jednotlivých třídách; (b) přiřaď hodnotě kód třídy procedurou *Assign*;
3. vytvoř grupy hodnot procedurou *Group*.

Procedura *Assign*: patří-li pro danou hodnotu všechny objekty do téže třídy, přiřaď hodnotě kód této třídy; jinak — liší-li se rozdělení tříd pro danou hodnotu významně (podle χ^2 testu) od apriorního rozdělení tříd — přiřaď kód nejčastější třídy, a pokud ne, přiřaď kód „?“.

Procedura *Group*: vytvoř grupu z hodnot, kterým byl přiřazen stejný kód třídy.

2.7 Redukce dat

2.7.1 Příliš mnoho objektů

Dávkové zpracování se stává problematickým. Řešení: (a) pracovat jen s **vzorkem** objektů; (b) uložit data tak, aby k nim šlo přistupovat bez nutnosti mít je celá v operační paměti; (c) postavit **více modelů nad podmnožinami** a poté je zkombinovat (ensemble, kap. 3).

Klíčová je **reprezentativnost** vybraných dat — vzorek má zachytit povahu dat co nejlépe, zejména co se týče souhlasu rozdělení hodnot atributů na vzorku a na celých datech.

Nevyvážené třídy v původních datech vedou k tendenci preferovat majoritní třídu. Řešení: různé váhy pro různé typy chyb (cenově citlivé učení) nebo výběr objektů z různých tříd s různou pravděpodobností (převzorkování minority, podvzorkování majority).

2.7.2 Příliš mnoho atributů

Dvě principiálně odlišné cesty:

- **Redukce transformací** — z existujících atributů vytvoříme *méně nových*, typicky jako lineární kombinace původních (analýza hlavních komponent, PCA). Nevýhody: je nutné znát hodnoty *všech* původních atributů a nové atributy nemusí mít jasnou interpretaci.
- **Redukce selekcí** (*feature selection*) — z existujících atributů vybereme jen ty nejdůležitější. Interpretace se zachovává a nepotřebné atributy se vůbec nemusí měřit.

♦ **nad rámec slidů** PCA hledá ortogonální směry maximálního rozptylu (vlastní vektory kovarianční matice s největšími vlastními čísly); je to *nesupervizovaná* transformace, takže může zahodit směr, který je pro predikci klíčový, ale má malý rozptyl.

2.8 Analýza relevance atributů: filtry a wrappery

Hledáme atributy, které nejlépe přispívají ke správné klasifikaci objektů.

Filtry vs. wrappery

Filtrační metody (*filter*) — pro každý atribut spočítáme charakteristiku odrážející jeho přínos ke klasifikaci, atributy podle ní seřadíme a vybereme určitý počet nejlepších. Charakteristika je *nezávislá* na následně použitém modelu; metoda je rychlá.

Obalové metody (*wrapper*) — pomocí algoritmu strojového učení se postaví model nad *podmnožinou* atributů a model se vyhodnotí; použije se nejlepší z postavených modelů (s odpo-

vídající podmnožinou atributů). Metoda je výpočetně drahá, ale zohledňuje interakci atributů s konkrétním modelem.

Prohledávání prostoru podmnožin:

- „zdola nahoru“ (*bottom-up, dopředná selekce*) — začneme od modelů nad jednotlivými atributy a postupně přidáváme další;
- „shora dolů“ (*top-down, zpětná eliminace*) — začneme od modelu nad úplnou množinou atributů a postupně odstraňujeme nepotřebné.

Zásadní omezení filtrů: každý atribut se hodnotí *samostatně*, takže vzájemný vliv několika atributů na přesnost klasifikace se zachycuje obtížně. Lze počítat kritérium pro *množinu* atributů (tj. pro všechny kombinace), ale kombinatorická explozivnost to prakticky omezuje na hladové dopředné či zpětné prohledávání. [♦ nad rámec slidů] Praktickým důsledkem je, že filtry propustí dvojici silně korelovaných atributů (oba dostanou vysoké skóre, přitom druhý nepřináší nic nového) a naopak zahodí atribut, který je užitečný jen v kombinaci s jiným (typicky XOR-vztah).

2.8.1 Kritéria na kontingenčních tabulkách

Vztah vstupního atributu A (hodnoty $v_1, \dots, v_R$) a cílového atributu C (třídy $v_1, \dots, v_S$) se popisuje kontingenční tabulkou o četnostech a_{kl} kombinace $A(v_k) \wedge C(v_l)$, s marginály

$$r_k = \sum_{l=1}^S a_{kl}, \quad s_l = \sum_{k=1}^R a_{kl}, \quad n = \sum_{k=1}^R \sum_{l=1}^S a_{kl}.$$

Odhady pravděpodobností: $P(A(v_k) \wedge C(v_l)) = a_{kl}/n$, $P(A(v_k)) = r_k/n$, $P(C(v_l)) = s_l/n$. Očekávaná četnost při nezávislosti je $e_{kl} = r_k s_l / n$.

χ^2 jako kritérium relevance

$$\chi^2 = \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - e_{kl})^2}{e_{kl}} = n \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - \frac{r_k s_l}{n})^2}{r_k s_l} \quad (\text{čím větší, tím lepší}).$$

Alternativní tvar pro ruční výpočet: $\chi^2 = \sum_k \sum_l \frac{a_{kl}^2}{e_{kl}} - n$.

Jako statistický test: při platnosti nulové hypotézy o nezávislosti

$$H_0: P(A = v_k \wedge C = v_l) = P(A = v_k) P(C = v_l) \quad \forall k, l$$

má statistika χ^2 rozdělení s $(R-1)(S-1)$ stupni volnosti. Je-li $\chi^2 \geq \chi_{(R-1)(S-1)}^2(\alpha)$, nulovou hypotézu na hladině významnosti α zamítáme a přijímáme alternativní hypotézu o *závislosti*.

Příklad — χ^2 test na čtyřpolní tabulce

		Poskytnutí úvěru		
		ANO	NE	r_k
Výše příjmu	vysoký	50	0	50
	nízký	30	40	70
s_l		80	40	120

Očekávané četnosti: $e_{11} = 50 \cdot 80 / 120 = 33,3$; $e_{12} = 50 \cdot 40 / 120 = 16,7$; $e_{21} = 70 \cdot 80 / 120 = 46,7$; $e_{22} = 70 \cdot 40 / 120 = 23,3$. Rozdíly jsou tedy $+16,7$, $-16,7$, $-16,7$, $+16,7$ a hodnota statistiky $\chi^2 = 42,857$.

Na hladině významnosti $\alpha = 0,05$ je kritická hodnota $\chi_{(1)}^2(0,05) = 3,84$. Protože $42,857 > 3,84$, **mezi výší příjmu a poskytnutím úvěru je závislost**.

Fisherův exaktní test

χ^2 test lze použít jen pro dostatečně velké četnosti — musí platit $(r_k s_l)/n \geq 5$ pro všechna k, l . Pro čtyřpolní tabulky s malými četnostmi se používá *Fisherův test*. Pravděpodobnost, že čtyřpolní tabulka má při daných marginálech r_1, r_2, s_1, s_2 právě četnosti a_{kl} :

$$p = \frac{r_1! r_2! s_1! s_2!}{n! a_{11}! a_{12}! a_{21}! a_{22}!} = \frac{\binom{s_1}{a_{11}} \binom{s_2}{a_{12}}}{\binom{n}{r_1}}.$$

Pravděpodobnosti se sečtou přes všechny uvažované hodnoty skutečných četností při daných marginálech (nechť $m = \min(r_1, s_1)$):

$$P = \sum_{i=0}^m \frac{r_1! r_2! s_1! s_2!}{n! (a_{11} - i)! (a_{12} + i)! (a_{21} + i)! (a_{22} - i)!}.$$

Je-li $P \leq \alpha$, nulová hypotéza se na hladině α zamítá.

Příklad. Pro tabulku s marginály $r = (12, 18)$, $s = (6, 24)$, $n = 30$ vychází $P_0 = 0,031$ a $P_1 = 0,173$, tedy $P = P_0 + P_1 = 0,204 > 0,05$ — **nulovou hypotézu o nezávislosti přijímáme**. (Kontrola: součet $P_0, \dots, P_6 = 0,031 + 0,173 + 0,340 + 0,302 + 0,128 + 0,024 + 0,002 = 1,000$.)

2.8.2 Kritéria založená na entropii

Entropie, vzájemná informace, informační míra závislosti

Entropie atributu A jako vážený průměr entropií jeho hodnot (čím *menší*, tím lepší):

$$H(A) = \sum_{k=1}^R \frac{r_k}{n} H(A(v_k)), \quad H(A(v_k)) = - \sum_{l=1}^S \frac{a_{kl}}{r_k} \log_2 \frac{a_{kl}}{r_k}.$$

Vzájemná informace (*mutual information*):

$$\text{MI}(A, C) = \sum_{k=1}^R \sum_{l=1}^S \frac{a_{kl}}{n} \log_2 \frac{n a_{kl}}{r_k s_l}.$$

Informační míra závislosti (*information measure of dependence*) — normalizovaná vzájemná informace (čím *větší*, tím lepší):

$$\text{ID}(A, C) = \frac{\text{MI}(A, C)}{H(C)} = \frac{\text{MI}(A, C)}{- \sum_{l=1}^S \frac{s_l}{n} \log_2 \frac{s_l}{n}}.$$

Normalizace v ID je podstatná: samotná MI systematicky zvýhodňuje atributy s mnoha hodnotami (v limitě je identifikátor objektu „nejinformativnější“ atribut). Totéž zvýhodnění řeší poměrný informační zisk v C4.5 (kap. 3).

2.9 Vztahy mezi atributy: regrese a korelace

Zatímco kontingenční tabulky popisují vztah *kategoriálních* atributů, u numerických se používá regresní a korelační analýza. *Korelační analýza* odpovídá na otázku, zda mezi dvěma numerickými veličinami *existuje* lineární vztah; *lineární regrese* určuje jeho *parametry*.

Důvody pro regresní analýzu: (1) měření výstupu je nákladné $\Rightarrow$ predikovat jej ze snadno dostupných vstupů; (2) vstupy jsou známy dříve než výstup $\Rightarrow$ odhad výstupu; (3) řízené vstupní hodnoty pomohou najít správné chování výstupu; (4) mezi vstupem a výstupem může být kauzální vztah, který chceme najít.

Lineární regrese jedné proměnné metodou nejmenších kvadrátů

Vzory $[x_1, y_1], \dots, [x_p, y_p]$, regresní rovnice $y = \beta x + \alpha$, kvadratická chyba

$$E = \sum_{i=1}^p \varepsilon_i^2 = \sum_{i=1}^p (y_i - \hat{y}_i)^2 = \sum_{i=1}^p (y_i - \beta x_i - \alpha)^2.$$

Položíme parciální derivace nule:

$$\frac{\partial E}{\partial \alpha} = -2 \sum_i (y_i - \beta x_i - \alpha) = 0, \quad \frac{\partial E}{\partial \beta} = -2 \sum_i (y_i - \beta x_i - \alpha) x_i = 0,$$

odkud po úpravě $\alpha = \bar{y} - \beta \bar{x}$ a

$$\beta = \frac{\sum_{i=1}^p (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^p (x_i - \bar{x})^2} \quad (\text{tj. výběrová kovariance dělená výběrovým rozptylem } x).$$

Predikce: $\hat{y} = \beta x + \alpha$.

Korelační koeficient

Posouzení „množství“ lineární závislosti:

$$\rho(x, y) = \frac{\sigma_{xy}}{\sqrt{\sigma_x^2 \sigma_y^2}} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}, \quad \sigma_{xy} = \frac{1}{n-1} \sum_i (x_i - \bar{x})(y_i - \bar{y}),$$

$$\sigma_x^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2, \quad \sigma_y^2 = \frac{1}{n-1} \sum_i (y_i - \bar{y})^2.$$

Platí $\rho \in [-1, 1]$; $|\rho| = 1$ znamená přesný lineární vztah, $\rho = 0$ žádný *lineární* vztah (nikoli žádnou závislost — ρ nezachytí nelineární vztahy).

Vícerozměrná lineární regrese. Předpokládáme lineární závislost vysvětlované veličiny Y na několika vysvětlujících veličinách $X_1, \dots, X_m$: $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m$, maticově $\hat{Y} = X\beta$, kde X je rozšířená matice vstupních vzorů (první sloupec jedničky) a $\beta = (\beta_0, \dots, \beta_m)$. Kvadratická odchylka $E = (Y - X\beta)^\top (Y - X\beta)$; z $\partial E / \partial \beta = 0$ dostáváme normální rovnice $X^\top X \beta = X^\top Y$, tedy

$$\beta = (X^\top X)^{-1} X^\top Y.$$

Pro složité praktické úlohy je tento výpočet nákladný $\Rightarrow$ používají se aproximativní (iterativní) řešení.

Nelineární regrese předpokládá složitější funkční závislost (kvadratickou, exponenciální, ...). Nej důležitějším případem je *logistická regrese*, která modeluje kategoriální (dvouhodnotový) výstup — protože jde současně o klasifikátor, je zpracována v kap. 3, odd. 3.2.5.

2.10 Shrnutí ke zkoušce

- **Cíl přípravy dat:** vybrat relevantní data, reprezentovat je pro zvolený algoritmus, výsledkem *jedna* tabulka objekty $\times$ atributy.
- **Chybějící hodnoty:** ignorovat objekt / nová hodnota „nevím“ / existující hodnota (modus, poměrně, libovolně) / doplnit modelem. Umět říct, kdy je která volba vhodná (absence jako informace $\rightarrow$ „nevím“).
- **Odlehle hodnoty:** $\text{IQR} = Q_3 - Q_1$, outlier mimo $[Q_1 - 1,5 \text{ IQR}, Q_3 + 1,5 \text{ IQR}]$. Umět spočítat kvantily z vzestupně seřazených dat (pravidlo $n/4$ a $3n/4$, zaokrouhlení nahoru) a nakreslit box plot.

- **Standardizace:** desítkové škálování, min-max do $[0, 1]$ a $[-1, 1]$, z-skóre podle střední absolutní odchylky (robustnější), std-skóre podle směrodatné odchylky. Motivace: bez ní vzdálenost ovládne atribut s největším rozsahem.
- **Diskretizace:** ekvidistantní, ekvifrekvenční, s využitím třídy (entropie, χ^2); **fuzzy diskretizace** — rozostřené hranice, jedné hodnotě odpovídají dvě kategorie se součtem charakteristických funkcí 1, váha fuzzy-objektu je součin $\prod_j \mu_j$; rizika: ztráta informace a kontradikce. **KEX** seskupuje hodnoty kategoriálního atributu podle χ^2 testu proti apriornímu rozdělení tříd (procedury *Assign* a *Group*).
- **Redukce dat:** příliš mnoho objektů $\rightarrow$ vzorkování (reprezentativnost!), efektivní uložení, více modelů nad podmnožinami; nevyvážené třídy $\rightarrow$ váhy chyb nebo různé pravděpodobnosti výběru. Příliš mnoho atributů $\rightarrow$ *transformace* (PCA: méně nových atributů jako lineární kombinace, ale nutné znát všechny původní a chybí interpretace) vs. *selekce* (podmnožina původních, interpretace zůstává).
- **Filtry vs. wrappery:** filtr počítá charakteristiku pro každý atribut samostatně a je nezávislý na modelu (rychlý, ale nezachytí interakce); wrapper staví model nad podmnožinou a vyhodnocuje jej (drahý, zohledňuje interakce). Prohledávání *bottom-up* (přidávání) nebo *top-down* (odebírání).
- **Filtrační kritéria:** χ^2 (větší = lepší), entropie $H(A)$ (menší = lepší), $MI(A, C)$, $ID(A, C) = MI/H(C)$ (větší = lepší). Umět zdůvodnit normalizaci ID: bez ní jsou zvýhodněny atributy s mnoha hodnotami.
- χ^2 **test:** $(R - 1)(S - 1)$ stupňů volnosti, podmínka $r_{kl}/n \geq 5$; pro malé četnosti **Fisherův exaktní test**. Umět projít výpočet na čtyřpolní tabulce (příklad s úvěrem: $\chi^2 = 42,857 > 3,84 \Rightarrow$ závislost).
- **Regrese a korelace:** metoda nejmenších kvadrátů, β jako kovariance dělená rozptylem, $\alpha = \bar{y} - \beta\bar{x}$; vícerozměrně $\beta = (X^T X)^{-1} X^T Y$; korelační koeficient $\rho = \sigma_{xy}/(\sigma_x \sigma_y)$ měří jen *lineární* závislost.

3 Metody pro dobývání znalostí

Nejobsáhlejší věta okruhu. Jmenuje tři skupiny metod, které pokrývají tři různé situace: *asociační pravidla* pro transakční data bez cílového atributu, *učení s učitelem* pro data s cílovým atributem a *klastrovou analýzu* pro metrická data bez cílového atributu. Kapitola je podle nich rozdělena na tři oddíly.

3.1 Asociační pravidla a frekventované vzory

3.1.1 Analýza nákupního košíku

Analýza nákupního košíku (*market basket analysis*, MBA) je archetypální úloha: „které položky se v košíku vyskytují společně?“ Výsledky se vyjadřují ve formě pravidel a lze je okamžitě použít — při plánování a rozvržení prodejny, nabídce kuponů a časově omezených slev, nebo „balíčkování“ produktů.

Položka (*item*) je nabízený produkt nebo služba, **transakce** obsahuje jednu nebo více položek. **Frekvenční tabulka** udává počet společných výskytů každých dvou položek v transakcích; na její diagonále je počet transakcí obsahujících danou položku.

Příklad — frekvenční tabulka

Transakce v obchodě s potravinami:

Zák.	Položky		chléb	máslo	mléko	káva
1	chléb, máslo	chléb	4	3	1	2
2	mléko, chléb, máslo	máslo	3	4	1	2
3	chléb, káva	mléko	1	1	1	0
4	chléb, máslo, káva	káva	2	2	0	3
5	káva, máslo					

Z tabulky je povaha prodeje vidět okamžitě: *chléb a máslo se kupují spolu, mléko se nikdy nekupuje s kávou*.

Pravidla mají formu IF *podmínka* THEN *závěr*. Asociační pravidla mají být:

- **snadno pochopitelná** — nalezený vztah lze snadno ověřit;
- **použitelná** — obsahují užitečnou informaci, která vede k dalším zásahům;

a naopak nemají být **triviální** (výsledek už každý zná) ani **nevysvětlitelná** (neexistuje pro ně vysvětlení a nevedou k akci).

3.1.2 Support, confidence a lift

Support, confidence, lift

Support (podpora) — jak často lze pravidlo použít:

$$\text{sup}(R) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet všech transakcí}} \cdot 100\%.$$

Confidence (spolehlivost) — jak spolehlivé jsou výsledky pravidla:

$$\text{conf}(R) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet transakcí obsahujících } i} \cdot 100\%.$$

Lift (zdvih) — kolikrát je lepší použít pravidlo pro predikci než jen předpokládat jeho výsledek:

$$\text{lift}(R) = \frac{p(i \wedge j)}{p(i)p(j)}.$$

Je-li $\text{lift} < 1$, je pravidlo pro predikci *horší než náhodná volba* a **negace závěru** může dát lepší pravidlo (IF *podmínka* THEN NOT *závěr*).

Příklad — support, confidence, lift

Nad daty z předchozího příkladu (5 transakcí):

Pravidlo 1: „kdo kupuje chléb, kupuje i máslo.“ $\text{sup} = 3/5 = 60\%$, $\text{conf} = 3/4 = 75\%$.

Pravidlo 2: „kdo kupuje kávu, kupuje i máslo.“ $\text{sup} = 2/5 = 40\%$, $\text{conf} = 2/3 = 66\%$.

Pravidlo 3: „kdo kupuje mléko, kupuje i máslo.“ $\text{sup} = 1/5 = 20\%$, $\text{conf} = 1/1 = 100\%$,
 $\text{lift} = \frac{1/5}{(1/5) \cdot (4/5)} = \frac{5}{4} = 1,25$.

Pravidlo 3 má *stoprocentní* spolehlivost, ale support jen 20 % — opírá se o jedinou transakci.

Lift 1,25 říká, že zdvih proti náhodě je jen malý (máslo kupuje 80 % zákazníků tak jako tak).

To je základní past: *vysoká confidence sama o sobě nestačí*.

3.1.3 Volba položek, taxonomie a virtuální položky

Transakční data bývají hůř kvalitní a vyžadují rozsáhlejší předzpracování; relevance položek se navíc mění v čase. Vhodná úroveň zpracování je kompromis mezi rostoucím počtem kombinací a přímou použitelností výsledků (konkrétní položky), při zachování dostatečně vysokého supportu.

Taxonomie je hierarchie kategorií. Zpočátku se používají obecnější položky, později se generují pravidla pro konkrétní položky, a to jen z transakcí, které je obsahují. Aby byly výsledky použitelné, mají se uvažované položky vyskytovat v přibližně stejném počtu transakcí: *vzácné položky se posunou výše* v taxonomii (tam se vyskytují relativně častěji) a *častější položky se nechají na nižších úrovních* (aby nejfrekventovanější položky nedominovaly pravidlům).

Virtuální položky přesahují rámec tradiční taxonomie: rozostřují hranici mezi typy produktů (např. značky — Calvin Klein) a mohou obsahovat informaci o transakci samotné — anonymní (den v týdnu, čas) nebo podepsanou (informace o zákazníkovi a jeho chování). Mohou ale způsobit *redundanci* v pravidlech: buď virtuální položka odpovídá právě jedné položce z taxonomie („IF produkt_Škoda THEN Škoda“), nebo se v pravidle vyskytne virtuální i obecná položka současně („IF produkt_Škoda AND malé auto THEN stan“ místo „IF Fabia THEN stan“).

Srovnání prodejen pomocí virtuálních položek: virtuální položka označí typ prodejny, kde transakce proběhla (neodpovídá žádnému produktu ani službě). Postup: (1) poskytnout transakční data z nových prodejen za určitou dobu; (2) poskytnout přibližně stejné množství dat z již existujících prodejen; (3) aplikovat MBA na oba soubory transakcí; (4) zaměřit se na pravidla obsahující virtuální položky.

Prořezávání podle minimálního supportu eliminuje málo frekventované položky — podniknuté akce se musí týkat dostatečného počtu transakcí. Dvě možnosti: eliminovat vzácné položky (a následně odpovídající pravidla), nebo použít taxonomii a vytvořit obecné položky (které musí splnit

stanovený prah). Proměnný minimální support vede ke *kaskádovému efektu*.

3.1.4 Algoritmus Apriori

Frekventovaná množina položek a Apriori princip

Frekventovaná množina položek (frequent itemset) je kombinace (konjunkce) kategorií, která na datech dosáhne předem stanovené frekvence — supportu *minsup*.

Vlastnost uzavřenosti dolů (downward closure property) — klíčová idea Apriori: *každá podmnožina frekventované množiny je také frekventovaná*. Ekvivalentně: je-li množina nefrekventovaná, jsou nefrekventované i všechny její nadmnožiny.

Při hledání kombinací délky k se proto využijí známé kombinace délky $k - 1$ — *prohledávání do šířky*. Aby byla kombinace délky k vygenerována, požaduje se, aby *všechny* její podkombinace délky $k - 1$ splňovaly požadavek na frekvenci.

Algoritmus APRIORI (R. Agrawal)

```

1:  $L_1 \leftarrow$  všechny kategorie, které dosahují alespoň požadované frekvence
2:  $k \leftarrow 2$ 
3: while  $L_{k-1} \neq \emptyset$  do
4:    $C_k \leftarrow$  APRIORI-GEN( $L_{k-1}$ ) ▷ generování kandidátů
5:    $L_k \leftarrow$  ty kombinace z  $C_k$ , které dosáhly alespoň požadované frekvence
6:    $k \leftarrow k + 1$ 
7: end while
```

Funkce APRIORI-GEN(L_{k-1}):

```

1: for každou dvojici kombinací  $Comb_p, Comb_q \in L_{k-1}$  do
2:   if  $Comb_p$  a  $Comb_q$  se shodují v  $k - 2$  kategoriích then
3:     přidej  $Comb_p \cup Comb_q$  do  $C_k$  ▷ krok spojení
4:   end if
5: end for
6: for každou kombinaci  $Comb \in C_k$  do ▷ krok prořezání
7:   if některá její podkombinace délky  $k - 1$  není v  $L_{k-1}$  then
8:     odstraň  $Comb$  z  $C_k$ 
9:   end if
10: end for
```

Generování pravidel. Po nalezení kombinací s vyhovující frekvencí se vytvoří asociační pravidla (platí *frekvence nadkombinace* $\leq$ *frekvence kombinace*). Každá kombinace $Comb$ se rozdělí na všechny možné dvojice podkombinací Ant a Suc tak, že

$$Suc = Comb \setminus Ant, \quad Ant \cap Suc = \emptyset, \quad Ant \cup Suc = Comb.$$

Pro pravidlo $Ant \Rightarrow Suc$ pak *support* odpovídá frekvenci kombinace $Comb$ a *confidence* poměru frekvencí kombinací $Comb$ a Ant .

Výhody MBA	Nevýhody MBA
jasné a pochopitelné výsledky — IF-THEN pravidla okamžitě použitelná	výpočetní náklady: prostor asociačních pravidel je exponenciální, $O(2^m)$ pro m položek
dobývání bez cílových výstupních hodnot (bez apriorní znalosti)	nutnost taxonomií a virtuálních položek
umí zpracovat data proměnné délky	obtížné určení vhodného počtu položek; položky by měly mít přibližně stejnou frekvenci
jednoduchý a snadno pochopitelný výpočet	<i>produkuje nadměrné množství pravidel</i> — tisíce, desetitisíce, miliony

3.1.5 Vícenásobný minimální support: MS-Apriori

Problém vzácných položek. Jediný *minsup* předpokládá, že všechny položky mají podobnou povahu a frekvenci. V praxi se ale některé položky vyskytují velmi často a jiné vzácně (kuchyňský robot vs. chléb a mléko). Důsledek:

- je-li *minsup* příliš **vysoký**, pravidla se vzácnými položkami se nenajdou;
- je-li příliš **nízký** (aby se našla pravidla s vzácnými i častými položkami), hrozí *kombinatorická explozivnost* — časté položky se pospojují všemi možnými způsoby.

Model vícenásobných minimálních supportů (MIS)

Minimální support pravidla se vyjádří pomocí *minimálních supportů položek* (*minimum item support*, MIS) těch položek, které v pravidle vystupují. Každá položka může mít vlastní $MIS(i)$; zadáním různých hodnot vyjádří uživatel různé požadavky na support pro různá pravidla.

Minsup pravidla $R : a_1, \dots, a_k \Rightarrow a_{k+1}, \dots, a_r$ je *nejnižší* hodnota MIS mezi jeho položkami: pravidlo splňuje minimální support, je-li jeho skutečný support $\geq \min(MIS(a_1), \dots, MIS(a_r))$.

Omezení rozdílu supportů: aby se v téže množině neobjevily velmi časté a velmi vzácné položky, požaduje se

$$\max_{i \in s} \{sup(i)\} - \min_{i \in s} \{sup(i)\} \leq \varphi,$$

kde $0 \leq \varphi \leq 1$ je uživatelem zadaný maximální rozdíl supportů (stejný pro všechny množiny).

Příklad — minsup pravidla podle MIS

Položky *chléb*, *obuv*, *oděvy* s $MIS(\text{chléb}) = 2\%$, $MIS(\text{obuv}) = 0,1\%$, $MIS(\text{oděvy}) = 0,2\%$.

R_1 : *oděvy* $\Rightarrow$ *chléb* [$sup = 0,15\%$] minsup **nesplňuje**, protože $0,15\% < MIS(\text{oděvy}) = 0,2\%$.

R_2 : *oděvy* $\Rightarrow$ *obuv* [$sup = 0,15\%$] minsup **splňuje**, protože $0,15\% \geq MIS(\text{obuv}) = 0,1\%$.

Uzavřenost dolů v novém modelu neplatí! MIS totiž dovoluje vyšší minimální supporty pro pravidla s jen frekventovanými položkami a nižší pro pravidla s méně frekventovanými. Příklad: $MIS(1) = 10\%$, $MIS(2) = 20\%$, $MIS(3) = 5\%$, $MIS(4) = 6\%$. Množina $\{1, 2\}$ se supportem 9% je nefrekventovaná, ale $\{1, 2, 3\}$ a $\{1, 2, 4\}$ frekventované být mohou — $\{1, 2\}$ tedy *nesmíme* zahodit.

Řešení: položky se seřadí *vzestupně podle hodnot MIS* a toto pořadí se pak používá ve všech dalších operacích algoritmu. Každá množina w má formu $\{w[1], \dots, w[k]\}$, kde $MIS(w[1]) \leq \dots \leq MIS(w[k])$.

Algoritmus MS-Apriori

```

1:  $M \leftarrow \text{SORT}(I, MS)$  ▷ setřídění položek podle  $MIS(i)$ 
2:  $L \leftarrow \text{INIT-PASS}(M, T)$  ▷ první průchod daty;  $L$  je množina semen
3:  $F_1 \leftarrow \{\{i\} \mid i \in L, i.\text{count}/n \geq MIS(i)\}$ 
4: for  $k = 2$ ;  $F_{k-1} \neq \emptyset$ ;  $k++$  do
5:   if  $k = 2$  then
6:      $C_k \leftarrow \text{LEVEL2-CANDIDATE-GEN}(L, \varphi)$ 
7:   else
8:      $C_k \leftarrow \text{MSCANDIDATE-GEN}(F_{k-1}, \varphi)$ 
9:   end if
10:  for každou transakci  $t \in T$  do
11:    for každého kandidáta  $c \in C_k$  do
12:      if  $c \subseteq t$  then
13:         $c.\text{count}++$ 
14:      end if
15:      if  $c \setminus \{c[1]\} \subseteq t$  then
16:         $c.\text{tailCount}++$  ▷ pro generování pravidel

```

```

17:         end if
18:     end for
19: end for
20:  $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{MIS}(c[1])\}$ 
21: end for
22: return  $F = \bigcup_k F_k$ 

```

První průchod daty (INIT-PASS): algoritmus zjistí support každé položky, pak v setříděném pořadí najde *první* položku i , která splní $\text{MIS}(i)$, a vloží ji do množiny semen L . Pro každou další položku j za i platí: je-li $j.\text{count}/n \geq \text{MIS}(i)$, patří j také do L .

Příklad: $\text{MIS}(1) = 10\%$, $\text{MIS}(2) = 20\%$, $\text{MIS}(3) = 5\%$, $\text{MIS}(4) = 6\%$; 100 transakcí, četnosti $\{3\} = 6$, $\{4\} = 3$, $\{1\} = 9$, $\{2\} = 25$. Pak $L = \{3, 1, 2\}$ a $F_1 = \{\{3\}, \{2\}\}$. Položka 4 není v L , protože $4.\text{count}/n < \text{MIS}(3) = 5\%$; $\{1\}$ není v F_1 , protože $1.\text{count}/n < \text{MIS}(1) = 10\%$.

Generování kandidátů úrovně 2 (LEVEL2-CANDIDATE-GEN): pro každou položku $l \in L$ v daném pořadí, splňuje-li $l.\text{count}/n \geq \text{MIS}(l)$, se pro každou položku h za l přidá kandidát $\{l, h\}$, pokud $h.\text{count}/n \geq \text{MIS}(l)$ a $|\text{sup}(h) - \text{sup}(l)| \leq \varphi$. Podstatné je, že se používá L , ne $F_1 \rightarrow F_1$ totiž neobsahuje položky, které splňují MIS nějaké dřívější položky, ale ne své vlastní (v příkladu položka 1).

Generování kandidátů vyšších úrovní (MSCANDIDATE-GEN) má dvě části. *Krok spojení* je jako v Apriori: spojí se $f_1 = \{i_1, \dots, i_{k-2}, i_{k-1}\}$ a $f_2 = \{i_1, \dots, i_{k-2}, i'_{k-1}\}$, které se liší jen v poslední položce, pokud $i_{k-1} < i'_{k-1}$ a $|\text{sup}(i_{k-1}) - \text{sup}(i'_{k-1})| \leq \varphi$. *Krok prořezání*: pro každou $(k-1)$ -podmnožinu s kandidáta c — není-li $s \in F_{k-1}$, lze c smazat, s **jedinou výjimkou**: neobsahuje-li s položku $c[1]$ (taková s je právě jedna), tedy položku s nejnižší hodnotou MIS, nesmíme c smazat — nevíme totiž, zda s nesplňuje $\text{MIS}(c[1])$, víme jen, že nesplňuje $\text{MIS}(c[2])$; to platí, jen pokud $\text{MIS}(c[2]) = \text{MIS}(c[1])$.

Proč tailCount. U Apriori platí: je-li f frekventovaná a $f_{\text{sub}} \subseteq f$, je i f_{sub} frekventovaná — k určení confidence pravidel tedy stačí supporty frekventovaných množin. U MS-Apriori to nestačí; problém vzniká, když by v závěru pravidla byla položka s *nejnižší* hodnotou MIS. Proto se zvlášť počítá $c.\text{tailCount}$ — support množiny c bez první položky.

Jak zadat hodnoty MIS: (a) podle skutečného supportu položky v datech, např. $\lambda \cdot \text{sup}(i)$ pro parametr $0 \leq \lambda \leq 1$ stejný pro všechny položky; (b) položky seskupit do bloků s podobnými frekvencemi a všem položkám z bloku dát stejnou hodnotu MIS. Nastavením $\text{MIS} = 100\%$ (nebo více) se generování pravidel obsahujících pouze tyto položky efektivně zakáže — praktický trik. Model vícenásobných minsupů *zahrnuje* model jediného supportu jako speciální případ.

3.1.6 Třídní asociační pravidla (CAR)

Běžné dolování asociačních pravidel necílí na žádný atribut — najde všechna pravidla, která v datech existují, a libovolná položka může být podmínkou i závěrem. V některých aplikacích však uživatele zajímají právě *cílové* atributy či třídy (např. které slovo je asociováno s kterým tématem dokumentu).

Třídní asociační pravidlo

Nechť T je množina n transakcí, každá navíc označovaná třídou y ; I je množina všech položek, Y množina všech tříd a $I \cap Y = \emptyset$.

Třídní asociační pravidlo (class association rule, CAR) je implikace $X \rightarrow y$, kde $X \subseteq I$ a $y \in Y$. Support a confidence jsou definovány stejně jako u běžných asociačních pravidel.

Příklad — CAR na textové kolekci

Dokument	Třída
1: Student, Učitel, Škola	Vzdělávání
2: Student, Škola	Vzdělávání
3: Učitel, Škola, Město, Hra	Vzdělávání
4: Baseball, Basketbal	Sport
5: Basketbal, Hráč, Divák	Sport
6: Baseball, Trenér, Hra, Tým	Sport
7: Basketbal, Tým, Město, Hra	Sport

Pro $\text{minsup} = 20\%$ a $\text{minconf} = 60\%$ jsou příklady CAR:

Student, Škola $\rightarrow$ Vzdelávání [$\text{sup} = 2/7$, $\text{conf} = 2/2$], Hra $\rightarrow$ Sport [$\text{sup} = 2/7$, $\text{conf} = 2/3$].

CAR lze dolovat **přímo v jednom kroku** (na rozdíl od běžných asociačních pravidel). Klíčovou operací je nalezení všech *pravidlových položek* (*rule-items*) tvaru $(\text{condset}, y)$, kde $\text{condset} \subseteq I$ a $y \in Y$, se supportem nad minsup ; každá pravidlová položka reprezentuje pravidlo $\text{condset} \rightarrow y$. Apriori lze na generování CAR snadno upravit.

Také zde lze použít ideu vícenásobných minimálních supportů: uživatel zadá *různé minimální supporty různým třídám* (tím efektivně různý minsup pravidlům každé třídy) — např. pravidla třídy *Ano* s $\text{minsup} 5\%$ a třídy *Ne* s $\text{minsup} 10\%$. Nastavením minimálního třídního supportu na 101% se generování pravidel pro danou třídu zakáže.

3.1.7 Dolování sekvenčních vzorů

Asociační pravidla neuvažují *pořadí* transakcí. V mnoha aplikacích je ale pořadí podstatné: v MBA je zajímavé vědět, zda lidé kupují položky v posloupnosti (nejprve postel, o nějakou dobu později povlečení); v dolování z webových logů (*web usage mining*) se hledají navigační vzory podle posloupností návštěv stránek.

Sekvence, velikost, délka, podsekvence

Nechť $I = \{i_1, \dots, i_m\}$ je množina položek. *Sekvence* je uspořádaný seznam množin položek. *Množina položek* (prvek sekvence) je neprázdná $X \subseteq I$; sekvenci s značíme $\langle a_1 a_2 \dots a_r \rangle$, kde a_i je prvek. Bez újmy na obecnosti předpokládáme položky v prvku v lexikografickém pořadí.

Velikost sekvence je počet jejích prvků, *délka* počet položek; sekvence délky k se nazývá k -sekvence.

$s_1 = \langle a_1 \dots a_r \rangle$ je *podsekvencí* $s_2 = \langle b_1 \dots b_v \rangle$ (a s_2 *nadsekvencí* s_1 , též s_2 *obsahuje* s_1), existují-li celá čísla $1 \leq j_1 < j_2 < \dots < j_r \leq v$ taková, že $a_1 \subseteq b_{j_1}, \dots, a_r \subseteq b_{j_r}$.

Příklad. Pro $I = \{1, \dots, 9\}$ je $\langle \{3\}\{4, 5\}\{8\} \rangle$ podsekvencí $\langle \{6\}\{3, 7\}\{9\}\{4, 5, 8\}\{3, 8\} \rangle$, protože $\{3\} \subseteq \{3, 7\}$, $\{4, 5\} \subseteq \{4, 5, 8\}$ a $\{8\} \subseteq \{3, 8\}$. Naopak $\langle \{3\}\{8\} \rangle$ *není* obsažena v $\langle \{3, 8\} \rangle$ ani naopak. Velikost $\langle \{3\}\{4, 5\}\{8\} \rangle$ je 3, délka 4.

Úloha: pro danou množinu S vstupních datových sekvencí (sekvenční databázi) najít všechny sekvence, které mají uživatelem zadaný minimální support; taková sekvence se nazývá *frekventovaná sekvence* nebo *sekvenční vzor*. Support sekvence je podíl datových sekvencí v S , které ji obsahují.

Algoritmus **GSP** (*Generalized Sequential Pattern*) generuje sekvenční vzory způsobem obdobným Apriori — prohledávání do šířky s generováním a testováním kandidátů, kde kandidáti délky k vznikají spojováním frekventovaných sekvencí délky $k - 1$.

Analýza časových řad pomocí MBA — analýza „příčina–následek“: zavedou se nové položky pro transakce, které nastaly *před* uvažovanou událostí (příčiny) a *po* ní (následky); duplicitní položky se z transakcí odstraní. *Okno* je snímek všech dat za určité období (např. všechny transakce z posledního měsíce) — slouží k zachycení trendů u vzácných položek. Může být nutná identifikace zákazníka.

Disociační pravidla tvaru „IF A AND NOT B THEN C “: zavedou se nové položky komplementární k původním (neobsahuje-li transakce původní položku, obsahuje její negaci). Nevýhody: dvojnásobný počet položek, zvětšená velikost transakcí a negované položky se vyskytují častěji než původní (pravidla se všemi položkami negovanými se obtížně využívají).

3.1.8 FP-Growth a FP-strom

Apriori používá přístup *generuj a testuj*: generování kandidátů je nákladné (prostorově i časově) a nákladné je i počítání supportu — kontrola podmnožin je výpočetně drahá a databáze se prochází *opakovaně* (I/O). **FP-Growth** umožňuje hledat frekventované množiny *bez generování kandidátů* ve dvou krocích: (1) postavit kompaktní datovou strukturu *FP-strom* (dvěma průchody daty); (2) extrahovat frekventované množiny přímo z FP-stromu jeho procházením.

FP-strom

Uzly FP-stromu odpovídají položkám a mají počítadlo. FP-Growth čte transakce po jedné a mapuje každou na *cestu* ve stromu. Používá se *fixní pořadí* položek, takže cesty se mohou překrývat, mají-li transakce stejný *prefix*; v takovém případě se počítadla inkrementují. Mezi uzly obsahujícími tutéž položku se udržují ukazatele, které tvoří jednosměrné spojové seznamy. Čím více cest se překrývá, tím vyšší je komprese a tím spíše se FP-strom vejde do paměti.

Konstrukce (2 průchody). *Průchod 1*: zjistí support každé položky, zahodí nefrekventované, seřadí frekventované *klesajícím* supportem (např. a, b, c, d, e) — toto pořadí se použije při stavbě stromu, aby se sdílely společné prefixy. *Průchod 2*: postav strom. Např. transakce $\{a, b\}$ vytvoří cestu $\text{null} \rightarrow a \rightarrow b$ s počítadly 1; transakce $\{b, c, d\}$ vytvoří *disjunktní* cestu $\text{null} \rightarrow b \rightarrow c \rightarrow d$ (nemá s první společný prefix), přidá se ukazatel mezi oba uzly b ; transakce $\{a, c, d, e\}$ má s první společný prefix a , takže cesty se překryjí a počítadlo uzlu a se inkrementuje.

Velikost FP-stromu. Obvykle menší než nekomprimovaná data — transakce typicky sdílejí položky, a tedy prefixy. *Nejlepší případ*: všechny transakce obsahují stejnou množinu položek $\Rightarrow$ jediná cesta. *Nejhorší případ*: každá transakce má unikátní množinu položek $\Rightarrow$ strom je alespoň tak velký jako původní data a nároky na uložení jsou *vyšší* (nutno uložit ukazatele a počítadla). Velikost závisí na pořadí položek — klesající support je jen *heuristika* a nevede vždy k nejmenšímu stromu.

Extrakce frekventovaných množin z FP-stromu

Algoritmus postupuje *zdola nahoru*, od listů ke kořeni, metodou *rozděl a panuj*: nejprve hledá frekventované množiny končící na e , pak na de , $\dots$, pak na d , cd atd.

1. **Prefixový podstrom** (*prefix path sub-tree*) končící danou položkou se získá pomocí spojových seznamů.
2. Ověř, zda je položka frekventovaná — sečti počítadla po spojovém seznamu. (V příkladu pro $\text{minSup} = 2$ vyjde $\text{count}(e) = 3$, takže $\{e\}$ je frekventovaná množina.)
3. Je-li položka frekventovaná, hledej rekurzivně frekventované množiny končící na de , ce , be , ae . K tomu je nutné získat *podmíněný FP-strom*.

Podmíněný FP-strom (*conditional FP-tree*) pro položku e je strom, který by vznikl, kdybychom uvažovali *jen transakce obsahující e* , a pak e ze všech transakcí odstranili. Získá se z prefixového podstromu ve třech krocích:

1. aktualizuj počítadla po prefixových cestách tak, aby odrážela počet transakcí obsahujících e ;
2. odstraň uzly obsahující e (informace o nich už není potřeba);
3. odstraň z prefixových cest nefrekventované položky — např. má-li b support 1 (což znamená, že be má support 1), je be nefrekventovaná a b lze odstranit.

Podmíněný strom se pak použije k nalezení množin končících na de , ce , ae (be se neuvažuje, protože b v podmíněném stromu není); pro každou z nich se opět získají prefixové cesty, extrahují frekventované množiny, vytvoří podmíněný strom — rekurzivně (např. $e \rightarrow de \rightarrow ade$).

Výhody FP-Growth	Nevýhody FP-Growth
jen 2 průchody daty „komprimuje“ data	FP-strom se nemusí vejít do paměti stavba FP-stromu je nákladná (ale poté se množiny čtou snadno)
žádné generování kandidátů, výrazně rychlejší než Apriori	prořezávat lze jen na úrovni jednotlivých položek; support lze spočítat teprve po vložení <i>všech</i> dat do stromu

♦ **nad rámec slidů** Doplnkově: *uzavřená* frekventovaná množina (*closed*) je taková, jejíž žádná vlastní nadmnožina nemá stejný support; *maximální* (*maximal*) je taková, jejíž žádná vlastní nadmnožina není frekventovaná. Maximální množiny jsou nejkompaktnější reprezentace (ztrácejí ale informaci o supportech podmnožin), uzavřené jsou kompromis — bezztrátová reprezentace všech supportů.

3.2 Přístupy založené na principu učení s učitelem

3.2.1 Formulace úlohy

Klasifikace a predikce

Je dána trénovací množina $\{(x_1, y_1), \dots, (x_p, y_p)\}$, kde x_i je vektor hodnot vstupních atributů a y_i hodnota *cílového* atributu. Úkolem je najít model $\hat{y} = f(x)$, který dobře předpovídá cílový atribut i pro dosud neviděné vstupy.

Je-li cílový atribut *diskrétní*, jde o **klasifikaci**, je-li *spojitý*, o **regresi** (predikci). Řešení má vždy dvě fáze: (1) **indukce** (učení, trénování) modelu z trénovacích dat; (2) **aplikace** modelu na nová data.

Zásadní je, že model se hodnotí na *nových* datech — schopnost model naučit trénovací data nazpaměť je bezcenná (*přeučení*, *overfitting*). Metody odhadu chyby a míry kvality jsou v kap. 5, odd. 5.2.

3.2.2 Rozhodovací stromy

Rozhodovací strom

Nechť D je databáze $\{d_1, \dots, d_p\}$, $\{A_1, \dots, A_n\}$ množina atributů a $C = \{C_1, \dots, C_m\}$ množina tříd. *Rozhodovací strom* pro D je strom, kde:

- každý vnitřní uzel je označen atributem A_a ;
- každá hrana je označena predikátem (aplikovatelným na atribut rodičovského uzlu);
- každý list má označení třídy C_j .

Strom dělí prostor příznaků na *obdélníkové oblasti*; vzory se klasifikují podle oblasti, do níž patří.

Výhody	Nevýhody
snadná implementace a interpretace	obtížné zpracování spojitých dat (nutná kategorizace → obdélníkové oblasti)
efektivnost	nelze vždy použít — hranice tříd může být diagonální a strom s osově zarovnanými řezy ji umí jen schodovitě aproximovat (v příkladu bankovního úvěru vede hranice napříč rovinou <i>zůstatek na kontě × příjem</i>)
extrakce jednoduchých pravidel	obtížné zpracování chybějících dat
použitelnost i na velké datové soubory	přeučení ⇒ nutné <i>prořezávání</i> neuvažují vzájemné korelace mezi atributy

Terminologie: *rozhodovací atributy* pro dělení datové množiny označují rozhodovací uzly stromu; *rozhodovací predikáty* označují hrany; *zastavovací kritérium* je např. „všechny vzory z redukované množiny patří do téže třídy“.

Algoritmus TDIDT (*Top Down Induction of Decision Trees*)

Indukce stromu shora dolů metodou *rozděl a panuj*:

1. Vyber jeden atribut jako *kořen* částečného (pod)stromu.
2. Rozděl data přicházející do tohoto uzlu na podmnožiny podle hodnot vybraného atributu a přidej uzel pro každou podmnožinu.
3. Existuje-li uzel takový, že vzory do něj přicházející nepatří všechny do téže třídy, zopakuj proces pro něj od kroku 1; jinak skonči.

Entropie a volba dělicího atributu

Cílem je vybrat takový atribut, který nejlépe rozdělí vzory z různých tříd. *Entropie* jako míra neuspořádanosti („překvapení“) v systému:

$$H = - \sum_{j=1}^m p_j \log_2 p_j,$$

kde p_j je pravděpodobnost výskytu třídy j (relativní frekvence na uvažované množině vzorů) a m počet tříd. Konvence: $0 \log_2 0 = 0$; přepočít $\log_2 x = \log_{10} x / 0,301$.

Pro atribut A : pro každou jeho možnou hodnotu v se spočítá entropie $H(A(v))$ na množině příkladů pokrytých kategorií $A(v)$,

$$H(A(v)) = - \sum_{j=1}^m p_j^{A(v)} \log_2 p_j^{A(v)},$$

a *vážená průměrná entropie* atributu jako vážený součet

$$H(A) = \sum_v \frac{|A(v)|}{|D|} H(A(v)).$$

Vybírá se atribut s *minimální* $H(A)$.

Příklad — žádost o úvěr

Klient	Příjem	Konto	Pohlaví	Nezaměstnaný	Úvěr
C1	vysoký	vysoké	žena	ne	ano
C2	vysoký	vysoké	muž	ne	ano
C3	nízký	nízké	muž	ne	ne
C4	nízký	vysoké	žena	ano	ano
C5	nízký	vysoké	muž	ano	ano
C6	nízký	nízké	žena	ano	ne
C7	vysoký	nízké	muž	ne	ano
C8	vysoký	nízké	žena	ano	ano
C9	nízký	střední	muž	ano	ne
C10	vysoký	střední	žena	ne	ano
C11	nízký	střední	žena	ano	ne
C12	nízký	střední	muž	ne	ano

Krok 1 — výběr atributu podle minimální entropie. Čtyřpolní tabulka pro *příjem* a *úvěr*: vysoký příjem 5 ano / 0 ne, nízký příjem 3 ano / 4 ne.

$$H(\text{příjem}(\text{vysoký})) = -\frac{5}{5} \log_2 \frac{5}{5} - \frac{0}{5} \log_2 \frac{0}{5} = 0,$$

$$H(\text{příjem}(\text{nízký})) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0,9852,$$

$$H(\text{příjem}) = \frac{5}{12} \cdot 0 + \frac{7}{12} \cdot 0,9852 = 0,5747.$$

Obdobně $H(\text{konto}) = \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 1 = 0,6667$, $H(\text{pohlaví}) = \frac{1}{2} \cdot 0,9183 + \frac{1}{2} \cdot 0,9183 = 0,9183$

a $H(\text{nezaměstnaný}) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0,65 = 0,825$. Minimum má **příjem** $\Rightarrow$ první větvení.

Krok 2. Příjem(vysoký): úvěr(ano) pro všechny vzory $\rightarrow$ list. Příjem(nízký): 7 klientů, další větvení; $H(\text{konto}) = \frac{2}{7}H(\dots) + \frac{3}{7}H(\dots) + \frac{2}{7}H(\dots) = 0,3935$, $H(\text{pohlaví}) = 0,965$, $H(\text{nezaměstnaný}) = 0,9792 \Rightarrow$ větvíme podle **konta**.

Krok 3. Konto(vysoké): ano pro všechny; konto(nízké): ne pro všechny; konto(střední): 3 klienti, $H(\text{pohlaví}) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot 0 = 0,6667$, $H(\text{nezaměstnaný}) = 0 \Rightarrow$ třetí větvení podle **nezaměstnanosti**.

Výsledný strom a převod na pravidla:

```
IF příjem(vysoký) THEN úvěr(ano)
IF příjem(nízký) ∧ konto(vysoké) THEN úvěr(ano)
IF příjem(nízký) ∧ konto(nízké) THEN úvěr(ne)
IF příjem(nízký) ∧ konto(střední) ∧ nezam.(ano) THEN úvěr(ne)
IF příjem(nízký) ∧ konto(střední) ∧ nezam.(ne) THEN úvěr(ano)
```

Převod stromu na pravidla obecně: každá cesta od kořene k listu odpovídá jednomu pravidlu; vnitřní uzly (s hodnotou pro odpovídající hranu) tvoří levou stranu (tělo), list (cílová třída) pravou stranu (hlavu).

Faktory důležité pro indukci stromu:

- *volba rozhodovacích atributů* — ovlivňuje efektivitu stromu; užitečný je „informovaný“ vstup experta pro danou oblast;
- *pořadí rozhodovacích atributů* — eliminuje redundantní porovnání;
- *větvení* — počet nutných dělení; náročnější pro spojitě hodnoty nebo mnoho hodnot;
- *forma stromu* — vhodnější jsou vyvážené stromy s co nejmenším počtem úrovní (za cenu složitějších porovnání); některé algoritmy tvoří pouze binární stromy (často hluboké);
- *zastavovací kritéria* — správná klasifikace trénovacích dat; *předčasné zastavení* zabraňuje budování velkých stromů a přeučení (přesnost vs. efektivita); *Occamova břitva* (Vilém Occam, 1320) — preference nejjednodušší hypotézy pro D ;
- *trénovací data a jejich množství* — příliš málo $\Rightarrow$ strom nemusí být dost spolehlivý pro obecnější data; příliš mnoho $\Rightarrow$ nebezpečí přeučení;
- *prořezávání (pruning)* — vyšší přesnost a efektivita na testovacích datech; odstranění redundantních porovnání, celých podstromů a irrelevantních atributů; lze provádět už během indukce (zabrání tvorbě nadměrně velkých stromů), nebo až na hotovém stromu;
- *časová a prostorová složitost* — závisí na počtu trénovacích dat p , počtu atributů n a formě stromu. Indukce stromu: $O(np \log p)$; klasifikace q vzorů stromem hloubky $O(\log p)$: $O(q \log p)$.

Algoritmus ID3 (*Iterative Dichotomiser*, Ross Quinlan, 1986): učení funkcí s booleovskými hodnotami, *hladová* metoda, staví strom shora dolů; v každém uzlu volí atribut, který nejlépe klasifikuje lokální trénovací data — nejlepší atribut má *největší informační zisk*. Proces pokračuje, dokud nejsou všechna trénovací data správně klasifikována, nebo dokud nejsou využity všechny atributy.

Algoritmus ID3 a informační zisk

ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES):

- 1: vytvoř kořen *ROOT*
- 2: **if** všechny *EXAMPLES* jsou pozitivní **then return** *ROOT* s jediným uzlem označeným „+“
- 3: **end if**
- 4: **if** všechny *EXAMPLES* jsou negativní **then return** *ROOT* s jediným uzlem označeným „-“
- 5: **end if**

```

6: if  $ATTRIBUTES = \emptyset$  then return  $ROOT$  s uzlem označeným nejčastější hodnotou  $TARGET\_ATTRIBUTE$  v  $EXAMPLES$ 
7: end if
8:  $A \leftarrow$  atribut z  $ATTRIBUTES$ , který nejlépe klasifikuje  $EXAMPLES$ 
9: rozhodovací atribut pro  $ROOT \leftarrow A$ 
10: for každou možnou hodnotu  $v_i$  atributu  $A$  do
11:   připoj k  $ROOT$  novou hranu odpovídající predikátu  $A = v_i$ 
12:    $EXAMPLES_{v_i} \leftarrow$  podmnožina  $EXAMPLES$  s hodnotou  $v_i$  pro  $A$ 
13:   if  $EXAMPLES_{v_i} = \emptyset$  then
14:     připoj k hraně list označený nejčastější hodnotou  $TARGET\_ATTRIBUTE$ 
15:   else
16:     připoj podstrom  $ID3(EXAMPLES_{v_i}, TARGET\_ATTRIBUTE, ATTRIBUTES \setminus \{A\})$ 
17:   end if
18: end for
19: return  $ROOT$ 

```

Informační zisk (*information gain*) — očekávané snížení entropie po rozdělení dat podle uvažovaného atributu:

$$GAIN(D, A) = H(D) - \sum_{v \in \text{VALUES}(A)} \frac{|D_v|}{|D|} H(D_v).$$

Protože $H(D)$ je pro všechny atributy stejná, je maximalizace informačního zisku ekvivalentní minimalizaci vážené entropie $H(A)$.

Algoritmus C4.5 (Quinlan, 1993) — modifikace ID3:

- **Chybějící data** — ignorována při konstrukci stromu (při vyhodnocení kritéria dělení se uvažují jen známé hodnoty daného atributu); při klasifikaci se chybějící hodnoty odhadují podle hodnot známých u jiných vzorů.
- **Spojitá data** — kategorizují se podle hodnot atributu na trénovacích vzorech.
- **Prořezávání validací** — trénovací data se rozdělí na trénovací ($\frac{2}{3}$) a validační ($\frac{1}{3}$) množinu. *Reduced-error pruning*: podstromy se nahrazují listy označenými nejčastější třídou příslušných trénovacích vzorů; nový strom *nesmí* být na validační množině horší než původní, jinak se podstrom nenahradí. *Subtree raising*: podstrom se nahradí svým nejčastěji používaným podstromem, který se zvedne na vyšší úroveň; testuje se klasifikační přesnost.
- **Prořezávání pravidel** (*rule post-pruning*) — ze trénovací množiny se odvodí strom (přeučení je dovoleno), převede se na ekvivalentní množinu pravidel (jedno pravidlo na cestu od kořene k listu), pravidla se prořezávají (zobecní) vypouštěním všech možných předpokladů, pokud to nezhorší odhadovanou klasifikační přesnost, a seřadí se podle očekávané přesnosti; v tomto pořadí se pak používají ke klasifikaci. Prořezávání je snazší a radikálnější než u celých stromů a výsledná pravidla jsou průhledná a snadno pochopitelná.
- **Odhad přesnosti pravidel** — standardně validační množinou; C4.5 hodnotí přesnost *na trénovacích datech* jako podíl vzorů správně klasifikovaných pravidlem ke všem vzorům pravidlem pokrytým, $acc = k/n$. Směrodatná odchylka přesnosti se odhaduje z binomického rozdělení; jako charakteristika pravidla se bere *dolní mez* přesnosti pro zvolený interval spolehlivosti, na 95% hladině

$$acc - 1,96 \sqrt{\frac{acc(1 - acc)}{n}}.$$

- **Poměrný informační zisk** (*information gain ratio*) jako kritérium dělení:

$$GAIN_RATIO(D, A) = \frac{GAIN(D, A)}{-\sum_v \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}}.$$

Dělí se podle atributu s *největším* poměrným informačním ziskem; jmenovatel penalizuje *nadměrné větvení* (atributy s mnoha hodnotami).

Algoritmus C5.0 — komerční verze C4.5 pro velké databáze; podobná indukce stromu, zlepšené (efektivnější) generování pravidel a vyšší přesnost díky *boostingu*: z původních trénovacích dat se staví různé trénovací podmnožiny, každému trénovacímu vzoru se přiřadí váha odpovídající jeho vlivu při klasifikaci, pro každou kombinaci vah se vytvoří samostatný klasifikátor a při klasifikaci má každý klasifikátor svůj hlas — vítězí většinová třída.

CART — Classification And Regression Trees

Generuje *binární* stromy — při každém dělení dat se tvoří právě dvě hrany. Volba nejlepšího rozhodovacího atributu podle entropie nebo **Gini indexu**: pro atribut o r hodnotách $v_1, \dots, v_r$, n vzorů a m tříd

$$G = \sum_{i=1}^r \frac{n_i}{n} G(v_i), \quad G(v_i) = 1 - \sum_{j=1}^m p_j^2, \quad \sum_{i=1}^r n_i = n,$$

kde n_i je počet vzorů s hodnotou v_i a p_j pravděpodobnost třídy j v těchto n_i vzorech. Nižší hodnoty G znamenají lepší diskriminaci.

Dělení se provádí podle „nejlepšího“ nalezeného bodu pro daný uzel t ; procházejí se všechny možné hodnoty v rozhodovacího atributu A (při rovnosti se použije pravý podstrom). Označme P_L, P_R pravděpodobnosti zpracování levým, resp. pravým podstromem a $P(C_j | t_L), P(C_j | t_R)$ pravděpodobnosti, že vzor patří do třídy C_j a je v levém, resp. pravém podstromu uzlu t . Kritérium dělení:

$$\Phi = 2P_L P_R \sum_{j=1}^m |P(C_j | t_L) - P(C_j | t_R)|,$$

které nabývá maxima, jsou-li všechny vzory dané třídy seskupeny v témže podstromu.

Charakteristické vlastnosti CART: pořadí atributů odpovídá jejich vlivu při klasifikaci; chybějící data jsou ignorována a nezapočítávají se do vyhodnocení rozhodovacích kritérií; zastavovací kritérium — žádné další dělení už nezlepší klasifikační přesnost; vysoká přesnost na trénovací množině nemusí odrážet přesnost na testovacích datech.

CHAID — Chi-square Automatic Interaction Detection

Kritériem větvení je χ^2 . Algoritmus automaticky *seskupuje hodnoty* kategoriálních atributů — hodnoty se postupně slučují od původního počtu, dokud nezůstanou jen dvě grupy; následně se vybere atribut a jeho kategorizace, které jsou pro dělení dat v daném kroku nejlepší. Při větvení tedy nevzniká tolik větví, kolik má atribut hodnot.

Seskupování hodnot atributu:

1. Opakuj, dokud nezůstanou jen dvě grupy hodnot: (a) vyber pár kategorií atributu, které jsou si z hlediska χ^2 nejvíce podobné a lze je sloučit; (b) novou kategorií uvažuj jako možné seskupení v daném kroku.
2. Pro každou možnou grupu hodnot testuj χ^2 testem její relevanci k predikované proměnné.
3. Vyber seskupení, které dává „nejlepší“ rozdělení s nejvýznamnějším rozštěpením.
4. Urči, zda toto seskupení statisticky významně přispívá k rozlišení vzorů z různých tříd.

3.2.3 Bayesovská klasifikace

Bayesova formule a hypotéza MAP

Pravděpodobnost, že hypotéza H platí, pozorujeme-li E :

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)},$$

kde $P(H)$ je *apriorní* pravděpodobnost hypotézy, $P(H | E)$ *aposteriorní* (podmíněná) pravděpodobnost a $P(E)$ pravděpodobnost jevu E .

Nejpravděpodobnější hypotéza H_{MAP} (*maximum a posteriori*) má největší aposteriorní pravděpodobnost. Protože jmenovatel $P(E)$ je pro všechny hypotézy stejný, lze jej zanedbat:

$$H_{MAP} = \arg \max_{H \in \mathcal{H}} P(H | E) = \arg \max_{H \in \mathcal{H}} P(E | H) P(H).$$

Naivní bayesovský klasifikátor

Předpoklad: jevy $E_1, \dots, E_k$ jsou *podmíněně nezávislé* při platnosti hypotézy H . V reálných úlohách je tento předpoklad splněn jen zřídka — odtud označení „naivní“. Pak

$$P(H | E_1, \dots, E_k) = \frac{P(E_1, \dots, E_k | H) P(H)}{P(E_1, \dots, E_k)} = \frac{P(H)}{P(E_1, \dots, E_k)} \prod_{i=1}^k P(E_i | H),$$

a klasifikujeme hypotézou s největší aposteriorní pravděpodobností

$$H_{MAP} = \arg \max_{H_t} P(H_t) \prod_{i=1}^k P(E_i | H_t).$$

Odhady z dat: $P(H_t) = P(\text{třída } t) = \frac{\text{počet vzorů z } t}{\text{počet všech vzorů}}$ a $P(E_k | H_t) = \frac{\text{počet vzorů z } t \text{ splňujících } E_k}{\text{počet vzorů z } t}$.

Výhoda: možnost klasifikovat i *nekompletně popsané* vzory. **Nevýhoda:** nulová aposteriorní pravděpodobnost $P(E_k | H_t)$, není-li v trénovací množině žádný odpovídající vzor, resp. podhodnocené členy při nízkém společném výskytu E_k a H_t .

Příklad — bayesovská klasifikace úvěru

Jednoduchý případ: $P(\text{půjčit}) = 0,667$, $P(\text{zamítnout}) = 0,333$, $P(\text{vysoký příjem} | \text{půjčit}) = 0,91$, $P(\text{vysoký příjem} | \text{zamítnout}) = 0,12$. Pak $0,91 \cdot 0,667 = 0,607$ vs. $0,12 \cdot 0,333 = 0,040$, tedy $H_{MAP} = \text{půjčit}$.

Naivní klasifikátor nad daty z příkladu o úvěru (12 klientů): $P(\text{úvěr(ano)}) = 8/12 = 0,667$, $P(\text{úvěr(ne)}) = 4/12 = 0,333$; $P(\text{konto(střední)} | \text{ano}) = 2/8 = 0,25$, $P(\text{konto(střední)} | \text{ne}) = 2/4 = 0,5$; $P(\text{nezam.(ne)} | \text{ano}) = 5/8 = 0,625$, $P(\text{nezam.(ne)} | \text{ne}) = 1/4 = 0,25$.

Pro klienta se středním kontem, který není nezaměstnaný:

$$0,667 \cdot 0,25 \cdot 0,625 = 0,1042 \quad \text{vs.} \quad 0,333 \cdot 0,5 \cdot 0,25 = 0,0416,$$

klient se tedy zařadí do třídy **úvěr(ano)**.

3.2.4 Support Vector Machines

SVM vynalezl V. Vapnik se spolupracovníky v 70. letech v Rusku; na Západě se metoda stala známou v roce 1992. Jde o *lineární* klasifikátory hledající hyperrovinu, která oddělí dvě třídy (pozitivní a negativní); pro nelineární separaci se používají *kernelové* funkce. SVM má nejen důkladný teoretický základ, ale i vyšší přesnost než většina jiných metod v aplikacích, zejména pro vysokorozměrná data — patří k nejlepším klasifikátorům pro klasifikaci textu.

Lineární SVM, separabilní případ

Trénovací množina $D = \{(x_1, y_1), \dots, (x_r, y_r)\}$, $x_i \in X \subseteq \mathbb{R}^n$, $y_i \in \{1, -1\}$. SVM hledá lineární funkci $f(x) = \langle w \cdot x \rangle + b$ tak, že

$$y_i = \begin{cases} 1 & \text{pro } \langle w \cdot x_i \rangle + b \geq 0, \\ -1 & \text{pro } \langle w \cdot x_i \rangle + b < 0. \end{cases}$$

Hyperrovina $\langle w \cdot x \rangle + b = 0$ se nazývá *rozhodovací hranice* (*decision boundary*). Takových hyperrovin je mnoho — SVM hledá tu s *největším okrajem* (*margin*); teorie strojového učení říká, že tato hyperrovina minimalizuje mez chyby.

Vzdálenost bodu x_i od hyperroviny je $\frac{|\langle w \cdot x_i \rangle + b|}{\|w\|}$, kde $\|w\| = \sqrt{\langle w \cdot w \rangle}$. Pro okrajové hyperroviny H_+ a H_- platí

$$d_+ = d_- = \frac{1}{\|w\|}, \quad \text{margin} = d_+ + d_- = \frac{2}{\|w\|}.$$

Optimalizační úloha:

$$\text{minimalizuj } \frac{\langle w \cdot w \rangle}{2} \quad \text{za podmíněk } y_i(\langle w \cdot x_i \rangle + b) \geq 1, \quad i = 1, \dots, r,$$

což je souhrnný zápis podmíněk $\langle w \cdot x_i \rangle + b \geq 1$ pro $y_i = 1$ a $\langle w \cdot x_i \rangle + b \leq -1$ pro $y_i = -1$.

Duální formulace a podpůrné vektory

Úloha se řeší standardní *Lagrangeovou* metodou:

$$L_P = \frac{\langle w \cdot w \rangle}{2} - \sum_{i=1}^r \alpha_i [y_i(\langle w \cdot x_i \rangle + b) - 1], \quad \alpha_i \geq 0.$$

Optimální řešení musí splňovat *Kuhn–Tuckerovy podmínky*, které jsou pro *konvexní* účelovou funkci s *lineárními* omezeními nejen nutné, ale i postačující. Podmínka komplementarity ukazuje, že $\alpha_i > 0$ mohou mít jen body *na* okrajových hyperrovinách (pro ně $y_i(\langle w \cdot x_i \rangle + b) - 1 = 0$) — to jsou **podpůrné vektory** (*support vectors*); pro všechny ostatní je $\alpha_i = 0$.

Vynulováním parciálních derivací L_P podle w a b a substitucí zpět dostaneme *Wolfeho duální* úlohu

$$L_D = \sum_{i=1}^r \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle x_i \cdot x_j \rangle \rightarrow \max,$$

jejíž maximum se nabývá pro tytéž hodnoty w, b, α_i jako minimum L_P . Duální úloha se řeší snáz než primární (numerické techniky).

Rozhodovací hranice a testování vzoru z :

$$\langle w \cdot x \rangle + b = \sum_{i \in sv} y_i \alpha_i \langle x_i \cdot x \rangle + b = 0, \quad \text{sign} \left(\sum_{i \in sv} \alpha_i y_i \langle x_i \cdot z \rangle + b \right).$$

Vrátí-li výraz 1, je z klasifikován jako pozitivní, jinak jako negativní.

Kernelový trik

Reálná data mohou vyžadovat nelineární separaci. Data se proto *transformují* do jiného (obvykle mnohem vícerozměrného) prostoru, v němž *lineární* hranice třídy oddělí: $\phi : X \rightarrow F$, $x \mapsto \phi(x)$; transformovaný prostor F se nazývá *prostor příznaků* (*feature space*), původní *vstupní prostor*.

Problém explicitní transformace: počet dimenzí prostoru příznaků může být pro užitečné transformace enormní i při rozumném počtu vstupních atributů — *prokletí dimenzionality* činí

explicitní výpočet nezvládnutelným.

Klíčové upozornění: v duální formulaci vyžadují jak konstrukce optimální hyperroviny, tak vyhodnocení rozhodovací funkce *pouze skalární součiny* $\langle \phi(x) \cdot \phi(z) \rangle$, nikdy transformovaný vektor $\phi(x)$ samotný. Umíme-li tedy počítat skalární součin přímo ze vstupních vektorů, nemusíme znát ϕ vůbec — *kernelová funkce*

$$K(x, z) = \langle \phi(x) \cdot \phi(z) \rangle.$$

Ve všech výrazech se skalární součin nahradí kernelem — **kernelový trik**.

Příklad (polynomiální kernel $d = 2$, 2-D vstup):

$$\begin{aligned} \langle x \cdot z \rangle^2 &= (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 z_1 x_2 z_2 + x_2^2 z_2^2 \\ &= \langle (x_1^2, x_2^2, \sqrt{2}x_1 x_2) \cdot (z_1^2, z_2^2, \sqrt{2}z_1 z_2) \rangle = \langle \phi(x) \cdot \phi(z) \rangle. \end{aligned}$$

Kernel $\langle x \cdot z \rangle^d$ je tedy skalární součin v transformovaném prostoru, aniž bychom ϕ kdy vyčíslili. Zda je daná funkce kernelem (tj. skalárním součinem v nějakém prostoru příznaků), odpovídá *Mercerova věta*. Běžné kernely: lineární (identita), polynomiální, gaussovský (RBF), sigmoidální.

Další vlastnosti SVM. Pracuje jen v reálném prostoru — kategoriální atributy je nutné převést na numerické. Provádí jen *dvoutřídní* klasifikaci; pro více tříd se používají strategie *jeden proti ostatním* (*one-against-rest*) nebo kódy opravující chyby (*error-correcting output coding*). Hyperrovina produkovaná SVM je pro člověka obtížně pochopitelná a kernely to ještě zhoršují — SVM se proto používá v aplikacích, kde se srozumitelnost pro člověka nevyžaduje. ♦ nad rámec slidů Pro nelineární separabilní data se zavádí *proměnné vůle* (*slack variables*) $\xi_i \geq 0$ a minimalizuje se $\frac{\langle w \cdot w \rangle}{2} + C \sum_i \xi_i$; parametr C řídí kompromis mezi šířkou okraje a počtem chyb.

3.2.5 Logistická regrese

Logistická regrese je nejdůležitější případ nelineární regrese (odd. 2) a zároveň klasifikátor: závislá proměnná y je kategoriální, typicky dvouhodnotová, a modeluje se *pravděpodobnost*, že y nabude určité hodnoty, v závislosti na kombinaci hodnot nezávislých veličin.

Logistická regrese

Pro binární $y \in \{0, 1\}$ se modeluje logaritmus „podmíněné šance“ (*conditional odds*):

$$\ln \frac{P(y = 1 \mid x_1, \dots, x_m)}{1 - P(y = 1 \mid x_1, \dots, x_m)} = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m = z,$$

odkud

$$P(y = 1 \mid x) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z) \quad (\text{sigmoida}).$$

Učení maximalizací věrohodnosti (*maximum likelihood*):

$$L = \prod_{i=1}^p P(y_i \mid x_{i,1}, \dots, x_{i,m}) = \prod_{i=1}^p \left(\frac{1}{1 + e^{-z_i}} \right)^{y_i} \left(\frac{1}{1 + e^{z_i}} \right)^{1-y_i}.$$

Místo maximalizace L se minimalizuje *křížová entropie* $E = -\log L$:

$$E = - \sum_i \left[y_i \log \sigma(z_i) + (1 - y_i) \log (1 - \sigma(z_i)) \right].$$

Protože derivace sigmoidy je $\sigma' = \sigma(1 - \sigma)$, dostaneme po zjednodušení velmi jednoduchá adaptační pravidla

$$\Delta \beta_j = -\eta \frac{\partial E}{\partial \beta_j} = \eta \sum_i (y_i - \sigma(z_i)) x_{i,j}, \quad \Delta \beta_0 = \eta \sum_i (y_i - \sigma(z_i)),$$

kde η je rychlost učení. Chyba tedy vstupuje do úpravy vah přímo jako rozdíl *pozorované* a *predikované* hodnoty.

Logistická regrese je běžnou volbou základního klasifikátoru tam, kde se požaduje pravděpodobnostní výstup a interpretovatelné koeficienty — např. při predikci vazeb v sociálních sítích (kap. 6).

3.2.6 Diskriminační analýza

Diskriminační analýza

Klasifikace vzorů do známých tříd: hledá se závislost nominální veličiny (určující příslušnost ke třídě) na ostatních numerických veličinách. Předpokládáme, že ke každé třídě C_j , $j = 1, \dots, m$ existuje *diskriminační funkce* g_j ; vzor $x = (x_1, \dots, x_m)$ patří do třídy C_j , právě když

$$g_j(x) = \max_k g_k(x).$$

Lineární diskriminační analýza: $g_j(x) = \beta_{j0} + \beta_{j1}x_1 + \dots + \beta_{jm}x_m$. Pro dvě třídy lze místo g_1, g_2 hledat jedinou funkci $g = g_1 - g_2$ a klasifikovat podle *znaménka*: „+“ znamená třídu 1, „-“ třídu 2.

Optimální klasifikace ve smyslu minimální chyby nastane, jsou-li diskriminační funkce $\propto$ aposteriorní (podmíněné) pravděpodobnosti zařazení pozorování x do třídy C_j : $g_j(x) = P(C_j | x) = \frac{p(x | C_j)P(C_j)}{p(x)}$; pro dvě třídy $g(x) = p(x | C_1)P(C_1) - p(x | C_2)P(C_2)$.

Pro *normální rozdělení* s různými středními hodnotami μ_1, μ_2 a obecnými kovariančními maticemi S_1, S_2 vychází *kvadratická* diskriminační funkce. Jsou-li kovarianční matice *shodné* ($S_1 = S_2 = S$), redukuje se na *lineární*

$$g(x) = (\mu_1 - \mu_2)^\top S^{-1}x - \frac{1}{2}(\mu_1 - \mu_2)^\top S^{-1}(\mu_1 + \mu_2) - \ln \frac{P(C_2)}{P(C_1)},$$

a jsou-li navíc obě třídy stejně pravděpodobné a $S = I$, jde o jednoduché srovnávání vzdáleností od středních hodnot. Pro normální rozdělení se tedy hledání diskriminačních funkcí redukuje na **odhad středních hodnot** (výběrovými průměry) a **kovariančních matic** (korigovanými výběrovými rozptyly).

3.2.7 Neuronové sítě a extrémní učicí stroje (ELM)

Síť ELM (*Extreme Learning Machine*, G.-B. Huang)

Dopředná síť s *jednou* skrytou vrstvou: n vstupních neuronů, L skrytých, 1 výstupní. Výstup skrytých neuronů $h(x) = [G(a_1, b_1, x), \dots, G(a_L, b_L, x)]$, kde a_i je váhový vektor a b_i bias i -tého neuronu; např. $G(a_i, b_i, x) = g(\langle a_i \cdot x \rangle + b_i)$ (sigmoida) nebo $G(a_i, b_i, x) = g(b_i \|x - a_i\|)$ (RBF). Výstup sítě $f_L(x) = \sum_{i=1}^L \beta_i G(a_i, b_i, x)$.

Klíčová myšlenka: lze-li spojitou funkci f aproximovat dopřednou sítí se skrytou vrstvou neuronů počítajících G , *není nutné parametry skrytých neuronů adaptovat* — mohou být vygenerovány *náhodně*, dokonce bez znalosti trénovacích dat. Pro každou spojitou f a náhodně generovanou posloupnost $\{(a_i, b_i)\}$ platí s pravděpodobností 1

$$\lim_{L \rightarrow \infty} \left\| f - \sum_{i=1}^L \beta_i G(a_i, b_i, \cdot) \right\| = 0$$

při nastavení β_i tak, aby minimalizovaly $\|f - f_L\|$.

Maticový model: soustava $\sum_{i=1}^L \beta_i G(a_i, b_i, x_j) = t_j$, $j = 1, \dots, N$ je ekvivalentní $H\beta = T$, kde H je *výstupní matice skryté vrstvy* (j -tý řádek = výstupy skrytých neuronů pro vzor x_j , i -tý sloupec = výstupy i -tého neuronu pro všechny vzory).

Algoritmus učení (tři neiterativní kroky):

- 1: zvol *náhodně* parametry skrytých neuronů (a_i, b_i) , $i = 1, \dots, L$
- 2: spočítej výstupní matici skryté vrstvy H
- 3: spočítej výstupní váhy $\beta = H^+ T$

kde H^+ je Mooreova–Penroseova pseudoinverze H . Stabilnější řešení s lepší generalizací dá regularizace — přičtení I/C na diagonálu $H^T H$: $f(x) = h(x)(I/C + H^T H)^{-1} H^T T$.

Výhody ELM: jednoduchá neiterativní procedura ze tří kroků (tradiční algoritmy jsou podstatně složitější); velmi rychlé a přímočaré trénování — není nutné řešit lokální minima, vhodnou inicializaci parametrů, přeučení apod.; parametry skrytých neuronů a_i, b_i jsou nezávislé na trénovacích datech i na sobě navzájem, takže je lze vygenerovat *už před* předložením trénovacích dat (konvenční techniky musí data „vidět“); použitelnost s *libovolnou* omezenou po částech spojitou nekonstantní přenosovou funkcí (gradientní algoritmy vyžadují hladké funkce).

3.2.8 Ensemble metody

Bagging (*bootstrapped aggregating*)

Snaží se snížit *rozptyl* klasifikátoru: je-li rozptyl klasifikátoru σ^2 , pak rozptyl průměru k nezávislých a stejně rozdělených (i.i.d.) klasifikátorů klesne na σ^2/k .

Bootstrapping aproximuje i.i.d. klasifikátory: body se vybírají s *opakováním* uniformně z původních dat; vzorek má přibližně stejnou velikost jako původní data, může obsahovat duplikáty a obsahuje podíl

$$1 - \left(1 - \frac{1}{n}\right)^n \approx 1 - \frac{1}{e} = 63,2\%$$

různých bodů z původních dat (pravděpodobnost, že bod ve vzorku *není*, je $(1 - 1/n)^n$). Nakreslí se k nezávislých bootstrapových vzorků velikosti n , na každém se natrénuje klasifikátor a pro testovací vzor se predikce určí *většinovým hlasem*.

Vhodnost: rozhodovací stromy jsou pro bagging ideální — mají *nízké zkreslení* a *vysoký rozptyl* (jsou-li dostatečně hluboké). **Omezení:** bagging *nesnižuje* zkreslení modelu; je-li zkreslení hlavním zdrojem chyby, může přesnost i mírně zhoršit kvůli korelacím mezi komponentami (a tedy porušenému i.i.d. předpokladu).

Náhodné lesy (*random forests*)

Mají-li k klasifikátorů s rozptylem σ^2 vzájemnou *pozitivní korelaci* ρ , je rozptyl zprůměrované predikce

$$\rho\sigma^2 + \frac{(1 - \rho)\sigma^2}{k}.$$

Člen $\rho\sigma^2$ je *nezávislý* na počtu komponent k — právě on omezuje přínos baggingu. Bootstrapované stromy přitom bývají pozitivně korelované.

Náhodný les je ensemble rozhodovacích stromů, do jejichž *konstrukce* je explicitně vložena náhodnost, aby byly komponenty méně korelované. Dvě cesty: (1) **bagging** — stromy se trénují na mírně odlišných datech (bootstrapové vzorky); (2) **omezený počet příznaků** — v každém uzlu se stromu předloží *náhodná podmnožina* m příznaků a informační zisk se počítá jen na ní. Druhá cesta navíc zrychluje trénování (méně příznaků k prohledání) a stromy není nutné prořezávat; snížený rozptyl přitom nezhoršuje zkreslení. Výstup lesa je většinový hlas stromů.

Algoritmus: pro každý z N stromů: vytvoř nový bootstrapový vzorek trénovací množiny; nad ním trénuj rozhodovací strom; v každém uzlu vyber náhodně m příznaků, spočítej informační zisk jen na nich a vyber optimální; opakuj, dokud není strom hotový.

Parametry: počet příznaků m — běžná volba je $\sqrt{\text{počet příznaků}}$ (náhodné lesy na tomto parametru nejsou příliš citlivé); počet stromů — lze pokračovat, dokud chyba klesá.

Boosting a AdaBoost (Freund & Schapire, 1996)

Princip: každému trénovacímu vzoru je přiřazena *váha* a různé klasifikátory se trénují s těmito vahami; váhy se iterativně upravují podle výkonu klasifikátoru. Budoucí modely tedy *závisí na předchozích* — každý klasifikátor se staví tímtož algoritmem, ale na jinak váženém trénovacím souboru. **Idea:** v dalších iteracích se soustředit na *nesprávně klasifikované* vzory zvýšením jejich relativní váhy; iterativním opakováním a vytvořením váženého kombinace klasifikátorů lze získat klasifikátor s nižším celkovým *zkreslením* (na rozdíl od baggingu, který snižuje rozptyl).

ADABOOST(data D , bazový klasifikátor A , max. počet kol T):

- 1: $t \leftarrow 0$; pro každý vzor i inicializuj $W_1(i) = 1/n$
- 2: **repeat**
- 3: $t \leftarrow t + 1$
- 4: urči váženou chybu ϵ_t na D , aplikuje-li se A na data s vahami $W_t(\cdot)$
- 5: $\alpha_t \leftarrow \frac{1}{2} \ln((1 - \epsilon_t)/\epsilon_t)$
- 6: **for** každý *nesprávně* klasifikovaný $X_i \in D$ **do**
- 7: $W_{t+1}(i) \leftarrow W_t(i)e^{\alpha_t}$
- 8: **end for**
- 9: **for** každý *správně* klasifikovaný $X_i \in D$ **do**
- 10: $W_{t+1}(i) \leftarrow W_t(i)e^{-\alpha_t}$
- 11: **end for**
- 12: normalizuj $W_{t+1}(i) \leftarrow W_{t+1}(i) / \sum_j W_{t+1}(j)$
- 13: **until** $t \geq T$ **nebo** $\epsilon_t = 0$ **nebo** $\epsilon_t \geq 0,5$
- 14: klasifikuj testovací vzory ensemblem s vahami komponent α_t

ϵ_t je podíl nesprávně predikovaných trénovacích vzorů (po vážení W_t) modelem v t -té iteraci.

Vlastnosti: produkuje *posloupnost* klasifikátorů se stejným bazovým učicím algoritmem, každý závisí na předchozím a soustředí se na jeho chyby. Při testování se výsledky posloupnosti kombinují. **Citlivost na šum:** je-li odlehlých hodnot velmi mnoho, důraz na „obtížné“ příklady může výkon zhoršit.

Náhodné lesy	Boosting
rychlé — mohou běžet <i>paralelně</i> a v každém uzlu prohledávají malou množinu příznaků	vyčerpávající — v každé fázi prohledává <i>celou</i> množinu příznaků a každá fáze závisí na předchozí
stromy jsou levné na trénování, lze jich ve stejném čase vypěstovat více i na velkých a složitých datech	musí běžet <i>sekvenčně</i> , což může být nákladné
překvapivě dobré výsledky vzhledem k malé části prostoru problému, kterou každý strom vidí	při <i>stejném počtu</i> stromů boosting náhodné lesy překonává (lesy v každé fázi prohledávají jen podmnožinu)

3.2.9 Srovnání klasifikačních metod

Metoda	Srozumitel- nost	Přesnost	Rychlost	Poznámka
Rozhodovací strom	vysoká	střední	vysoká	obdélníkové oblasti, nutné prořezávání, ignoruje korelace atributů
Pravidla z C4.5	nejvyšší	střední	vysoká	prořezávání pravidel je radikálnější než stromů
Naivní Bayes	střední	překvapivě dobrá	velmi vysoká	zvládne nekompletní vzory; problém s nulovými pravděpodobnostmi
SVM	velmi nízká	vysoká	nízká (trénink)	nejlepší pro vysokorozměrná data (text); jen 2 třídy, jen reálný prostor
Logistická regrese	vysoká	střední	vysoká	pravděpodobnostní výstup, interpretovatelné koeficienty stačí odhadnout μ_j a S_j
Diskriminační analýza	střední	dobrá při normalitě	vysoká	
Neuronová síť / ELM	velmi nízká	vysoká	ELM velmi vysoká	ELM negeneruje lokální minima, netrénuje skrytou vrstvu
Náhodný les	nízká	velmi vysoká	vysoká (paralelně)	snižuje rozptyl, nezhoršuje zkreslení
Boosting / AdaBoost	nízká	velmi vysoká	nízká (sekvencně)	snižuje zkreslení; citlivý na šum a odlehlé hodnoty

3.3 Klastrová analýza

3.3.1 Formulace úlohy a míry vzdálenosti

Klastrová analýza (shlukování, *cluster analysis*) rozděljuje pozorované vzory do skupin (klastřů, shluků) vzájemně podobných vzorů. **Předpoklad:** umíme měřit *vzdálenost* mezi vzory. Každý vzor je charakterizován m numerickými veličinami.

Míry vzdálenosti

Pro vzory $x_1 = (x_{11}, \dots, x_{1m})$ a $x_2 = (x_{21}, \dots, x_{2m})$:

1. **Hammingova** (*Manhattan*, L_1): $d_H(x_1, x_2) = \sum_{i=1}^m |x_{1i} - x_{2i}|$
2. **Euklidovská** (L_2): $d_E(x_1, x_2) = \sqrt{\sum_{i=1}^m (x_{1i} - x_{2i})^2}$
3. **Čebyševova** (L_∞): $d_C(x_1, x_2) = \max_i |x_{1i} - x_{2i}|$
4. **Minkowského metrika** (L_q) — předchozí tři jsou jejími speciálními případy: $d_q(x_1, x_2) = \left(\sum_{i=1}^m |x_{1i} - x_{2i}|^q \right)^{1/q}$

Platí $d_H = d_1$, $d_E = d_2$, $d_C = \lim_{q \rightarrow \infty} d_q$; pro vzor $(0, \dots, 0)$ a vzor jednotkové vzdálenosti dávají všechny tři metriky hodnotu 1.

Volba metriky závisí na zúčastněných veličinách a jejich rozsahu $\Rightarrow$ veličiny je třeba **normalizovat** (dělením průměrem, směrodatnou odchylkou, rozsahem max – min, ...; viz kap. 2). Uvedené metriky předpokládají *stejný rozptyl* všech veličin. Mají-li veličiny *různý* rozptyl, používá se

Mahalanobisova vzdálenost

$$d_M^2(x_1, x_2) = (x_1 - x_2)^\top S^{-1} (x_1 - x_2),$$

kde S je kovarianční matice. Zohledňuje jak různé rozptyly, tak korelace mezi veličinami (pro $S = I$ se redukuje na kvadrát euklidovské vzdálenosti).

Vzdálenost mezi dvěma klastry C_k a C_l

Metoda nejbližšího souseda (*single linkage*) — minimum ze vzdáleností jejich prvků: $D(C_k, C_l) = \min_{x_i, x_j} d(x_i, x_j)$, $x_i \in C_k$, $x_j \in C_l$.

Metoda nejvzdálenějšího souseda (*complete linkage*) — maximum: $D(C_k, C_l) = \max_{x_i, x_j} d(x_i, x_j)$.

Metoda průměrné vzdálenosti (*average linkage*) — průměr vzdáleností mezi vzory (n_k , n_l jsou počty vzorů v klastrech):

$$D(C_k, C_l) = \frac{1}{n_k n_l} \sum_{i=1}^{n_k} \sum_{j=1}^{n_l} d(x_i, x_j).$$

Centroidní metoda — vzdálenost středů klastrů: $D(C_k, C_l) = d(c_k, c_l)$, kde c_k , c_l jsou středy (centroidy) klastrů.

Centroid je střed klastru — „prototyp“ reprezentující daný klaster. Tentýž klaster může být reprezentován jedním nebo i více centroidy současně, podle tvaru klastru a zvolené metriky.

3.3.2 Metoda k -means

Algoritmus k -means (Lloyd, 1982)

```
1: rozděl vzory náhodně do  $k$  klastrů  $C_1, \dots, C_k$  (disjunktních, v sjednocení všechny vzory)
2: urči centroidy všech klastrů v aktuálním rozdělení, např.  $c_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$ 
3: for každý vzor  $x$  do
4:   urči vzdálenosti  $d(x, c_k)$  pro  $1 \leq k \leq K$ 
5:   nechť  $d(x, c_l) = \min_k d(x, c_k)$ 
6:   if  $x$  nepatří do klastru  $l$  then
7:     přesuň  $x$  do klastru  $l$ 
8:   end if
9: end for
10: if došlo k přesunu then
11:   goto krok 2
12: else
13:   konec
14: end if
```

Po skončení jsou klastry reprezentovány svými centroidy.

Varianty: při počátečním rozdělení prohlásit za centroidy *prvních k vzorů* (eliminuje krok 2 v první iteraci); upravovat centroidy *po každém* přesunu (v cyklu, nikoli po jeho skončení).

Rozšíření Lloydova algoritmu. Střídavé minimalizační kroky neposkytují záruku aproximace, přesto Lloydův algoritmus *nikdy nezvýší* cenu počátečního shlukování $\Rightarrow$ vyplatí se dodat algoritmu *prokazatelně dobrou inicializaci*:

- **k -means++** (Arthur, Vassilvitskii, 2007) — vyber k z P vzorů s pravděpodobností proporcionální kvadrátu vzdálenosti k nejbližšímu už existujícímu centroidu: $\frac{d^2(x_i, c)}{\sum_{j=1}^P d^2(x_j, c)}$.
- **LocalSearch++** (Lattanzi, Sohler, 2019) — nejprve k -means++ pro volbu k počátečních centroidů; poté se vzorkují nové vzory s analogickou pravděpodobností a nahrazují existující centroidy, pokud tím sníží celkovou kvadratickou vzdálenost vzorů k jejich nejbližším centroidům.

Metoda k -medians používá jako účelovou funkci Manhattanovu vzdálenost, $d(x_i, c) = \|x_i - c\|_1$. Lze ukázat, že optimálním reprezentantem c je pak *medián* datových bodů v klastru *podél každé dimenze*. Protože se medián volí v každé dimenzi nezávisle, výsledný m -rozměrný reprezentant obvykle *nepatří* do původní datové množiny. k -medians volí reprezentanty *robustněji* než k -means —

medián není tak citlivý na odlehlé hodnoty jako průměr.

3.3.3 Metoda k -medoids, CLARA a CLARANS

Algoritmus k -medoids

Pro datovou množinu D o n d -rozměrných bodech $x_1, \dots, x_n$ se hledá k reprezentantů $y_1, \dots, y_k$ z *původní databáze* D (počet k zadává uživatel), které minimalizují účelovou funkci danou součtem vzdáleností bodů k jejich nejbližším reprezentantům:

$$O = \sum_{i=1}^n \min_j d(x_i, y_j).$$

Časová složitost jedné iterace je $O(knd)$; obvykle stačí malý konstantní počet iterací.

Proč volit reprezentanty z D ? (1) Klastry k -means mohou být zkresleny odlehlými hodnotami — reprezentanti se mohou ocitnout v prázdných oblastech nerepresentativních pro většinu bodů klastru, což může vést k částečnému splynutí různých klastrů (lze částečně řešit pečlivým zacházením s odlehlými hodnotami). (2) Pro složitá data (např. množinu časových řad různé délky) může být obtížné optimálního centrálního reprezentanta vůbec spočítat. k -medoids lze proto definovat prakticky pro *libovolný typ dat*, stačí poskytnout vhodnou funkci podobnosti či vzdálenosti.

Postup: *hill-climbing* — množina reprezentantů S se inicializuje body z D a iterativně se zlepšuje výměnou bodu z S za datový bod z D . Strategie výměny:

1. vyzkoušet všech $|S| \cdot |D|$ možností a vybrat nejlepší — *extrémně drahé*;
2. použít *náhodně vybranou* množinu r párů (x, y) ($x \in D, y \in S$) a vyměnit nejlepší z nich. Vyžaduje čas proporcionální $r \cdot |D|$; algoritmus konverguje, když se účelová funkce nezlepší, nebo je-li průměrné zlepšení pod uživatelem zadaným prahem.

k -medoids je *pomalejší* než k -means, ale použitelný na různé typy dat.

Škálovatelné varianty: CLARA a CLARANS

V mnoha aplikacích jsou data tak velká, že se nevejdou do hlavní paměti a musí zůstat na disku — to omezuje návrh algoritmů. CLARA a CLARANS jsou škálovatelné implementace k -medoids; dědí výhody i nevýhody bazového algoritmu (CLARANS je snadno generalizovatelný na různé typy dat, ale dědí i relativně vysokou výpočetní složitost).

CLARA (*Clustering LARge Applications*) — vychází z varianty k -medoids zvané *PAM* (*Partitioning Around Medoids*), která testuje na výměnu *všech* $k(n-k)$ párů medoid–nemedoid a vymění pár s nejlepším zlepšením účelové funkce; výměny se provádějí až do konvergence k lokálnímu optimu. Každá iterace vyžaduje $O(kn^2)$ výpočtů vzdálenosti, tj. $O(kn^2d)$ času — *značně drahé*. CLARA proto algoritmus aplikuje na menší *vzorek* velikosti $f \cdot n$ ($f \ll 1$) a zbylé (nevzorkované) body přiřadí k nalezeným medoidům. Postup se opakuje nad nezávisle volenými vzorky stejné velikosti a vybere se nejlepší shlukování. Složitost iterace je pak $O(kf^2n^2d + k(n-k))$ — pro malá f o řády rychlejší. **Hlavní problém:** předvybrané vzorky nemusí obsahovat dobrou volbu medoidů.

CLARANS (*Clustering Large Applications based on RANdomized Search*) — pracuje s *plnými* daty, čímž se problému špatně předvybraných vzorků vyhýbá. Iterativně testuje výměny *náhodných* medoidů za *náhodné* nemedoidy; po každém pokusu se kontroluje, zda se kvalita zlepšila. Zlepší-li se, výměna se provede definitivně, jinak se počítá počet neúspěšných pokusů; lokální optimum je nalezeno po dosažení uživatelem zadaného počtu neúspěšných pokusů *MaxAttempt*. Celý proces se opakuje *MaxLocal*-krát a vybere se nejlepší z lokálních optim. CLARANS prozkoumá *větší rozmanitost* prostoru řešení než CLARA.

3.3.4 Hierarchické shlukování

Aglomerativní hierarchické shlukování

Přístup „zdola nahoru“ (*bottom-up*).

Inicializace: (1) určí vzájemné vzdálenosti mezi všemi vzory; (2) přiřadí každý vzor do samostatného klastru.

Hlavní cyklus: dokud je klastrů více než jeden: (1) najdi dva vzájemně nejbližší klastry a slouč je; (2) pro nový klaster spočítej jeho vzdálenost od ostatních klastrů.

Výsledek se zobrazuje jako **dendrogram** — ukazuje (zleva doprava) postupné slučování klastrů; výška spojení odpovídá vzdálenosti, při níž ke sloučení došlo. *Optimální počet klastrů není znám předem* — volí se řezem dendrogramu ve vhodné výšce.

Divizivní hierarchické shlukování („shora dolů“) postupuje obráceně: začíná jedním klastrem obsahujícím všechny vzory a ten postupně dělí. Nejznámějším příkladem v kontextu sítí je Girvanové-Newmanův algoritmus (kap. 6).

CURE (*Clustering Using REpresentatives*)

Aglomerativní hierarchický algoritmus, *velmi rychlý* a schopný odhalit klastry *libovolného tvaru* (na rozdíl např. od CLARANS). Postup:

1. Vyber vzorek s bodů z databáze D o n bodech.
2. Rozděl s bodů na p částí velikosti s/p .
3. Každou část shlukuj *nezávisle* hierarchickým slučováním do k' klastrů; celkový počet $k' \cdot p$ klastrů přes všechny části je stále větší než uživatelem požadované k .
4. Nad $k' \cdot p$ klastry proveď hierarchické shlukování až na požadovaných k .
5. Každý z $(n - s)$ nevzorkovaných bodů přiřadí ke klastru s nejbližším reprezentantem.

3.3.5 Vektorová kvantizace: algoritmus LVQ

LVQ — *Learning Vector Quantization*

Slouží pro **shlukování s učitelem** (třídy jsou označeny C):

- 1: inicializuj všechny váhové vektory $w_j(0)$, rychlosti učení $\mu(0)$, $k \leftarrow 0$
- 2: **while** není splněna zastavovací podmínka **do**
- 3: **for** každý trénovací vzor x **do**
- 4: urči index $j = q$ váhového vektoru s *minimální* euklidovskou vzdáleností, tj. $\min_j \|x - w_j\|_2^2$
- 5: **if** třída x souhlasí s třídou w_q **then**
- 6: $w_q(k+1) \leftarrow w_q(k) + \mu(k)(x - w_q(k))$ ▷ přitáhni
- 7: **else**
- 8: $w_q(k+1) \leftarrow w_q(k) - \mu(k)(x - w_q(k))$ ▷ odtlač
- 9: **end if**
- 10: **end for**
- 11: $k \leftarrow k + 1$; sniž rychlost učení, např. $\mu(k) = \mu(k-1)/(k+1)$ pro $k > 0$
- 12: **end while**

Váhové vektory se obvykle inicializují *prvními m vektory* trénovací množiny (musí zahrnovat vzory ze všech tříd) — tím se neurony automaticky „kalibrují“ s třídami.

3.3.6 Mřížkové a hustotní metody

Mřížkové metody (např. MAFIA)

- Data se diskretizují do p (ekvidistantních) intervalů, což pro d -rozměrná data dá p^d hyperkrychlí.
- Výsledné hyperkrychle jsou stavebními bloky pro shlukování.
- Pomocí prahu hustoty τ se identifikují *husté* hyperkrychle (alespoň τ bodů).
- Klastř libovolného tvaru se skládá z více hustých oblastí, které jsou *propojeny*; dvě mřížkové oblasti jsou propojené, sdílejí-li stěnu.

Cíl: určit klastry jako husté vzájemně propojené oblasti. Hustým mřížkovým buňkám odpovídají uzly grafu, hrany reprezentují sousedskou propojenost; data pokrytá buňkami z komponent souvislosti tohoto grafu tvoří výsledné klastry.

DBSCAN — *Density-Based Spatial Clustering of Applications with Noise*

Stavebními bloky jsou místo mřížkových buněk *jednotlivé datové body*, resp. kruhové oblasti okolo nich; hustota je dána počtem bodů ležících v jejich radiu ε (včetně bodu samotného):

1. **Jádrový bod** (*core point*) — kulová oblast o radiu ε okolo něj obsahuje alespoň τ datových bodů.
2. **Hraniční bod** (*border point*) — oblast obsahuje méně než τ bodů, ale zároveň alespoň jeden jádrový bod ve vzdálenosti ε .
3. **Šumový bod** (*noise point*) — není ani jádrový, ani hraniční.

Algoritmus:

1. Urči jádrové, hraniční a šumové body.
2. Zkonstruuuj graf propojenosti, kde každý uzel odpovídá jádrovému bodu; hrana mezi dvěma jádrovými body existuje *právě když* leží od sebe ve vzdálenosti nejvýše ε .
3. Najdi všechny komponenty souvislosti tohoto grafu — odpovídají klastrům (postaveným na jádrových bodech). Hraniční body se přiřadí ke klastru, s nímž mají nejvyšší úroveň propojenosti; šumové body se ohlásí jako *odlehle hodnoty*.

Metoda	Tvar klastrů	Počet klastrů	Odlehle hodnoty	Poznámka
k -means	kulovité	zadán	citlivá	rychlá; závislá na inicializaci $\rightarrow k$ -means++
k -medians	kulovité	zadán	robustnější	medián po dimenzích; reprezentant není z dat
k -medoids	kulovité	zadán	robustní	reprezentanti z dat; libovolný typ dat; pomalejší
CLARA / CLARANS	kulovité	zadán	robustní	škálovatelné k -medoids (vzorkování / náhodné prohledávání)
Hierarchické	podle metriky spojování	určí se z dendrogramu	citlivé	bez nutnosti zadat k předem; $O(n^2)$ – $O(n^3)$
CURE	libovolný	zadán	robustní	vzorkování + hierarchické slučování po částech
LVQ	kulovité	dán počtem neuronů	—	shlukování s učitelem
Mřížkové (MAFIA)	libovolný	vyplyne	šum vypadne	citlivé na granularitu mřížky
DBSCAN	libovolný	vyplyne	detekuje je	parametry ε, τ ; problém s různou hustotou klastrů

3.4 Shrnutí ke zkoušce

Asociační pravidla.

- **Support** = podíl transakcí s i i j ; **confidence** = podíl transakcí s i i j mezi transakcemi s i ;

$\text{lift} = p(i \wedge j) / (p(i)p(j))$. $\text{Lift} < 1 \Rightarrow$ pravidlo horší než náhoda, negace závěru může být lepší. Umět spočítat všechny tři z malé tabulky transakcí.

- **Apriori** využívá *uzavřenost dolů* (každá podmnožina frekventované množiny je frekventovaná), prohledává *do šířky*; APRIORI-GEN = krok spojení (kombinace shodné v $k - 2$ kategoriích) + krok prořezání (chybí-li podkombinace délky $k - 1$). Prostor pravidel je $O(2^m)$, MBA produkuje *nadměrné množství pravidel*.
- **Taxonomie a virtuální položky** řeší volbu úrovně položek; vzácné položky výše v taxonomii, časté níže; virtuální položky nesou informaci o transakci (den, čas, zákazník) — pozor na redundanci pravidel.
- **MS-Apriori**: problém vzácných položek $\rightarrow$ každá položka má vlastní MIS, minsup pravidla je *minimum* MIS jeho položek, plus omezení rozdílu supportů φ . *Uzavřenost dolů přestává platit* $\Rightarrow$ položky se seřadí vzestupně podle MIS, používá se množina semen L (ne F_1) a **tailCount** pro generování pravidel. Umět uvést protipříklad k uzavřenosti dolů.
- **CAR**: transakce označované třídou, pravidlo $X \rightarrow y$; dolují se *přímo v jednom kroku* přes pravidlové položky ($\text{condset}, y$); lze zadat různé minsupy různým třídám (101 % = zakázat třídu).
- **Sekvenční vzory**: sekvence = uspořádaný seznam množin položek; *velikost* = počet prvků, *délka* = počet položek; podsekvence přes vnořené inkluze. Algoritmus **GSP** pracuje obdobně jako Apriori. Pozor: $\langle \{3\}\{8\} \rangle$ a $\langle \{3, 8\} \rangle$ se navzájem *neobsahují*.
- **FP-Growth**: 2 průchody daty, žádné generování kandidátů; FP-strom sdílí *prefixy* (pořadí položek podle klesajícího supportu — heuristika), spojové seznamy položek; extrakce zdola nahoru *rozděl a panuj* přes *prefixové podstromy* a *podmíněné FP-stromy* (aktualizovat počítadla $\rightarrow$ odstranit uzly položky $\rightarrow$ odstranit nefrekventované položky). Nejhorší případ: unikátní transakce $\Rightarrow$ strom větší než data.

Učení s učitelem.

- **Rozhodovací stromy**: TDIDT (rozděl a panuj), volba atributu podle *minimální vážené entropie* $H(A) = \sum_v \frac{|A(v)|}{|D|} H(A(v))$, ekvivalentně maximálního *informačního zisku*. Umět projít příklad s úvěrem (příjem 0,5747; konto 0,6667; pohlaví 0,9183; nezaměstnaný 0,825) a převést strom na pravidla. Složitost indukce $O(np \log p)$, klasifikace $O(q \log p)$.
- **ID3** (Quinlan 1986) — hladový, informační zisk. **C4.5** (1993) — chybějící a spojitá data, prořezávání validací (*reduced-error pruning*, *subtree raising*), *rule post-pruning*, odhad přesnosti dolní mezi $\text{acc} - 1,96\sqrt{\text{acc}(1 - \text{acc})/n}$, *poměrný* informační zisk (penalizuje nadměrné větvení). **C5.0** — komerční, boosting. **CART** — *binární* stromy, Gini index $G(v_i) = 1 - \sum_j p_j^2$ (nižší = lepší), kritérium $\Phi = 2P_L P_R \sum_j |P(C_j|t_L) - P(C_j|t_R)|$. **CHAID** — χ^2 , automatické seskupování hodnot kategoriálních atributů.
- **Bayes**: $H_{\text{MAP}} = \arg \max_H P(E | H)P(H)$ (jmenovatel lze zanedbat); *naivní* předpoklad podmíněné nezávislosti $\Rightarrow \arg \max_{H_t} P(H_t) \prod_i P(E_i | H_t)$. Umí nekompletní vzory; problém nulových pravděpodobností. Umět spočítat příklad (0,1042 vs. 0,0416).
- **SVM**: maximální okraj $2/\|w\|$, primární úloha $\min \frac{\langle w \cdot w \rangle}{2}$ za podmínek $y_i(\langle w \cdot x_i \rangle + b) \geq 1$; Kuhn–Tuckerovy podmínky jsou pro konvexní úlohu s lineárními omezeními nutné i *postačující*; $\alpha_i > 0$ jen pro *podpůrné vektory*; Wolfeho duál. **Kernelový trik**: duál potřebuje jen skalární součiny $\Rightarrow K(x, z) = \langle \phi(x) \cdot \phi(z) \rangle$, ϕ nikdy nevyčíslíme (Mercerova věta). Jen 2 třídy, jen reálný prostor, neinterpretovatelné.
- **Logistická regrese**: $\ln \frac{p}{1-p} = \beta^\top x$, tedy $p = \sigma(z) = 1/(1 + e^{-z})$; učení maximální věrohodnosti / minimalizací křížové entropie; $\sigma' = \sigma(1 - \sigma) \Rightarrow$ adaptační pravidlo $\Delta \beta_j = \eta \sum_i (y_i - \sigma(z_i)) x_{ij}$.
- **Diskriminační analýza**: $g_j(x)$ pro každou třídu, klasifikuje se maximem; pro dvě třídy stačí znaménko $g = g_1 - g_2$; optimální je $g_j \propto P(C_j | x)$; normální rozdělení $\rightarrow$ kvadratická funkce, při shodných kovariančních maticích *lineární* $\Rightarrow$ stačí odhadnout μ_j a S .
- **ELM**: náhodné parametry skrytých neuronů (i bez znalosti dat), výstupní váhy *jednou* pseudoinverzí $\beta = H^+ T$; regularizace I/C na diagonále. Rychlé, bez lokálních minim, funguje s libovolnou omezenou po částech spojitou přenosovou funkcí.
- **Ensemble**: *bagging* snižuje *rozptyl* ($\sigma^2 \rightarrow \sigma^2/k$ pro i.i.d.), bootstrap obsahuje $\approx 63,2\%$ rů-

ných bodů $(1 - 1/e)$, stromy jsou ideální komponenty (nízké zkreslení, vysoký rozptyl); zkreslení nesnižuje. *Náhodné lesy* řeší korelaci komponent — rozptyl je $\rho\sigma^2 + (1 - \rho)\sigma^2/k$, první člen na k nezávisí; náhodnost přes bootstrap a náhodnou podmnožinu $m \approx \sqrt{\#\text{příznaků příznaků}}$ v každém uzlu. *Boosting/AdaBoost* snižuje *zkreslení*, váhy chybně klasifikovaných vzorů rostou, $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, běží *sekvenčně*, je citlivý na šum. Umět srovnat lesy vs. boosting (paralelní a levné vs. vyčerpávající a přesnější při stejném počtu stromů).

Klastrová analýza.

- **Metriky**: Hammingova/Manhattan (L_1), euklidovská (L_2), Čebyševova (L_∞), Minkowského (L_q , ostatní jsou její speciální případy); **Mahalanobisova** $d_M^2 = (x_1 - x_2)^T S^{-1} (x_1 - x_2)$ pro veličiny s různým rozptylem. Před shlukováním *normalizovat*.
- **Vzdálenost klastrů**: nejbližší soused (min), nejvzdálenější soused (max), průměrná vzdálenost, centroidní metoda.
- **k-means** (Lloyd 1982): náhodné rozdělení → centroidy → přesuny → opakuj, dokud dochází k přesunům. Nikdy nezvýší cenu, ale bez záruky aproximace ⇒ **k-means++** (pravděpodobnost $\propto$ kvadrát vzdálenosti k nejbližšímu centroidu), **LocalSearch++**.
- **k-medians** (Manhattan, medián po dimenzích, robustnější) vs. **k-medoids** (reprezentanti z *dat*, hill-climbing s výměnami, $O(knd)$ na iteraci, libovolný typ *dat*). **PAM** testuje všech $k(n - k)$ výměn ($O(kn^2d)$); **CLARA** vzorkuje ($O(kf^2n^2d + k(n - k))$, riziko špatného vzorku), **CLARANS** prohledává náhodně nad plnými *daty* (*MaxAttempt*, *MaxLocal*).
- **Hierarchické shlukování** zdola nahoru: každý vzor svůj klast → slučuj dva nejbližší; výstupem **dendrogram**, počet klastrů se volí až řezem. **CURE**: vzorek → p částí → nezávisle hierarchicky na k' klastrů → hierarchicky na k → přiřaď zbytek; rychlé, libovolný tvar klastrů.
- **LVQ**: shlukování s *učitelem* — vítězný váhový vektor se ke vzoru přitáhne při souhlasu tříd a odtlačí při nesouhlasu; μ klesá jako $\mu(k - 1)/(k + 1)$.
- **Mřížkové metody** (MAFIA): p^d hyperkrychlí, prah hustoty τ , klastry jako propojené husté oblasti (komponenty souvislosti grafu buněk). **DBSCAN**: *jádrový bod* ($\geq \tau$ bodů v radiu ε), *hraniční* ($< \tau$, ale jádrový bod do ε), *šumový*; klastry = komponenty souvislosti grafu jádrových bodů, šumové body se hlásí jako odlehlé. Umí klastry *libovolného tvaru* a sama detekuje odlehlé hodnoty.

4 Charakteristická a diskriminační pravidla a jejich zajímavost

♦ **nad rámec slidů** Celá kapitola je nad rámec slidů obou kurzů — okruh ji ale jmenuje výslovně a jde o standardní látku, zpracovanou zde podle Han–Kamber. Souvislost s generalizací v OLAP viz kap. 1.

4.1 Charakterizace vs. diskriminace

Deskriptivní dobývání znalostí odpovídá na dvě příbuzné, ale odlišné otázky:

Charakterizace vs. diskriminace

Charakterizace dat (*data characterization*) — shrnutí obecných vlastností *cílové třídy* (*target class*). Odpověď na otázku: „Jak vypadá typický klient s hypotékou?“ Výsledkem jsou *charakteristická pravidla* (*characteristic rules*).

Diskriminace dat (*data discrimination*) — porovnání obecných vlastností cílové třídy s vlastnostmi jedné či více *kontrastních tříd* (*contrasting classes*). Odpověď na otázku: „Čím se klient s hypotékou liší od klienta bez hypotéky?“ Výsledkem jsou *diskriminační pravidla* (*discriminant rules*).

Rozdíl je zásadní: vlastnost může být pro cílovou třídu *typická* (vysoký podíl uvnitř třídy), a přesto nemá žádnou *rozlišovací sílu*, protože je stejně typická i pro kontrastní třídu.

Obě úlohy řeší metoda *indukce orientované na atributy* (*attribute-oriented induction*, AOI), navržená Hanem, Cai a Cercone (1991). Alternativou je datově-kostkový (*data cube*) přístup známý z OLAP: operace *roll-up* odpovídá generalizaci, *drill-down* specializaci. AOI je flexibilnější (zvládne i atributy bez předem materializované kostky), OLAP je rychlejší (předpočítané agregace).

4.2 Koncepční hierarchie

Koncepční hierarchie

Koncepční hierarchie (*concept hierarchy*) atributu A je částečné uspořádání jeho hodnot od nejkonkrétnějších (listy) po nejobecnější (kořen ANY). Typy:

- *schémátová* (*schema hierarchy*) — daná strukturou dat: ulice $\prec$ město $\prec$ kraj $\prec$ stát;
- *seskupovací* (*set-grouping*) — explicitní seskupení hodnot: $\{20\text{--}29\} \prec \text{mladý}$;
- *operační* (*operation-derived*) — odvozená výpočtem: e-mailová adresa $\rightarrow$ doména;
- *pravidlová* (*rule-based*) — definovaná pravidly nad více atributy.

Hierarchii zadává expert, nebo se generuje automaticky (diskretizací u numerických atributů, podle počtu různých hodnot u kategoriálních — atribut s více různými hodnotami je obvykle na nižší úrovni hierarchie).

4.3 Indukce orientovaná na atributy (AOI)

Algoritmus AOI

Vstup: databáze, dotaz vymezující cílovou třídu, seznam atributů, prahy generalizace T_i pro jednotlivé atributy a celkový práh T pro velikost výsledné relace.

- 1: **sběr dat:** vyber dotazem množinu objektů cílové třídy (*initial working relation*)
- 2: ke každému objektu přidej atribut `count = 1`
- 3: **for** každý atribut A_i **do**
- 4: **if** A_i má mnoho různých hodnot a *neexistuje* pro něj koncepční hierarchie **then**
- 5: **odstranění atributu** (*attribute removal*)
- 6: **else if** A_i má mnoho různých hodnot, ale hierarchie existuje **then**
- 7: **generalizace atributu** (*attribute generalization*): nahraď hodnoty
- 8: jejich předky o úroveň výš, dokud počet různých hodnot $\leq T_i$

```

9:   end if
10: end for
11: agregace: slouč identické zobecněné objekty a sečti jejich count
12: while počet řádků výsledné relace >  $T$  do
13:   zobecni dále atribut, jehož generalizace nejvíce sníží počet řádků
14: end while
15: prezentace: zobraz primární relaci (prime relation) jako tabulku, křížovou
16: tabulku, graf, nebo ji převed' na kvantitativní pravidla

```

Dvě základní generalizační operace:

- **Odstranění atributu** — má-li atribut příliš mnoho různých hodnot a (a) neexistuje pro něj generalizační operátor, nebo (b) jeho vyšší úrovně jsou vyjádřeny jinými atributy, atribut se z relace odstraní. Příklad: *jméno, telefon, rodné číslo*.
- **Generalizace atributu** — existuje-li generalizační operátor (konceptní hierarchie), nahradí se hodnoty hodnotami na vyšší úrovni. Příklad: *datum narození* $\rightarrow$ *věk* $\rightarrow$ *věková kategorie*; *město* $\rightarrow$ *kraj* $\rightarrow$ *region*.

Řízení generalizace probíhá dvěma prahy: *prah generalizace atributu* T_i — maximální povolený počet různých hodnot atributu (typicky 2–8); *prah velikosti relace* T — maximální počet řádků výsledné primární relace (typicky 10–30). Příliš vysoká generalizace vede k triviálním výsledkům („všichni klienti jsou lidé“), příliš nízká k nepřehledné tabulce. Vhodná úroveň se hledá interaktivně (drill-down / roll-up).

Složitost je $O(na)$ pro n objektů a a atributů (jeden průchod daty a agregace), pokud se generalizace realizuje přes hašovací tabulku zobecněných záznamů — AOI je tedy *lineární* v počtu objektů, což je pro velké databáze zásadní.

4.4 Míry t -weight a d -weight

t-weight a d-weight

Nechť primární relace obsahuje zobecněné záznamy (*generalized tuples*) $q_1, \dots, q_N$.

Typičnost (*typicality*, t -weight) záznamu q_a v cílové třídě:

$$t\text{-weight}(q_a) = \frac{\text{count}(q_a)}{\sum_{i=1}^N \text{count}(q_i)} \in [0, 1],$$

tj. podíl objektů cílové třídy, které záznam q_a pokrývá. Součet t -weight přes všechny záznamy primární relace je 1 (100 %). Slouží pro *charakteristická* pravidla.

Diskriminační váha (d -weight) záznamu q_a vůči cílové třídě C_{target} a kontrastním třídám $C_1, \dots, C_m$:

$$d\text{-weight}(q_a) = \frac{\text{count}(q_a \in C_{\text{target}})}{\sum_{j=1}^m \text{count}(q_a \in C_j) + \text{count}(q_a \in C_{\text{target}})} \in [0, 1],$$

tj. podíl objektů pokrytých záznamem q_a , které patří do cílové třídy. Slouží pro *diskriminační* pravidla. Vysoká d -weight ($\rightarrow 1$) znamená, že záznam cílovou třídu silně odlišuje; hodnota blízká podílu tříd znamená nulovou rozlišovací schopnost.

Kvantitativní charakteristické, diskriminační a popisné pravidlo

Kvantitativní charakteristické pravidlo (nutná podmínka — implikace z třídy):

$$\forall X \text{ target_class}(X) \Rightarrow \text{cond}_1(X) [t: w_1] \vee \dots \vee \text{cond}_N(X) [t: w_N]$$

Čte se: patří-li X do cílové třídy, pak splňuje cond_1 s typičností w_1 , nebo cond_2 s typičností w_2 ,

...

Kvantitativní diskriminační pravidlo (postačující podmínka — implikace *do* třídy):

$$\forall X \text{ target_class}(X) \Leftarrow \text{cond}(X) [d: w]$$

Čte se: splňuje-li X podmínku cond , patří s pravděpodobností w do cílové třídy.

Kvantitativní popisné pravidlo (*quantitative description rule*) kombinuje obojí (nutná i postačující podmínka):

$$\forall X \text{ target_class}(X) \Leftrightarrow \text{cond}_1(X) [t: w_1, d: w'_1] \vee \dots \vee \text{cond}_N(X) [t: w_N, d: w'_N]$$

Příklad — charakterizace a diskriminace klientů banky

Cílová třída: klienti s hypotékou (200 osob). **Kontrastní třída:** klienti bez hypotéky (800 osob).

Po AOI (odstraněno *jméno, rodné číslo, telefon*; generalizováno *datum narození* → *věková kategorie, plat* → *příjmová kategorie, obec* → *region*) vznikne primární relace:

Věk	Příjem	Region	počet		t-weight	d-weight
			cíl	kontrast	(cíl)	
30–40	vysoký	Praha	90	10	45 %	90 %
30–40	střední	Praha	50	150	25 %	25 %
40–50	vysoký	mimo Prahu	40	160	20 %	20 %
20–30	nízký	mimo Prahu	20	480	10 %	4 %
Celkem			200	800	100 %	

Výpočet. t -weight prvního záznamu = $90/200 = 45\%$; d -weight = $90/(90 + 10) = 90\%$. Pro druhý záznam $t = 50/200 = 25\%$, $d = 50/(50 + 150) = 25\%$. Pro čtvrtý $t = 20/200 = 10\%$, $d = 20/(20 + 480) = 4\%$.

Kvantitativní charakteristické pravidlo:

$$\forall X \text{ hypotéka}(X) \Rightarrow (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}) [t: 45\%] \\ \vee (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{střední} \wedge \text{region}(X)=\text{Praha}) [t: 25\%] \vee \dots$$

Kvantitativní diskriminační pravidlo:

$$\forall X \text{ hypotéka}(X) \Leftarrow (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}) [d: 90\%]$$

Interpretace rozdílů. Poslední řádek ukazuje, proč jsou obě míry potřeba: profil „20–30, nízký příjem, mimo Prahu“ pokrývá 10 % klientů s hypotékou (nezanedbatelná typičnost), ale jeho d -weight je pouhá 4 % — výrazně *méně* než apriorní podíl cílové třídy $200/1000 = 20\%$. Tento profil tedy hypotéku spíše *vylučuje*. U druhého řádku je d -weight jen nepatrně *nad* apriorním podílem (25 % vs. 20 %), takže rozlišovací síla je zanedbatelná, a u třetího je d -weight *přesně rovna* apriornímu podílu (20 %), tedy rozlišovací síla je **nulová** — profil o příslušnosti k cílové třídě neříká vůbec nic, přestože pokrývá pětinu klientů s hypotékou.

4.5 Měření zajímavosti pravidel

Zajímavost odvozených pravidel se posuzuje obdobně jako u asociačních pravidel (odd. 3.1), s důrazem na následující míry:

Míra	Význam
t -weight	typičnost profilu uvnitř cílové třídy; pravidlo s nízkou t -weight popisuje okrajový jev
d -weight	rozlišovací síla profilu; <i>zajímavá</i> jsou pravidla s d -weight výrazně vyšší (nebo nižší) než apriorní podíl třídy
Lift / zdvih	$\frac{d\text{-weight}}{P(C_{target})}$ — kolikrát profil zvýší šanci na cílovou třídu
Pokrytí (<i>coverage</i>)	podíl všech objektů pokrytých pravidlem; nízké pokrytí $\Rightarrow$ statisticky nespolehlivé
χ^2	statistická významnost vztahu profilu a třídy
Informační zisk	snížení entropie třídy po znalosti profilu
Jednoduchost	délka pravidla (počet podmínek), počet pravidel; Occamova břitva
Neočekávanost	rozpor s modelem světa uživatele (subjektivní míra)
Akceschopnost	lze na jeho základě jednat (subjektivní míra)

Prakticky: pravidlo je zajímavé, je-li současně (a) *srozumitelné*, (b) *platné* na nových datech s vysokou spolehlivostí, (c) *potenciálně užitečné* a (d) *nové*. První tři jsou objektivně měřitelné, poslední vyžaduje znalost uživatele.

Klasická past: pravidlo s vysokou t -weight, ale d -weight rovnou apriornímu podílu třídy, není objevem, i když vypadá impozantně („80 % klientů s hypotékou má bankovní účet“ — ale ten má i 80 % klientů bez hypotéky).

4.6 Shrnutí ke zkoušce

- **Charakterizace** popisuje cílovou třídu; **diskriminace** ji porovnává s kontrastními třídami. Rozdíl: typičnost $\neq$ rozlišovací síla.
- **AOI:** sběr dat dotazem $\rightarrow$ přidání **count** $\rightarrow$ *odstranění atributu* (mnoho hodnot, žádná hierarchie) nebo *generalizace atributu* (podle koncepční hierarchie) $\rightarrow$ agregace stejných záznamů $\rightarrow$ další generalizace, dokud relace není menší než práh $T \rightarrow$ prezentace. Složitost $O(na)$ — lineární.
- **Koncepční hierarchie:** schémátová, seskupovací, operační, pravidlová.
- **t-weight** = $\frac{\text{count}(q_a)}{\sum_i \text{count}(q_i)}$ — typičnost uvnitř cílové třídy, součet přes všechny záznamy je 100 %; pro *charakteristická* pravidla ($\Rightarrow$, nutná podmínka).
- **d-weight** = $\frac{\text{count}(q_a \in C_{target})}{\sum_j \text{count}(q_a \in C_j)}$ — rozlišovací síla; pro *diskriminační* pravidla ($\Leftarrow$, postačující podmínka). Popisné pravidlo kombinuje obojí ($\Leftrightarrow$).
- Umět spočítat obě váhy z tabulky a vysvětlit, proč vysoká t -weight neznamena vysokou d -weight (srovnání s *apriorním* podílem třídy).
- **Zajímavost:** objektivní (t/d -weight, lift, pokrytí, χ^2 , informační zisk, jednoduchost) vs. subjektivní (neočekávanost, akceschopnost).

5 Reprezentace, vyhodnocování a vizualizace znalostí

Kapitola má tři části podle tří slov ve větě okruhu: *v jaké formě* znalost vyjádříme, *jak ověříme*, že je dobrá, a *jak ji zobrazíme*. Prostřední část — vyhodnocování klasifikátorů — je jediná, kterou slidy pokrývají v celém rozsahu, a je zároveň ta, na kterou se u zkoušky nejčastěji ptá.

5.1 Formy reprezentace znalostí

♦ nad rámec slidů: tabulka jako přehled; jednotlivé metody v ní jsou ve slidech doloženy]

Forma	Typický zdroj	Vlastnosti
IF-THEN pravidla	asociační pravidla, C4.5	nejrozumitelnější; lokální (každé pravidlo se čte samostatně); riziko konfliktů a nadprodukce
Rozhodovací strom	ID3/C4.5/CART	globální model, snadno čitelný do hloubky 4–5; nad ni nepřehledný
Rozhodovací tabulka	diskretizovaná data	úplný výčet kombinací; přehledná jen pro málo atributů
Matematická funkce	regrese, SVM, NN	kompaktní a přesná; obtížně interpretovatelná (kromě lineárního modelu s normalizovanými vahami)
Prototypy / centroidy	k -means, k -medoids, LVQ	„typický zástupce“ skupiny; velmi intuitivní pro byznys
Pravděpodobnostní model	naivní Bayes, bayesovská síť	explicitní nejistota; graf sítě je čitelný
Grafy a sítě	analýza sociálních sítí	vztahy mezi entitami; vizuálně silné
Instance (paměť případů)	k -NN, CBR	„model“ je samotná data; vysvětlení odkazem na podobné případy

Interpreovatelnost a vysvětlitelnost

♦ nad rámec slidů: vysvětlitelnost a post-hoc metody; interpreovatelnost jako kritérium hodnocení modelu je ve slidech — viz odd. 5.2] *Interpreovatelnost* (*interpretability*) — schopnost člověka porozumět tomu, jak model dospěl k výstupu. *Vysvětlitelnost* (*explainability*) — schopnost podat srozumitelné odůvodnění konkrétního rozhodnutí.

Rozlišujeme *modely inherentně interpreovatelné* (lineární modely, rozhodovací stromy, sady pravidel, prototypy) a *post-hoc vysvětlení černých skříněk* — důležitost atributů (permutační, SHAP), lokální surogátní modely (LIME), grafy částečné závislosti (PDP), extrakce pravidel z neuronové sítě, analýza citlivosti, kontrafaktuální vysvětlení („kdyby byl váš příjem o 5000 vyšší, úvěr by byl schválen“).

Interpreovatelnost není jen pohodlí: v regulovaných doménách (úvěry, pojištění, medicína) je právním požadavkem (GDPR čl. 22 — právo na vysvětlení automatizovaného rozhodnutí; AI Act pro vysoce rizikové systémy).

5.2 Vyhodnocování modelů

5.2.1 Kritéria hodnocení klasifikačních metod

- **Prediktivní přesnost** (*accuracy*) — podíl správně klasifikovaných vzorů:

$$Accuracy = \frac{\text{počet správných klasifikací}}{\text{počet všech testovaných případů}}.$$

- **Efektivita** — čas potřebný pro konstrukci modelu a čas pro jeho použití.
- **Robustnost** — zacházení se šumem a chybějícími hodnotami.
- **Škálovatelnost** — efektivita pro databáze uložené na disku.
- **Interpreovatelnost** — jasnost a vzhled, který model poskytuje.
- **Kompaktnost modelu** — velikost stromu, počet pravidel, ...

5.2.2 Metody odhadu chyby

Holdout, křížová validace, leave-one-out, validační množina

Holdout (*holdout set*): dostupná množina D se rozdělí na dvě disjunktní podmnožiny — *trénovací* D_{train} (pro naučení modelu) a *testovací* D_{test} (pro otestování). **Zásadní pravidlo:** trénovací množina se nesmí použít při testování a testovací při učení — jen neviděná testovací množina dává *nezkreslený* odhad přesnosti. Vhodné především pro *velká* D .

k -násobná křížová validace (*k-fold cross-validation*): data se rozdělí uniformně na k stejně velkých disjunktních podmnožin. Každá podmnožina se použije jako testovací, zbývajících $k - 1$ jako trénovací. Procedura se provede k -krát, čímž se získá k přesností $Accuracy_i$; výsledný odhad je jejich průměr

$$Accuracy_{CV} = \frac{1}{k} \sum_{i=1}^k Accuracy_i.$$

Běžné jsou 10-násobná a 5-násobná křížová validace; metoda se používá spíše pro *menší* data.

Leave-one-out: speciální případ křížové validace, kde každý fold obsahuje jediný testovací vzor a vše ostatní se trénuje — pro m vzorů jde o m -násobnou křížovou validaci. Používá se pro *velmi malá* data.

Validační množina (*validation set*): data se rozdělí na *tři* části — trénovací, validační a testovací. Validační množina slouží k odhadu *parametrů* učících algoritmů: použijí se hodnoty, které na ní dávají nejlepší přesnost. K odhadu parametrů lze použít i křížovou validaci.

Interval spolehlivosti pro křížovou validaci

Přibližný $N\%$ interval spolehlivosti:

$$Accuracy_{CV} \pm t_{N,k-1} \cdot s_{Accuracy}, \quad s_{Accuracy} = \sqrt{\frac{1}{k(k-1)} \sum_{i=1}^k (Accuracy_i - Accuracy_{CV})^2}.$$

Hodnoty $t_{N,k-1}$ pro dvoustranné intervaly:

k	hladina spolehlivosti N		
	90 %	95 %	99 %
2	2,92	4,30	9,92
10	1,81	2,23	3,17
30	1,70	2,04	2,75
∞	1,64	1,96	2,58

Často se používá $t = 1,96$ (pro $k \rightarrow \infty$ a $N = 95\%$).

5.2.3 Matice záměn a odvozené míry

Přesnost je jen *jedna* míra ($error = 1 - accuracy$) a v některých aplikacích není vhodná. Při do-
lování z textu nás mohou zajímat jen dokumenty určitého tématu, což je malá část velké kolekce. Při klasifikaci silně *nevyvážených* dat (detekce průniků do sítě, finanční podvody) nás zajímá jen minoritní třída: při 1 % průniků dosáhneme 99% přesnosti tím, že *neuděláme nic* — a neodhalíme žádný průnik. Zajímavá třída se nazývá *pozitivní*, ostatní *negativní*.

Matice záměn, precision, recall, F_1

		klasifikováno jako	
		pozitivní	negativní
skutečnost	pozitivní	TP	FN
	negativní	FP	TN

TP — správné klasifikace pozitivních vzorů (*true positive*); FN — nesprávné klasifikace pozitivních vzorů (*false negative*); FP — nesprávné klasifikace negativních vzorů (*false positive*); TN — správné klasifikace negativních vzorů.

$$p = \text{precision} = \frac{TP}{TP + FP}, \quad r = \text{recall} = \frac{TP}{TP + FN}.$$

Precision p je počet správně klasifikovaných pozitivních vzorů dělený počtem všech vzorů klasifikovaných jako pozitivní. *Recall* r je počet správně klasifikovaných pozitivních vzorů dělený počtem všech skutečně pozitivních vzorů v testovací množině. **Obě míry hodnotí klasifikaci pouze na pozitivní třídě.**

F_1 -míra kombinuje obě do jedné — je jejich *harmonickým průměrem*:

$$F_1 = \frac{2pr}{p + r} = \frac{2}{\frac{1}{p} + \frac{1}{r}}.$$

Harmonický průměr dvou čísel je blíž tomu *menšímu* z nich, takže F_1 je velká jen tehdy, jsou-li obě míry velké.

Ilustrace pasti: klasifikátor, který správně označí jediný pozitivní vzor a žádný negativní neoznačí chybně, má $p = 100\%$ a $r = 1\%$. Vysoká precision sama o sobě tedy neříká nic o užitečnosti.

5.2.4 Skórování, řazení a lift analýza

Skórování (*scoring*) je příbuzné klasifikaci: zajímá nás jediná (pozitivní) třída — např. třída kupujících v marketingové databázi. Místo definitivního zařazení se každému testovacímu vzoru přiřadí *odhad pravděpodobnosti* (*probability estimate*, PE), že patří do pozitivní třídy. Klasifikační systémy lze pro skórování použít, musí ale poskytnout pravděpodobnostní odhad — např. u rozhodovacích stromů lze použít hodnotu confidence v listu.

Po oskórování se vzory *seřadí* podle PE a rozdělí do n (např. 10) přihrádek (*bins*); podle počtu pozitivních vzorů v každé přihrádce se nakreslí **lift křivka** — *lift analýza*.

Příklad — lift analýza pro cílenou kampaň

Chceme rozeslat propagační materiály potenciálním zákazníkům (hodinky). Každá zásilka stojí \$0,50, prodej jedné hodinky přinese \$5 zisku. Kolik zásilek poslat a komu?

Testovací množina má 10 000 vzorů, z nichž 500 je pozitivních. Po oskórování a seřazení do 10 přihrádek (po 1000 vzorech):

Přihrádka	1	2	3	4	5	6	7	8	9	10
Pozitivních	210	120	60	40	22	18	12	7	6	5
% z celku	42	24	12	8	4,4	3,6	2,4	1,4	1,2	1,0
Kumulativně	42	66	78	86	90,4	94	96,4	97,8	99	100

První přihrádka obsahuje 42 % všech pozitivních případů, první tři 78 %. Náhodné rozeslání by v každé přihrádce dalo 10 % — rozdíl mezi lift křivkou a touto diagonálou je přínos modelu. Rozhodnutí „kolik zásilek“ pak vychází z ekonomiky: zásilkou do první přihrádky získáme $210 \cdot \$5 - 1000 \cdot \$0,50 = \$550$, do desáté $5 \cdot \$5 - 1000 \cdot \$0,50 = -\$475$.

5.2.5 ROC křivka a AUC

ROC křivka (*Receiver Operating Characteristic*)

Graf *míry správně pozitivních* (TPR) proti *míře falešně pozitivních* (FPR):

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}, \quad TNR = \frac{TN}{FP + TN}.$$

TPR je podíl skutečně pozitivních případů, které jsou správně klasifikovány — je to v podstatě *recall* pozitivní třídy, nazývá se též *senzitivita*. FPR je podíl skutečně negativních případů klasifikovaných jako pozitivní. TNR (recall negativní třídy) se nazývá *specificita* a platí $FPR = 1 - \text{specificita}$.

Každá ROC křivka začíná v $(0, 0)$ (vše klasifikováno jako negativní) a končí v $(1, 1)$ (vše jako pozitivní). **Hlavní diagonála** odpovídá náhodnému hádání — predikci pozitivní třídy s fixní pravděpodobností, kdy $TPR = FPR$.

Konstrukce ROC křivky z klasifikace ve formě žebříčku testovacích případů:

1. Získej žebříček setříděním testovacích případů *klesajícím* výstupem klasifikátoru (např. aposteriorní pravděpodobností).
2. Rozděl žebříček *na každém* testovacím případě na dvě části a považuj každý případ v horní části za pozitivní a každý v dolní za negativní.
3. Pro každé takové rozdělení spočítej dvojici (FPR, TPR) . Je-li horní část prázdná, dostaneme bod $(0, 0)$; je-li prázdná dolní část, bod $(1, 1)$.
4. Spoj sousední body.

AUC — plocha pod ROC křivkou

Dva klasifikátory se mohou v ROC prostoru *křížit*: v příkladu ze slidů je pro $FPR < 0,43$ lepší C_1 a pro $FPR > 0,43$ lepší C_2 — jednoznačná odpověď „který je lepší“ tedy neexistuje. Celkově lze použít *plochu pod ROC křivkou* (*area under the curve*, AUC): je-li $AUC(C_i) > AUC(C_j)$, říkáme, že C_i je lepší než C_j . Perfektní klasifikátor má $AUC = 1$, náhodné hádání $AUC = 0,5$.

5.3 Vizualizační techniky

♦ nad rámec slidů: tabulka jako přehled; krabicový graf, dendrogram a ROC/lift křivka samy ve slidech jsou

Technika	K čemu	Omezení / poznámka
Histogram	rozdělení jedné numerické veličiny	citlivý na volbu šířky košů
Krabicový graf (<i>box plot</i>)	medián, kvartily, odlehlé hodnoty; srovnání skupin	neukáže bimodalitu (řeší <i>violin plot</i>)
Bodový graf (<i>scatter plot</i>)	vztah dvou numerických veličin	při mnoha bodech překryv → průhlednost, hexbin
Matice bodových grafů (SPLOM)	všechny dvojice atributů najednou	použitelné do zhruba 10 atributů
Paralelní souřadnice	vysokorozměrná data — objekt je lomená čára přes svislé osy atributů	silně závisí na pořadí os; nutné škálování; křížení čar = negativní korelace
Teplotní mapa (<i>heatmap</i>)	korelační matice, matice záměn	volba barevné škály (divergentní pro korelace)
Dendrogram	výsledek hierarchického shlukování	pořadí listů není jednoznačné
Projekce (PCA, MDS, t-SNE, UMAP)	2-D pohled na vysokorozměrná data	t-SNE/UMAP zkreslují globální vzdálenosti — <i>velikosti</i> shluků a mezery mezi nimi nelze interpretovat
ROC / lift křivka	kvalita klasifikátoru	viz odd. 5.2
Mozaikový graf	kontingenční tabulka kategoriálních atributů	vizuální alternativa k χ^2
Vizualizace sítě	sociální sítě, grafy vztahů	„chlupatá koule“ u velkých grafů → agregace komunit

Tři role vizualizace v KDD: (1) *explorativní analýza* (fáze porozumění datům) — odhalí rozdělení, odlehlé a chybějící hodnoty, nečekané vztahy; Anscombeho kvartet ukazuje čtyři datové sady s identickými průměry, rozptyly a korelacemi, ale zcela odlišnou strukturou, kterou odhalí jedinečně

graf; (2) *vizualizace modelu* — strom, dendrogram, síť, hranice rozhodování; (3) *prezentace výsledků manažerovi* — agregace, jednoduchost, jeden graf = jedno sdělení.

Zásady dobré vizualizace: vysoký poměr datové a inkoustové složky (Tufta); nezkreslovat (nulová osa u sloupcových grafů, lineární osy nebo zřetelně označené logaritmické); barvu volit podle typu dat (kategorie → kvalitativní paleta, pořadí → sekvenční, odchylka od středu → divergentní) a s ohledem na barvosleposti; popisky a jednotky vždy; vyhnout se 3-D efektům a koláčovým grafům s mnoha výsečemi.

5.4 Vyhodnocení znalostí z pohledu uživatele

Pojem „vyhodnocování získaných znalostí“ má tři různé roviny, které je třeba rozlišovat:

- *statistické vyhodnocení modelu* — přesnost, matice záměn, ROC/AUC, odhad chyby křížovou validací (odd. 5.2);
- *vyhodnocení zajímavosti jednotlivých vzorů* — support, confidence, lift, χ^2 , t-weight a d-weight (odd. 3.1 a kap. 4);
- *vyhodnocení z pohledu uživatele* — zda je nalezená znalost srozumitelná, platná, užitečná a nová.

První dvě roviny jsou objektivní a měřitelné, třetí je nutně subjektivní — tentýž vzor může být pro jednoho uživatele objevem a pro druhého notorií. Část výsledků může být nezajímavá nebo z hlediska uživatele zřejmá, část lze použít přímo a část je nutné prezentovat srozumitelněji. Bývá užitečné shrnout výsledky do *analytické zprávy*; výstupem může být i provedení rozumné akce.

5.5 Shrnutí ke zkoušce

- **Formy reprezentace znalostí:** IF-THEN pravidla (nejsrozumitelnější, lokální), rozhodovací strom (globální, čitelný do hloubky 4–5), rozhodovací tabulka, matematická funkce (kompaktní, neinterpretovatelná), prototypy/centroidy, pravděpodobnostní model, grafy a sítě, instance (k -NN, CBR). Umět ke každé formě říct, z jaké metody vzniká a čím se platí za její srozumitelnost.
- **Kritéria hodnocení metod:** prediktivní přesnost, efektivita (čas konstrukce i použití), robustnost, škálovatelnost, interpretovatelnost, kompaktnost modelu.
- **Odhad chyby:** *holdout* (velká data; trénovací a testovací množina se nesmí míchat), *k-násobná křížová validace* (menší data; 5 a 10 je běžné; přesnost jako průměr k přesností + interval spolehlivosti s $t_{N,k-1}$), *leave-one-out* (velmi malá data, m -násobná CV), *validační množina* (třetí část dat, na ní se ladí parametry algoritmu).
- **Matice záměn:** TP , FN , FP , TN ; $p = \frac{TP}{TP+FP}$, $r = \frac{TP}{TP+FN}$, $F_1 = \frac{2pr}{p+r}$ (harmonický průměr — blíží menšímu z čísel). Umět vysvětlit, proč accuracy nestačí u nevyvážených dat (1 % průniků $\Rightarrow$ 99 % přesnosti „nedělat nic“).
- **ROC:** $TPR = \frac{TP}{TP+FN}$ (senzitivita = recall pozitivní třídy) proti $FPR = \frac{FP}{FP+TN}$; $TNR = \text{specificita} = 1 - FPR$. Křivka vede z (0,0) do (1,1), diagonála = náhodné hádání. Umět popsat konstrukci ze žebříčku (dělení na každém případě). **AUC:** 1 pro perfektní, 0,5 pro náhodný klasifikátor; potřebná proto, že se ROC křivky dvou klasifikátorů mohou křížit.
- **Skórování a lift:** místo třídy se přiřadí odhad pravděpodobnosti (u stromu confidence v listu), vzory se seřadí a rozdělí do příhrádek; lift křivka proti diagonále náhodného výběru. Umět provést ekonomickou úvahu (kolik zásilek se vyplatí).
- **Tři roviny „vyhodnocování“:** statistická kvalita modelu, zajímavost vzorů, vyhodnocení uživatelem. První dvě objektivní, třetí subjektivní. **Čtyři kritéria zajímavosti znalosti:** srozumitelnost, platnost, užitečnost, novost.
- **Vizualizace:** technika podle úlohy (histogram, box plot, scatter, SPLOM, paralelní souřadnice, heatmapa, dendrogram, projekce, ROC/lift); tři role v KDD (explorace — Anscombeho kvartet, vizualizace modelu, prezentace); zásady (Tufta, nezkreslovat osami, barva podle typu dat). Znáť omezení: t-SNE a UMAP zkreslují globální vzdálenosti.

6 Modely pro analýzu sociálních sítí, centralita a detekce komunit

Věta okruhu jmenuje tři věci: *modely* pro analýzu sociálních sítí, *míry centrality* a *detekci komunit*. Kapitola je tak členěna: nejprve reprezentace sítí a míry na úrovni aktéra (centralita) a sítě (koheze), pak modely — vlastností reálných sítí, šíření vlivu, homofilie, segregace a strukturní balance, plus analýza vazeb (HITS, PageRank) — a nakonec detekce komunit a predikce vazeb.

6.1 Vymezení oboru a motivace

Analýza sociálních sítí

Analýza sociálních sítí (*social network analysis*, SNA) je interdisciplinární obor s původem ve třech oblastech:

- *sociální vědy* — otázky a problémy vysvětlující sociální chování analýzou společností;
- *analýza sítí* — formulace a řešení problémů, které mají síťovou strukturu;
- *teorie grafů* — matematický aparát pro reprezentaci a analýzu grafů.

Zásadní posun perspektivy: SNA se soustředí na **vztahy** mezi sociálními entitami (jednotlivci, skupinami, institucemi) *spíše než na entity samotné* a jejich atributy. Studium společnosti ze síťové perspektivy chápe jednotlivce jako *zasazené* do sítě vztahů a hledá vysvětlení sociálního chování ve struktuře těchto sítí, nikoli v jednotlivcích samotných.

Analýza vazeb (*link analysis*) jako motivace: cílem je najít vztahy mezi daty a vizualizovat vazby. Aplicační oblasti: telekomunikace; kriminalistika a právo (skupiny zločinců jsou vzájemně propojené, analýza vztahů může pomoci je odhalit); marketing (vztahy mezi zákazníky).

Šest stupňů odloučení (Milgram, 1967): experiment o fenoménu malého světa. Náhodně vybraní lidé z Nebraska měli poslat dopis (přes prostředníky) burzovnímu makléři v Bostonu; poslat směli jen někomu, koho znali osobně (na křestní jméno). Mezi dopisy, které cíl našly, byl průměrný počet vazeb *šest*. Do cíle však mnoho dopisů nedorazilo. Experiment předznamenal pozdější objevy — huby, tendenci postupovat podle geografie či profese. Dokládají to i *Erdősova čísla*: Paul Erdős napsal přes 1500 článků s 507 spoluautory, průměrné Erdősovo číslo mezi matematiky je 4,65, maximum 13.

Bezškálové sítě (*scale-free*, SF) jako druhá motivace: některé uzly mají extrémně mnoho spojení (*huby*), většina uzlů jen několik. Důsledkem je *odolnost proti náhodné poruše* a zároveň *zranitelnost koordinovaným útokem*: může selhat až 80 % náhodně vybraných uzlů bez fragmentace, ale eliminace 5–15 % *hubů* způsobí rozpad celého systému. Aplikace: ochrana proti šíření počítačových virů internetem (vymýcení internetových virů je v SF architektuře v podstatě nemožné), medicína (vaccinační kampaně cílené na huby; nové léky mířící na klíčové molekuly), business (kaskádové finanční krachy; marketing — šíření „nákazy“, influencer marketing).

Příklady SF sítí: *sociální* (vědecká spolupráce — spoluautorství článků; Hollywood — herci ve stejném filmu), *biologické* (metabolismus buňky, proteinové regulační sítě), *sociotechnické* (internet — routery a spoje; WWW — stránky a URL).

6.2 Reprezentace sítí

Základní reprezentace

Sít' je graf $G = (V, E)$: *vrcholy* (uzly) jsou aktéři, *hrany* (vazby) vztahy mezi nimi. Tři ekvivalentní zápisy:

- **obrázek grafu**;
- **seznam hran** (*edge list*) — dvojice vrcholů, u ohodnocených grafů plus sloupec vah;
- **matice sousednosti** (*adjacency matrix*) $A = (a_{ij})$, $a_{ij} = 1$, je-li vrchol i spojen s vrcholem j , jinak 0; u vážených grafů obsahuje váhy.

U **orientovaných** grafů (kdo koho kontaktuje) matice sousednosti *nemusí* být symetrická; u **neorientovaných** (kdo koho zná) symetrická je. Tentýž seznam hran má v obou případech jinou

interpretaci.

Váhy hran mohou vyjadřovat: frekvenci interakce v období pozorování; počet vyměněných předmětů; individuální percepci silné vazby; náklady komunikace nebo výměny (např. vzdálenost); kombinaci předchozích. Hrany mohou reprezentovat interakce, toky informací či zboží, podobnosti a afiliace, nebo sociální vztahy.

Ego sítě a „celé“ sítě. *Ego síť* je osobní síť jednoho aktéra (*ego*) a jeho sousedů (*alterů*); pro další analýzu *není* ego potřebné, protože všichni *alteri* jsou s *egem* spojeni z definice. „Celá“ síť obsahuje i izolované uzly (*isolates*). **Žádná studovaná síť není v praxi „celá“** — je to obvykle jen částečný obraz skutečné sítě (*problém specifikace hranic, boundary specification problem*).

6.3 Síla vazeb

Sociální média propojují uživatele extrémně snadno — uživatel může mít online tisíce „přátel“ — ale vazby nejsou stejně silné a hrají různou roli při formování komunit a difuzi informací. *Silné vazby* jsou blízcí přátelé, *slabé vazby* známí. **Síla slabých vazeb** (Granovetter, 1973): občasná setkání se vzdálenými známými mohou poskytnout důležitou informaci o nových příležitostech (např. při hledání práce).

Homofilie, tranzitivita, přemostění

Homofilie (*homophily*) — tendence vztahovat se k lidem s podobnými charakteristikami (status, přesvědčení, ...). Vede k tvorbě homogenních skupin (klastřů), kde je snazší vytvářet vztahy; *extrémní* homogenizace ale může působit proti inovaci a generování idejí, takže *heterofilie* je v některých kontextech žádoucí. Homofilní vazby mohou být silné i slabé.

Tranzitivita (*transitivity*) — „existuje-li vazba mezi *A* a *B* a mezi *B* a *C*, budou v tranzitivní síti spojeni i *A* a *C*.“ Silné vazby jsou tranzitivní častěji než slabé, takže tranzitivita *indukuje* existenci silných vazeb (není to ale nutná ani postačující podmínka). Tranzitivita a homofilie společně vedou ke vzniku *klik* (plně propojených klastřů).

Přemostění (*bridging*) — *most* (*bridge*) je uzel či hrana spojující různé skupiny; obvykle jde o slabé vazby, ale ne každá slabá vazba je most. Mosty usnadňují mezigrupovou komunikaci, zvyšují sociální kohezi a podporují inovaci.

Určování síly vazby z topologie sítě. Mosty spojující dvě různé komunity jsou slabé vazby; hrana je *most*, jestliže její odstranění rozpojí její koncové uzly. Mosty jsou v reálných sítích poměrně vzácné $\Rightarrow$ definice se *uvolní* na **zkratkový most** (*shortcut bridge*): kontroluje se, zda se po odstranění hrany *zvětší* vzdálenost jejích koncových uzlů (na více než 2). Čím větší vzdálenost, tím *slabší* vazba. Např. $d(2, 5) = 4$ po odstranění hrany $(2, 5)$, ale $d(5, 6) = 2$ po odstranění $(5, 6) \Rightarrow (5, 6)$ je silnější vazba než $(2, 5)$.

Překryv sousedství (*neighborhood overlap*)

Pro dva uzly i, j spojené vazbou — čím větší překryv, tím *silnější* vazba:

$$O(i, j) = \frac{\text{počet společných přátel } i \text{ a } j}{\text{počet přátel, kteří jsou sousedy alespoň } i \text{ nebo } j} = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2},$$

kde -2 ve jmenovateli vylučuje i a j samotné.

Příklady: $O(2, 5) = 0$; $O(5, 6) = \frac{|\{4\}|}{|\{2, 4, 5, 6, 7, 10\}| - 2} = \frac{1}{4}$; pro graf $G = (\{1, 2\}, \{(1, 2)\})$ je $O(1, 2) = 0$.

Určování síly vazby z profilů a interakcí. Na Twitteru (lze sledovat jiné bez jejich potvrzení) je skutečná síť přátelství určena *frekvencí*, s jakou spolu dva uživatelé mluví, nikoli sítí follower–followee — a právě tato skutečná síť má silnější vliv na používání Twitteru. Na Facebooku lze sílu vazeb predikovat z různých informací (příspěvky iniciované přítelem, zprávy vyměněné na zdi, počet společných přátel). Numerickou sílu vazby lze učit např. metodou maximální věrohodnosti: podob-

nost profilů určuje sílu vazby a síla vazby určuje interakci uživatelů $\Rightarrow$ maximalizuje se věrohodnost na základě pozorovaných profilů a interakcí.

6.4 Klíčoví hráči: míry centrality

Stupňová centralita (*degree centrality*)

Vstupní a výstupní stupeň uzlu = počet vazeb vedoucích do uzlu / z uzlu. V neorientovaných grafech jsou oba stupně shodné.

Měří *propojenost*, *vliv* nebo *popularitu* uzlu; pomáhá zjistit, které uzly jsou centrální z hlediska šíření informace a ovlivňování ostatních ve svém „sousedství“.

Cesty a nejkratší cesty

Cesta mezi dvěma uzly je libovolná posloupnost neopakujících se uzlů, která je spojuje. *Nejkratší cesta* spojuje dva uzly nejmenším počtem hran (*vzdálenost* uzlů). Kratší cesty jsou preferované pro rychlou komunikaci či výměnu — ale ne vždy, například při šíření nemoci.

Algoritmy nejkratších cest. *Jeden zdroj:* obecné grafy (orientované, záporné váhy dovoleny) — Bellmanův–Fordův algoritmus, $O(|V||E|)$; grafy bez záporných vah — Dijkstrův algoritmus, $O(|V|^2)$; hledá-li se cesta jen mezi dvěma vrcholy, lze algoritmus zastavit po jejím nalezení. *Všechny páry:* záporné váhy dovoleny — Floydův–Warshallův, $O(|V|^3)$; řidší grafy se zápornými vahami — Johnsonův, $O(|E||V| + |V|^2 \log_2 |V|)$; bez záporných vah — Dijkstra, $O(|V|^3)$.

Mezilehlá centralita (*betweenness centrality*)

Počet nejkratších cest procházejících uzlem, dělený všemi nejkratšími cestami v síti: pro uzel v se pro každou dvojici s, t spočítá počet nejkratších cest mezi s a t procházejících v , vydělí se všemi nejkratšími cestami mezi s a t , a hodnoty se sečtou přes všechny dvojice; někdy se normalizuje tak, aby nejvyšší hodnota byla 1.

Indikuje uzly, které leží na *komunikačních cestách* mezi jinými uzly, a pomáhá určit místa, kde by se síť mohla *rozpadnout*.

Blízkostní centralita (*closeness centrality*)

Míra *dosahu* — rychlost, s jakou se informace z daného uzlu dostane k ostatním: spočítá se *průměrná* délka všech nejkratších cest z uzlu ke všem ostatním uzlům sítě (kolik „skoků“ je v průměru potřeba), a vezme se *převrácená hodnota*. Vyšší hodnoty znamenají vyšší blízkost (jsou „lepší“). Užitečná, jde-li především o rychlost šíření.

Vlastní centralita (*eigenvector centrality*)

Je proporcionální součtu vlastních centralit všech uzlů přímo spojených s uvažovaným uzlem: uzel s vysokou vlastní centralitou je spojen s jinými uzly s vysokou vlastní centralitou. Metrika vychází z myšlenky, že moc a status aktéra jsou *rekurzivně* definovány mocí a statusem jeho sousedů; měří, jak dobře je aktér spojen s jinými *dobře propojenými* aktéry.

Formálně: pro $G = (V, E)$ s maticí sousednosti $A = (a_{i,j})$

$$x_i = \frac{1}{\lambda} \sum_{j \in M(i)} x_j = \frac{1}{\lambda} \sum_{j \in V} a_{i,j} x_j \iff Ax = \lambda x,$$

kde $M(i)$ je množina sousedů i . Existuje mnoho vlastních čísel λ , pro něž nenulové řešení existuje; *požadavek nezápornosti* všech složek vlastního vektoru však znamená, že žádanou míru centrality dává *jen největší* vlastní číslo. Vlastní vektor je určen až na společný násobek, takže je nutná normalizace (např. součet přes všechny vrcholy = 1 nebo = $|V|$).

Blíže souvisí s Google PageRank (odkazy ze silně odkazovaných stránek váží více, odd. 6.14).

Míra	Interpretace v sociálních sítích
Stupňová	Kolik lidí může tato osoba dosáhnout <i>přímo</i> ? — <i>Síť hudebních spoluprací</i> : s kolika lidmi spolupracoval?
Mezilehlá	Jak pravděpodobně leží tato osoba na nejpřímější cestě v síti? — <i>Síť špiónů</i> : přes koho nejspíš protéká většina informací?
Blízkostní	Jak rychle může tato osoba dosáhnout všech v síti? — <i>Síť sexuálních vztahů</i> : jak rychle se do zbytku sítě rozšíří přenosná nemoc?
Vlastní	Jak dobře je tato osoba spojena s jinými dobře propojenými lidmi? — <i>Citační síť</i> : který autor je nejvíce citován jinými dobře citovanými autory?

Množiny klíčových hráčů. Jednotlivé míry nestačí. V příkladu ze slidů je uzel 10 nejcentrálnější podle stupňové centrality, ale uzly 3 a 5 *společně* dosáhnou více uzlů; navíc je vazba mezi 3 a 5 *kritická* — při jejím přerušení se síť rozpadne na dvě izolované podsítě. Za jinak stejných podmínek jsou tedy uzly 3 a 5 *společně* pro síť důležitější než uzel 10 samotný. *Uvažovat množiny klíčových hráčů proto pomáhá.*

Strukturní ekvivalence

Vyjadřuje *podobnost* mezi aktéry v sociální síti; je založena na sousedech, které sdílejí (ekvivalentně na počtu identických vazeb). Dva aktéři jsou *strukturně ekvivalentní*, lze-li jejich pozice vzájemně vyměnit *bez změny celkové struktury* sítě. Přibližnou strukturní ekvivalenci lze použít pro hierarchické shlukování sociálních sítí.

6.5 Koheze: míry na úrovni sítě

Reciprocita, hustota, klastrovací koeficient

Reciprocita (*reciprocity*) — míra vzájemnosti a reciproční výměny v síti, související se sociální kohezí: podíl počtu *reciprokých* vztahů (hrana v obou směrech) na celkovém počtu vztahů v síti, kde dva vrcholy jsou „ve vztahu“, existuje-li mezi nimi alespoň jedna hrana. **Má smysl jen v orientovaných grafech.**

Hustota (*density*) — podíl počtu hran v síti na celkovém počtu *možných* hran:

$$\text{density} = \frac{|E|}{|V|(|V| - 1)/2} \quad (\text{neorientovaný graf}), \quad \frac{|E|}{|V|(|V| - 1)} \quad (\text{orientovaný}).$$

Běžná míra toho, jak dobře je síť propojená (jak „těsně upletená“ je). Perfektně propojená síť se nazývá *klika* a má hustotu 1. Orientované grafy mají poloviční hustotu než jejich neorientované protějšky, protože možných hran je dvakrát více.

Klastrovací koeficient (*clustering coefficient*) uzlu i měří hustotu jeho *sousedství* — sítě tvořené pouze uzly přímo spojenými s i . Pro $G = (V, E)$ je sousedství $N_i = \{v_j : e_{ij} \in E \vee e_{ji} \in E\}$, $k_i = |N_i|$, a

$$C_i = \frac{2|\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}|}{k_i(k_i - 1)} \quad (\text{neorientovaný}), \quad C_i = \frac{|\{e_{jk}\}|}{k_i(k_i - 1)} \quad (\text{orientovaný}).$$

Klastrovací koeficient sítě C je průměr koeficientů všech uzlů, $C = \frac{1}{|V|} \sum_{i=1}^{|V|} C_i$. Indikuje přítomnost (sub)komunit; shlukovací algoritmy se snaží maximalizovat počet hran padnoucích do téhož klastru.

Průměr, průměrná vzdálenost, excentricita, malé světy

Průměr sítě (*diameter*) — nejdelší nejkratší cesta (vzdálenost) mezi libovolnými dvěma uzly. Užitečná míra *dosahu* sítě: udává, jak dlouho nejdéle trvá dosáhnout libovolného uzlu. Řidší sítě mají obecně větší průměr.

Průměrná vzdálenost — průměr všech nejkratších cest v síti; udává, jak daleko od sebe budou

dva uzly v průměru.

Excentricita vrcholu — největší geodetická vzdálenost mezi daným vrcholem a libovolným jiným vrcholem grafu.

Malý svět (*small world*) — síť, která vypadá téměř náhodně, ale má *významně vysoký klastrovací koeficient a relativně krátkou průměrnou délku cesty*: uzly se lokálně klastrují a zároveň jsou dosažitelné v několika krocích. V sociálních sítích je to velmi častý jev, a to kvůli *tranzitivitě* silných vazeb a *schopnosti slabých vazeb dosáhnout přes klastry*. Sítě malého světa mají mnoho klastrů, ale i mnoho mostů mezi nimi, které zkracují průměrnou vzdálenost.

6.6 Model reálných komplexních sítí a jejich vlastnosti

Šest charakteristických vlastností reálných komplexních sítí

Reálné komplexní sítě jsou *nenáhodné a neregulární* grafy s unikátními rysy. Příklady: sociální sítě, informační a znalostní sítě, technologické sítě, biologické sítě. Vlastnosti:

1. **Efekt malého světa** — na existenci malých světů v reálných sociálních sítích poukázal první Stanley Milgram (experiment z 60. let, cca 300 účastníků; mediánová délka úspěšných cest byla 6). Vychází z konceptu „šesti stupňů odloučení“: každý je od každého v průměru šest kroků. Důsledky pro rychlost šíření informací i nemocí.
2. **Tranzitivita / klastrování** — v reálných, zejména sociálních sítích je vysoká pravděpodobnost nalezení *úplných podsítí* uvnitř větších sítí (každý zná každého).
3. **Mocninné rozdělení stupňů** — *rozdělení stupňů* je rozdělení pravděpodobnosti stupňů uzlů přes celou síť (histogram podílu uzlů se stupněm k). Reálné sítě mají rozdělení stupňů zcela odlišné od náhodných grafů.
4. **Odolnost sítě** (*resilience*) — vliv na propojenost sítě při odstranění jednoho či více uzlů. Většina sítí je *robustní* proti náhodnému odstranění, ale podstatně méně proti *cílenému* odstranění uzlů s nejvyšším stupněm. Odstraní-li se „vrátní“ (*gatekeepers*), výrazně se změní komunikační schopnost sítě, protože některé uzly se odpojí. Odstranění jediného uzlu obvykle strukturu neohrozí — vhodnější je testovat odolnost odstraněním určitého procenta uzlů.
5. **Vzorce míchání** (*mixing patterns*) — koexistují-li v síti různé typy uzlů, je běžná určitá *selektivita* ve tvorbě spojení. Selektivní propojování se nazývá *asortativní míchání* (*assortative mixing*) čili homofilie; klasickým příkladem je míchání podle rasy. Reálné sítě vykazují vyšší tendenci k asortativnímu míchání.
6. **Komunitní struktura** — většina reálných sociálních sítí vykazuje komunitní strukturu, která vzniká jako důsledek *globální i lokální heterogenity* rozložení hran: v některých oblastech grafu nacházíme vysoké koncentrace hran (*komunity*) a mezi těmito oblastmi koncentrace nízké.

Bezškálové sítě a preferenční připojování

Bezškálové sítě (Barabási, Albert, 1999) jsou sítě, jejichž rozdělení stupňů následuje *mocninný zákon* (WWW, citační sítě, biologické sítě, některé sociální sítě). Mocninná rozdělení vznikají typicky tehdy, *závisí-li množství, které něčeho dostanete, na množství, které už máte*; topologicky se to projeví jako síť s několika silně propojenými uzly (*huby*) a velkým počtem uzlů s nízkým stupněm.

Mechanismus se nazýval *rich-get-richer / poor-get-poorer* strategie, ekvivalentně *Matoušův efekt*; Price jej nazval *kumulativní výhoda*, Barabási a Albert **preferenční připojování** (*preferential attachment*). Vysvětlení: nové uzly vstupující do sítě se připojují k dobře propojeným uzlům, které jsou často spojeny s centrálními a prestižními pozicemi (více statusu, popularity, znalostí, peněz).

Dva hlavní mechanismy vzniku SF sítě jsou tedy **růst** a **preferenční připojování**. Výsledná síť má *dlouhý chvost* rozdělení stupňů a *strukturu malého světa*; tranzitivita a vlastnosti silných/slabých vazeb jsou běžné a mohou vést ke struktuře malého světa, nejsou k jejímu vysvětlení ale

nutné.

Důvody preferenčního připojování: *popularita* („bohatí bohatnou“) — chceme být spojeni s populárními lidmi, idejemi a věcmi, což jejich popularitu dále zvyšuje, bez ohledu na objektivní měřitelné charakteristiky; *kvalita* („dobří se zlepšují“) — hodnotíme podle objektivních kritérií kvality, takže kvalitnější uzly přirozeně přitáhnou pozornost rychleji; *smíšený model* — „nelze předpovědět, kdo se stane hvězdou, i když kvalita hraje roli“: mezi uzly s podobnými atributy se „hvězdami“ stanou ty, které první dosáhnou kritického množství (*halo efekt*).

Struktura jádro-periferie a centralizace

Centralizace (*centralization*) měří, do jaké míry je síť centralizovaná či decentralizovaná; vychází z *rozdílů ve stupních* mezi uzly a obvykle se normalizuje do $[0, 1]$. Síť, která silně závisí na 1–2 vysoce propojených uzlech (např. jako důsledek preferenčního připojování), vykazuje větší rozdíly ve stupňové centralitě. Centralizované struktury mohou být lepší v některých úlohách vyžadujících koordinaci (např. týmové řešení problémů), ale jsou *náchylnější k selhání*, odpojí-li se klíčoví hráči.

Mnoho velkých skupin a komunit má *jádro* hustě propojených uživatelů, které je kritické pro propojení mnohem větší *periferie*. Jádra lze identifikovat vizuálně, nebo zkoumáním polohy uzlů s vysokým stupněm a jejich společných rozdělení stupňů; pro rozlišení jádra a periferních elementů lze použít i *bow-tie analýzu* (použitou pro analýzu struktury webu).

6.7 Modelování vlivu

Společné vlastnosti metod modelování vlivu

- sociální síť je reprezentována *orientovaným* grafem, každý aktér je jeden uzel;
- každý uzel začíná jako *aktivní* nebo *neaktivní*;
- aktivovaný uzel aktivuje své sousedy;
- **jednou aktivovaný uzel už nelze deaktivovat.**

Model lineárního prahu (LTM)

Aktér (uzel) podnikne akci, pokud počet jeho přátel, kteří akci podnikli, překročí určitý *prah*.

- Každý uzel v si zvolí prah θ_v *náhodně* z uniformního rozdělení na $[0, 1]$.
- V každém diskrétním kroku zůstávají všechny uzly aktivní z předchozího kroku aktivní.
- Aktivují se uzly splňující

$$\sum_{u \in N_v^{\text{aktivní}}} w_{u,v} \geq \theta_v \sum_{u \in N_v} w_{u,v}.$$

LTM se soustředí na *příjemce* vlivu.

Model nezávislých kaskád (ICM)

Soustředí se na *vysílající* uzly.

- Uzel v aktivovaný v kroku t má *jedinou* příležitost aktivovat každého ze svých sousedů, a to náhodně.
- Pro sousední uzel u uspěje aktivace s pravděpodobností $p_{v,u}$ (např. $p = 0,5$).
- Uspěje-li, u se stane aktivním v kroku $t + 1$.
- V dalších kolech se už v o aktivaci u *nepokouší*.
- Difuze začíná počáteční aktivovanou množinou uzlů a pokračuje, dokud není možná žádná další aktivace.

Maximalizace vlivu

Úloha: pro danou síť a parametr k vybrat k uzlů do aktivační množiny B tak, aby se maximalizoval vliv ve smyslu počtu aktivních uzlů na konci. Označme $\sigma(B)$ očekávaný počet uzlů

ovlivněných množinou B :

$$\max_{|B| \leq k} \sigma(B).$$

Úloha je **NP-těžká**, ale lze dokázat, že *hladový* přístup dává řešení na úrovni **63 %** optima.

Hladový přístup: začni s $B = \emptyset$; vyhodnot $\sigma(\{v\})$ pro každý uzel a vyber uzel s maximálním σ jako první prvek B ; poté opakovaně vybírej uzel, který $\sigma(B)$ zvýší nejvíce, tj. $\arg \max_{v \in V \setminus B} \sigma(B \cup \{v\})$.

6.8 Vliv versus korelace

Atributy a chování uživatelů často korelují s jejich sociální sítí. Uvažujme binární atribut u každého uzlu (např. zda je kuřák): koreluje-li atribut se sítí, budou aktéři sdílet stejné hodnoty atributu pozitivně korelovaně se sociálními spoji (kuřáci interagují spíše s kuřáky).

Test korelace

Jestliže podíl hran spojujících uzly s *různými* hodnotami atributu je významně *nižší* než očekávaná pravděpodobnost, jde o důkaz korelace.

Příklad: jsou-li spoje nezávislé na kuřáctví, je podíl kuřáků $p = 4/9$ a nekuřáků $1 - p = 5/9$. Jedna hrana spojuje dva kuřáky s pravděpodobností $p \cdot p$, dva nekuřáky $(1 - p)(1 - p)$ a kuřáka s nekuřákem $2p(1 - p) = 2 \cdot \frac{4}{9} \cdot \frac{5}{9} = 49\%$. Ve skutečném grafu je tento podíl $2/14 = 14\% < 49\%$, takže síť **vykazuje určitý stupeň korelace** vzhledem ke kuřáctví.

Tři sociální procesy vysvětlující korelaci

- **Homofilie** — tendence propojovat se s těmi, kdo s námi sdílejí určitou podobnost („vrána k vráně sedá“).
- **Zmatení** (*confounding*) — korelace může být vytvořena i *vnějšími* vlivy prostředí („jednotlivci žijící ve stejném městě se spíše stanou přáteli než náhodní jednotlivci“).
- **Vliv** (*influence*) — proces způsobující korelaci chování mezi sousedními aktéry („přejde-li většina mých přátel k jinému mobilnímu operátorovi, mohu být jimi ovlivněn a přejít také“).

Proč vliv rozlišovat od homofilie a zmatení? *Analýza:* predikce dynamiky systému — zda se nová norma chování, technologie nebo idea může šířit jako epidemie. *Návrh:* konstrukce systému vyvolávajícího konkrétní chování, např. vakcinační strategie (náhodné, cílené na demografickou skupinu, náhodní známí) nebo virální marketingové kampaně.

Logistický model vlivu a shuffle test

Ve studiích modelování vlivu se vliv určuje *časovými značkami*: pro hranu (u, v) může u ovlivnit v v diskretních krocích $t = 0, 1, 2, \dots$; v každém kroku se každý neaktivní uzel stane aktivním s pravděpodobností $p(a)$, kde a je počet aktivních kontaktů. Pravděpodobnost aktivace se modeluje jako *logistická* funkce počtu aktivních přátel:

$$p(a) = \frac{e^{\alpha \ln(a+1) + \beta}}{1 + e^{\alpha \ln(a+1) + \beta}}, \quad \text{ekvivalentně} \quad \ln \frac{p(a)}{1 - p(a)} = \alpha \ln(a + 1) + \beta,$$

kde a je počet aktivních přátel, α *koefficient sociální korelace*, β konstanta vysvětlující vrozenou tendenci k aktivaci a $\ln(a + 1)$ vysvětlující proměnná.

Věrohodnost aktivace. Necht v čase t se $y_{a,t}$ uživatelů s a aktivními přáteli stane aktivními a $n_{a,t}$ uživatelů s a aktivními přáteli zůstane neaktivních. Věrohodnost v čase t je $\prod_a p(a)^{y_{a,t}} (1 - p(a))^{n_{a,t}}$ a pro celou posloupnost $\prod_t \prod_a p(a)^{y_{a,t}} (1 - p(a))^{n_{a,t}}$. Z logu aktivit uživatelů lze spočítat α a β maximalizující tuto věrohodnost; α **měří korelaci mezi uzly**.

Shuffle test (test na vliv). Klíčová myšlenka: *nehraje-li vliv roli, musí být časování aktivace nezávislé na časování ostatních aktérů* — i když časové značky aktivit náhodně zpřeházíme, měli bychom dostat podobnou sociální korelaci. *Test:* zpřeházej časové značky aktivit uživatelů; je-li nový odhad sociální korelace *významně odlišný* od odhadu z původního logu, jde o důkaz **vlivu**.

Postup: (1) pro všechna a spočítej $y_{a,t}$ a $n_{a,t}$; (2) spočítej α, β maximalizující věrohodnost; (3) vyber náhodnou permutaci π časů aktivace a nastav čas aktivace uzlu v_i na $t'_i = t_{\pi(i)}$; z nových časů spočítej $y'_{a,t}$, $n'_{a,t}$; (4) spočítej α', β' maximalizující novou věrohodnost; (5) model *nevykazuje* sociální vliv, jsou-li α a α' blízko sebe.

6.9 Homofilie, výběr a sociální vliv

Test homofilie

Motivace: „vrána k vráně sedá“ — vaši přátelé jsou vám podobnější věkem, rasou, zájmy a názory než náhodně vybraná skupina jednotlivců. *Homofilie* je princip, že máme tendenci být podobní svým přátelům.

Test: kdyby byl každému uzlu *náhodně* přiřazen atribut podle poměru v reálné síti, počet „různorodých“ hran by se od reálné sítě významně nelišil. Je-li podíl p mužů a q žen, je pravděpodobnost hrany muž–muž p^2 , žena–žena q^2 a *různopohlavní* hrany $2pq$.

Je-li podíl heterogenních (různopohlavních) hran významně nižší než $2pq$, jde o důkaz homofilie.

Příklad: různopohlavních hran je 5 z 18; $p = 6/9 = 2/3$, $q = 3/9 = 1/3$; bez homofilie by jich mělo být $2pq = 4/9$, tj. 8 z 18 $\Rightarrow$ *důkaz homofilie*.

Poznámky k testu homofilie. „Významně nižší“ — vhodné je posuzovat odchylku pod střední hodnotou. Co když má síť *významně více* než $2pq$ různorodých hran? Pak jde o *inverzní homofilii* (příklad: partnerské vztahy muž–žena). Test lze použít na libovolnou charakteristiku (rasa, věk, rodný jazyk, preference); pro charakteristiky s více než dvěma hodnotami se počítá analogicky — počet heterogenních hran se porovná s tím, jak by vypadal náhodně generovaný graf s reálnými daty jako pravděpodobnostmi.

Výběr vs. sociální vliv

Tranzitivní uzávěr (*triadic closure*): mají-li dva jednotlivci společného přítele, je přátelství mezi nimi pravděpodobnější. Homofilie naznačuje, že dva jednotlivci jsou si podobnější *kvůli* společnému příteli, takže vazba může vzniknout, i když ani jeden o společném příteli neví. Obvykle je obtížné přisoudit vznik vazby jedinému faktoru.

Výběr (*selection*) — lidé mají tendenci vybírat si přátele, kteří jsou jim podobní. Působí na různých úrovních intencionality: aktivně si vybíráte podobné přátele v malé skupině; *nebo* je populace vaší školy relativně homogenní vůči celkové populaci, takže vás prostředí *nutí* vybírat si podobné přátele.

Mutabilní vs. imutabilní charakteristiky: *imutabilní* se nemění (pohlaví, rasa) nebo se mění konzistentně s populací (věk, generace); *mutabilní* se mění v čase (chování, přesvědčení, zájmy, názory).

Sociální vliv je *opakem* výběru: *výběr* — individuální charakteristiky řídí tvorbu vazeb; *sociální vliv* — existující vazby formují mutabilní charakteristiky lidí.

Longitudinální studie (dlouhé časové rozpětí) jsou nutné proto, že z jediného snímku sítě nelze rozhodnout, zda působí výběr, nebo sociální vliv: sledují sociální spoje i individuální chování v čase a odpovídají na otázku *mění se chování po změnách v síti, nebo se mění síť po změnách v chování?* Longitudinální studie se použily např. ke zjištění, zda měl na školní výsledky a užívání drog u teenagerů větší vliv výběr, nebo sociální vliv — to je podstatné pro návrh intervencí: vykazuje-li užívání drog v síti homofilii, může nejlépe fungovat program cílený na *sociální vliv* (přátelé přesvědčí ostatní přestat); vznikla-li však homofilie *výběrem*, bývalí uživatelé si vyberou nové přátele a chování ostatních to silně neovlivní.

Christakis a Fowler sledovali obezitu a sociální síť 12 000 lidí po 32 let a našli homofilii podle obezity. Možná vysvětlení: *efekty výběru* (vybírají si lidé přátele s podobným stavem?), *zmatení* (koreluje se stavem obezity i jiné faktory?), *sociální vliv* (změnil-li se stav přátel, ovlivnilo to budoucí stav dané

osoby?). **Nalezen významný důkaz pro třetí hypotézu:** obezita je typ „nákazy“, který se může šířit sociálním vlivem.

6.10 Afiliační sítě a uzávěrové procesy

Afiliační a sociálně-afiliační síť

Afiliační síť vkládá do sítě kontext tím, že zobrazuje spojení k aktivitám, společnostem, organizacím, čtvrtím atd. Je to *bipartitní graf*: každá hrana spojuje dva uzly patřící do *různých* množin (osoby vs. *afiliace* čili *fokusy*).

Sociálně-afiliační síť má *dva typy hran*: *osoba–osoba* (přátelství nebo jiný sociální vztah) a *osoba–fokus* (účast v daném fokusu).

Tři uzávěrové procesy

- **Tranzitivní uzávěr** (*triadic closure*) — dva lidé se spojí, protože mají společného přítele.
- **Fokální uzávěr** (*focal closure*) — dva lidé se spojí, protože mají společný fokus; uzávěr *výběrem*.
- **Členský uzávěr** (*membership closure*) — osoba se zapojí do fokusu, protože je v něm zapojen její přítel; uzávěr *sociálním vlivem*.

Otázky o uzávěru. Byla by Sue pravděpodobněji přítelkyní Alice, kdyby měly více než jednoho společného přítele? Formálněji: *jaká je pravděpodobnost, že dva lidé vytvoří vazbu, jako funkce počtu společných přátel*? Analogicky pro fokální uzávěr (jako funkce počtu společných fokusů) a členský uzávěr (jako funkce počtu přátel už v daném fokusu zapojených).

Metodika měření tranzitivního uzávěru

1. Pořídí dva snímky sítě v časech t_1 a t_2 .
2. Pro každé k identifikuj všechny *páry* uzlů, které mají v t_1 přesně k společných přátel, ale *nejdou* přímo spojeny hranou.
3. Definuj $T(k)$ jako *podíl* těchto párů, které do t_2 vytvořily hranu. To je pravděpodobnost, že mezi dvěma lidmi s k společnými přáteli vznikne vazba.
4. Vynes $T(k)$ jako funkci k .

Bázový (nulový) model: je-li pravděpodobnost vytvoření vazby s jedním společným přítelem p a jednotlivé společné přátele považujeme za *nezávislé* příležitosti, pak

$$T_{\text{base}}(k) = 1 - (1 - p)^k \quad (\text{resp. } 1 - (1 - p)^{k-1} \text{ podle konvence}).$$

Empirické výsledky. *E-mailová síť* (Kossinets, Watts, 2006): e-mailová komunikace 22 000 studentů velké americké univerzity po jeden rok; vazba mezi dvěma lidmi se předpokládala, byl-li mezi nimi za posledních 60 dní odeslán e-mail; snímky den po dni, $T(k)$ zprůměrováno přes více párů snímků. Zjištění: při *žádném* společném příteli se e-maily téměř nevyměňují; *významný nárůst* při přechodu z 1 na 2 společné přátele; další nárůst je významný, ale na výrazně menší subpopulaci — tj. *klesající výnosy*.

Fokální uzávěr tamtéž: z rozvrhů 22 000 studentů vznikla sociálně-afiliační síť, kde fokusy jsou vyučované předměty. Zjištění: *sdílení předmětu má téměř stejný efekt jako sdílení přítele*, opět s klesajícími výnosy.

Členský uzávěr: Backstrom et al. (2006) vytvořili sociálně-afiliační síť pro LiveJournal (přátelství podle profilů, fokusy = členství v uživatelsky definovaných komunitách) — významný nárůst od 1 ke 2, poté klesající výnosy. Crandall et al. (2008) totéž pro Wikipedii (uzly = editoři s diskusní stránkou, vazba = komunikovali přes diskusní stránku, fokusy = editované články) — tentýž vzorec.

Výběr a sociální vliv na datech Wikipedie. Podobnost dvou editorů se měřila jako

$$\frac{\text{počet článků editovaných oběma editory}}{\text{počet článků editovaných alespoň jedním editorem}}$$

(obdoba překryvu sousedství pro hrany). Zjištění: *páry editorů, které komunikovaly, jsou si v chování významně podobnější než ty, které nikdy nekomunikovaly*. Otázka zůstává, zda homofilie vzniká proto, že editoři mluví s editory téhož článku (*výběr*), nebo proto, že je k článkům dovedou ti, s nimiž mluví (*sociální vliv*). Analýza průměrovala historie interakcí (0 = okamžik vzniku vazby, jen editoři s alespoň 100 akcemi před i po první interakci, baseline = podobnost náhodného páru editorů) — podobnost roste už před vznikem vazby (*výběr*) a pokračuje v růstu i po něm (*vliv*).

6.11 Schellingův model segregace

Schellingův model (T. Schelling, 70. léta)

Jednoduchý model, který ukazuje, jak mohou *globální* vzorce dobrovolné segregace vznikat z *lokální* homofilie.

- Populaci tvoří dva typy jednotlivců — agenti *X* a *O*; oba typy reprezentují *imutabilní* charakteristiku (rasa, etnicita, země původu, rodný jazyk).
- Agenti jsou náhodně rozmístěni v mřížce.
- Agent je se svou polohou *spokojen*, je-li alespoň *t* jeho sousedů agentem téhož typu (*t* je předem nastavený prah).
- V každém kole se nejprve identifikují všichni *nespokojení* agenti; poté se všichni nespokojení přesunou na místo, kde budou spokojeni.

Někdy agent nové uspokojující místo nenajde — pak zůstane, nebo se přesune na náhodné místo. Přesuny mohou způsobit, že *dříve spokojený agent přestane být spokojen* — proto proces pokračuje.

Výsledek: mřížka je segregovanější než na začátku.

Varianty modelu: agenti se přesouvají v náhodném pořadí nebo „zametacím“ pohybem; přesun na nejbližší uspokojující místo nebo na náhodné; prah může být procentní nebo se lišit mezi skupinami i v rámci nich; svět se může „obtočit“ (levý okraj navazuje na pravý). Mnoho dalších variant, ale *výsledek zůstává stejný: samovolně vzniklá segregace*.

Závěr: Schellingův model je příkladem, jak se *imutabilní* charakteristiky mohou stát silně korelovanými s *mutabilními*: volba místa k životu (mutabilní) se v čase přizpůsobí typu agenta (imutabilní), čímž vzniká segregace (homofilie).

6.12 Pozitivní a negativní vztahy: strukturní balance

Dosud měl vztah v síti pozitivní konotaci; existují ale sítě, kde lze vyjádřit i negativní (antagonistické, nepřátelské) vztahy. *Pozitivní* vztahy = přátelské, *negativní* = nepřátelské. Otázka: **kdy je síť v balancovaném stavu?** Uvidíme, jak lokální vlastnosti vynucují globální (srov. Schellingův model). Zjednodušení: uvažujeme síť, kde *všechny* páry uzlů jsou spojeny pozitivním nebo negativním vztahem (tj. úplný graf, kliku) — vhodné pro malé skupiny lidí nebo pro státy; později uvolníme.

Vlastnost strukturní balance

Pro tři uzly existují (až na symetrii) čtyři konfigurace znamének:

Znaménka	Interpretace	Stav
+, +, +	<i>A, B, C</i> jsou vzájemní přátelé	<i>balancovaný</i>
+, −, −	<i>A</i> a <i>B</i> jsou přátelé se společným nepřítelem <i>C</i>	<i>balancovaný</i>
+, +, −	<i>B</i> a <i>C</i> jsou přátelé <i>A</i> , ale spolu ne vycházejí	<i>nebalancovaný</i>
−, −, −	<i>A, B, C</i> jsou vzájemní nepřátelé	<i>nebalancovaný</i>

Vlastnost strukturní balance: pro každý trojúhelník jsou všechny tři hrany pozitivní, nebo je pozitivní právě jedna.

Věta o bilanci

Věta. Je-li označovaný úplný graf balancovaný, pak buď

- jsou všechny páry uzlů přátelé, *nebo*
- lze uzly rozdělit do dvou skupin X a Y tak, že každý pár uzlů v X se má rád, každý pár v Y se má rád, a každý v X je nepřítelem každého v Y .

Důkaz. Nechť G je balancovaná síť. Jsou-li všechny hrany pozitivní, jsme hotovi. Existuje-li alespoň jedna negativní hrana, vyber uzel A a polož $X =$ všichni přátelé A (včetně A), $Y =$ všichni nepřátelé A . Pak platí: (1) každé dva uzly v X jsou přátelé — kdyby ne, měli bychom trojúhelník A + dva jeho přátelé s jedinou negativní hranou, tedy dvě kladné a jednu zápornou, což je nebalancované; (2) každé dva uzly v Y jsou přátelé — kdyby ne, měli bychom trojúhelník A + dva jeho nepřátelé se třemi negativními hranami, což je nebalancované; (3) každý uzel v X je nepřítelem každého uzlu v Y — jinak by trojúhelník A , přítel z X , nepřítel z Y měl dvě kladné a jednu zápornou hranu. $\square$

Aplikace strukturní balance. *Vývoj aliancí v Evropě 1872–1907* (plné tmavé hrany = přátelství, tečkované červené = nepřátelství): Liga tří císařů 1872–81, Trojspolek 1882, přerušení německo-ruské smlouvy 1890, francouzsko-ruská aliance 1891–94, Entente Cordiale 1904, britsko-ruská aliance 1907 — síť postupně sklouzává k balancovanému označování, a tím do první světové války. *Konflikt o odtržení Bangladéše od Pákistánu (1972)*: poněkud překvapivá podpora Pákistánu ze strany USA je méně překvapivá, uvědomíme-li si, že SSSR byl nepřítelem Číny, Čína nepřítelem Indie a Indie měla tradičně špatné vztahy s Pákistánem — USA tehdy zlepšovaly vztahy s Čínou, a tak podpořily nepřátele nepřátel Číny.

Slabá strukturní balance a generalizace

Vlastnost slabé strukturní balance: neexistuje trojice uzlů, jejíž hrany tvoří právě dvě pozitivní a jednu negativní. (Konfigurace „tři negativní“ je tedy dovolena — ta odpovídá situaci, kdy pár vytvoří alianci proti třetímu; „dvě pozitivní a jedna negativní“ je silnější nebalancovanost, protože B a C se snaží svou animozitu vyřešit.)

Věta. Je-li označovaný úplný graf slabě balancovaný, lze jeho uzly rozdělit do skupin vzájemných přátel tak, že každé dva uzly patřící do různých skupin jsou nepřátelé.

Důkaz. (1) Nechť A je uzel a X množina přátel A včetně A . (2) Všichni přátelé A jsou přátelé mezi sebou. (3) A a všichni jeho přátelé jsou nepřáteli všech ostatních. (4) Odstraň X ze sítě a pokračuj krokem 1. $\square$

Generalizace na neúplné grafy. (a) Ne všechny páry v síti jsou spojeny — (neúplný) graf je balancovaný, lze-li jej „doplnit“ přidáním hran na označovaný úplný graf, který je balancovaný. (b) Ne všechny trojúhelníky musí být balancované — je-li většina trojúhelníků balancovaná, lze síť přibližně rozdělit na dvě frakce.

6.13 Analýza vazeb: huby, autority a algoritmus HITS

Rok 1998 byl pro modely analýzy vazeb na webu zásadní — byly v něm publikovány oba algoritmy, PageRank i HITS. Souvislosti mezi nimi jsou nápadné; PageRank se od té doby stal dominantním modelem díky své nezávislosti na dotazu, schopnosti bránit se spamu a obchodnímu úspěchu Googlu.

Autority a huby

Autorita (*authority*) — hrubě řečeno stránka s mnoha vstupními odkazy. Idea: stránka může mít dobrý či autoritativní obsah k nějakému tématu, a proto jí mnoho lidí věří a odkazuje na ni.

Hub — stránka s mnoha výstupními odkazy. Slouží jako organizátor informací k danému tématu a odkazuje na mnoho dobrých autorit.

Klíčová myšlenka HITS: dobrý hub odkazuje na mnoho dobrých autorit a dobrá autorita je odkazována mnoha dobrými huby. Autority a huby jsou tedy ve vztahu vzájemného zesilování (bipartitní podgraf hustě propojených hubů a autorit).

Proč huby oddělovat od autorit? Konkurenční firmy na sebe neodkazují a nelze je chápat tak, že se přímo vzájemně podporují. V PageRanku naopak podpora přechází přímo z jedné prominentní stránky na druhou (stránka je důležitá, je-li citována jinými důležitými stránkami) — to je dominantní způsob podpory u akademických a vládních stránek, mezi blogery, u osobních stránek a ve vědecké literatuře.

Algoritmus HITS (*Hypertext Induced Topic Search*)

Na rozdíl od PageRanku, který je *statický*, je HITS *závislý na dotazu*. Po zadání dotazu HITS nejprve rozšíří seznam relevantních stránek vrácených vyhledávačem a pak vytvoří *dvě* pořadí rozšířené množiny: pořadí autorit a pořadí hubů.

Sběr stránek. Pro obecný dotaz q : pošli q vyhledávači; posbírej t nejvýše hodnocených stránek (v původním článku $t = 200$) — tato množina se nazývá *kořenová množina* S ; rozšiř S o každou stránku, na kterou odkazuje nějaká stránka z S , a o každou stránku, která odkazuje na stránku z S — vznikne větší *bázová množina* B .

Skóre. Nechť $G = (V, E)$ je hypertextový graf B o n stránkách s maticí sousednosti L ($L_{ij} = 1$, existuje-li hrana (i, j) , jinak 0). Označme $a(i)$ skóre autority a $h(i)$ skóre hubu stránky i . Vzájemné zesilování:

$$a(i) = \sum_{(j,i) \in E} h(j), \quad h(i) = \sum_{(i,j) \in E} a(j),$$

maticově, s $a = (a(1), \dots, a(n))^T$ a $h = (h(1), \dots, h(n))^T$:

$$a = L^T h, \quad h = La.$$

Výpočet mocninnou iterací:

- 1: $a_0 \leftarrow h_0 \leftarrow (1, 1, \dots, 1)^T$; $k \leftarrow 1$
- 2: **repeat**
- 3: $a_k \leftarrow L^T h_{k-1}$; $h_k \leftarrow La_k$
- 4: $a_k \leftarrow a_k / \|a_k\|_1$; $h_k \leftarrow h_k / \|h_k\|_1$ ▷ normalizace
- 5: $k \leftarrow k + 1$
- 6: **until** $\|a_k - a_{k-1}\|_1 < \varepsilon$ a $\|h_k - h_{k-1}\|_1 < \varepsilon$
- 7: **return** a a h

Normalizace je nezbytná, protože velikost hodnot autorit a hubů s každou aktualizací roste; konvergence nastane jen s normalizací (dělí se součtem všech skóre).

Spektrální analýza HITS

Po k krocích platí (spektrum matice = množina jejích vlastních čísel)

$$a_k = (L^T L)^{k-1} a_0, \quad h_k = (L L^T)^k h_0.$$

Vektor a_k tedy musí konvergovat k *vlastnímu vektoru* matice $L^T L$: existují normalizační konstanty c_k takové, že a_k/c_k konverguje k limitě, a směr se při násobení maticí nezmění právě tehdy, je-li vektor vlastním vektorem.

Matice $L^T L$ je *symetrická*, má tedy n vlastních vektorů $z_1, \dots, z_n$ s vlastními čísly $c_1, \dots, c_n$, $|c_1| \geq |c_2| \geq \dots \geq |c_n|$. Rozvineme-li $a_0 = x_1 z_1 + \dots + x_n z_n$, pak

$$a_k = c_1^k x_1 z_1 + \dots + c_n^k x_n z_n, \quad \frac{a_k}{c_1^k} = x_1 z_1 + x_2 \left(\frac{c_2}{c_1}\right)^k z_2 + \dots$$

Za předpokladu $c_1 > c_2 \geq \dots$ jde pro $k \rightarrow \infty$ každý člen kromě prvního k nule, takže a_k/c_1^k konverguje k $x_1 z_1$ — k *hlavnímu* vlastnímu vektoru. (Konvergenci nezávisle na počátečním vektoru a uvolnění předpokladu $c_1 > c_2$ zde nedokazujeme.)

Vztah ke ko-citaci a bibliografickému párování

Ko-citace stránek i a j (např. počet článků, které citují *oba* články i a j):

$$C_{ij} = \sum_{k=1}^n L_{ki} L_{kj} = (L^T L)_{ij}.$$

Matice autorit $L^T L$ algoritmu HITS je tedy *ko-citační matice*.

Bibliografické párování stránek i a j (počet článků, které jsou citovány *oběma* články i a j):

$$B_{ij} = \sum_{k=1}^n L_{ik} L_{jk} = (L L^T)_{ij}.$$

Matice hubů $L L^T$ je tedy matice bibliografického párování.

Silné a slabé stránky HITS. *Silná stránka:* schopnost řadit stránky *podle tématu dotazu*, což může poskytnout relevantnější autority a huby. *Slabé stránky:* (1) lze jej snadno spamovat — přidat výstupní odkazy na vlastní stránku je velmi snadné; (2) *drift tématu (topic drift)* — mnoho stránek v rozšířené množině nemusí být k tématu; (3) neefektivnost v době dotazu — sběr kořenové množiny, její rozšíření i výpočet vlastních vektorů jsou drahé operace.

6.14 PageRank

Prestiž pořadí (*rank prestige*)

U měř prestiže se uvažuje důležitý faktor: *prominence jednotlivých aktérů, kteří „hlasují“*. V reálném světě je osoba vybraná důležitou osobou prestižnější než osoba vybraná méně důležitou (hlas generálního ředitele váží více než hlas zaměstnance). Je-li váš okruh vlivu plný prestižních aktérů, je i vaše vlastní prestiž vysoká.

Prestiž pořadí $P(i)$ je definována jako lineární kombinace pořadí těch, kteří odkazují na i :

$$P(i) = A_{1i}P(1) + A_{2i}P(2) + \dots + A_{ni}P(n), \quad \text{maticově} \quad P = A^T P,$$

kde $A_{ji} = 1$, odkazuje-li j na i , jinak 0. To je *charakteristická rovnice* vlastního systému matice A^T ; P je vlastní vektor A^T .

Základní PageRank

PageRank interpretuje hyperodkaz ze stránky u na stránku v jako *hlas* stránky u pro stránku v . Nezajímá jej ale pouhý počet hlasů — analyzuje i stránku, která hlas dává: *hlasy důležitých stránek váží více a pomáhají činit ostatní stránky důležitějšími*. Protože stránka může odkazovat na mnoho jiných, musí své skóre *rozdělit*.

Web jako orientovaný graf $G = (V, E)$, $n = |V|$. PageRank stránky i :

$$P(i) = \sum_{(j,i) \in E} \frac{P(j)}{O_j},$$

kde O_j je počet výstupních odkazů j . Maticově: nechť $A_{ij} = 1/O_i$ pro $(i, j) \in E$ a 0 jinak; pak $P = A^T P$.

Intuice: PageRank je jako „kapalina“, která cirkuluje sítí, přechází od uzlu k uzlu po hranách a shromažďuje se v nejdůležitějších uzlech. *Celkový PageRank v síti zůstává konstantní* — každá stránka svůj PageRank rozdělí a pošle po odkazech dál; PageRank se nikdy nevytváří ani nezaniká, jen se přemísťuje. Proto *není nutná normalizace* — na rozdíl od HITS.

Problém základní definice. V mnoha sítích skončí veškerý PageRank ve „špatných“ uzlech.

Odkazují-li si F a G navzájem místo na A , PageRank, který k nim přiteče z C , se už nikdy nemůže vrátit — pro velká k dostaneme $\frac{1}{2}$ pro každý z F , G a 0 pro všechny ostatní. Jde o „pomalý únik“ (*slow leak*): malé množiny uzlů dosažitelné z ostatního grafu, ale bez cest zpět. **Řešení:** posílat malý zlomek PageRanku *všem* uzlům.

Škálovaný PageRank a tlumicí faktor

Zvol *škálovací faktor* d mezi 0 a 1 a nahraď základní pravidlo aktualizace:

1. nejprve aplikuj základní pravidlo;
2. všechny hodnoty PageRanku sniž faktorem d (celkový PageRank v síti se tím zmenší z 1 na d);
3. zbývajících $1 - d$ jednotek rozděl rovnoměrně mezi všechny uzly, tj. $(1 - d)/n$ každému.

Proč to funguje: je to „vodní cyklus“, který v každém kroku $1 - d$ jednotek PageRanku „odpaří“ a rovnoměrně je „sprchne“ na všechny uzly.

Opakovaná aplikace škálovaného pravidla *konverguje* k limitním hodnotám PageRanku a ekvilibrium je *jednoznačné* — hodnoty ale závisejí na volbě d . V praxi se používá d mezi 0,8 a 0,9 ($d = 0,85$ v původním článku); d se nazývá *tlumicí faktor* (*damping factor*).

PageRank jako markovský řetězec

Web se modeluje jako *stochastický proces*: každá stránka je *stav*, hyperodkaz je *přechod* s určitou pravděpodobností. *Náhodné surfování*: každá přechodová pravděpodobnost je $1/O_i$, předpokládáme-li, že surfař klikne na odkazy uniformně náhodně (nepoužívá tlačítko „zpět“ ani nezadáva URL).

Nechť A je matice přechodových pravděpodobností a p_0 počáteční rozdělení pravděpodobnosti. Pak $p_1 = A^T p_0$ a obecně $p_k = A^T p_{k-1}$. *Věta o markovských řetězcích*: konečný markovský řetězec definovaný stochastickou maticí A má **jednoznačné stacionární rozdělení**, je-li A *ireducibilní* a *aperiodická*. Stacionární rozdělení znamená, že p_k konverguje ke stálému vektoru π nezávisle na volbě p_0 . V ustáleném stavu $\pi = A^T \pi$, tedy π je hlavní vlastní vektor A^T s vlastním číslem 1 — a právě π se použije jako vektor PageRanku. Je to rozumné a intuitivní: odráží *dlouhodobé pravděpodobnosti*, že náhodný surfař stránku navštíví, a stránka má vysokou prestiž, je-li pravděpodobnost její návštěvy vysoká.

Ani jedna podmínka však pro webový graf neplatí:

- A *není stochastická* — mnoho stránek nemá výstupní odkazy, což se v matici projeví řádky samých nul (*visící stránky*, *dangling pages*). Řešení: (1) takové stránky během výpočtu odstranit (neovlivňují přímo pořadí žádné jiné stránky); (2) přidat z každé takové stránky úplnou množinu odkazů na všechny stránky, tj. $A_{ij} = 1/n$.
- A *není ireducibilní* — ireducibilita znamená, že webový graf je *silně souvislý* (pro každý pár i, j existuje cesta z i do j); obecný webový graf silně souvislý není.
- A *není aperiodická* — stav i je *periodický* s periodou $k > 1$, je-li k nejmenší číslo takové, že všechny cesty ze stavu i zpět do i mají délku, která je násobkem k ; není-li stav periodický ($k = 1$), je aperiodický, a řetězec je aperiodický, jsou-li aperiodické všechny stavy. Příklad periodického řetězce s $k = 3$: z 1 se lze vrátit do 1 jen cestou $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$, takže každý návrat trvá 3 přechody.

Obě posledně jmenované vlastnosti se řeší jedinou strategií: přidat z každé stránky odkaz na každou stránku a dát každému takovému odkazu malou přechodovou pravděpodobnost řízenou parametrem d — rozšířená přechodová matice je zjevně ireducibilní i aperiodická.

Finální PageRank

Po augmentaci má náhodný surfař na stránce dvě možnosti: s pravděpodobností d zvolí náhodně odkaz, který bude následovat, a s pravděpodobností $1 - d$ skočí na náhodnou stránku:

$$P = \left(\frac{1 - d}{n} \right) \mathbf{E} + d A^T P,$$

kde $\mathbf{E} = \mathbf{e}\mathbf{e}^\top$ ($\mathbf{e}$ je sloupcový vektor jedniček), tj. $\mathbf{E}$ je matice $n \times n$ samých jedniček. Matice $\left(\frac{1-d}{n}\right)\mathbf{E} + d\mathbf{A}^\top$ je stochastická (transponovaná), ireducibilní i aperiodická. Po přeskálování tak, aby $\mathbf{e}^\top P = n$:

$$P = (1-d)\mathbf{e} + d\mathbf{A}^\top P, \quad \text{tj. pro každou stránku } i: \quad P(i) = (1-d) + d \sum_{j=1}^n A_{ji}P(j),$$

což je ekvivalentní formuli z původního článku o PageRanku

$$P(i) = (1-d) + d \sum_{(j,i) \in E} \frac{P(j)}{O_j}.$$

Výpočet mocninnou iterací:

- 1: $P_0 \leftarrow \mathbf{e}/n; k \leftarrow 1$
- 2: **repeat**
- 3: $P_k \leftarrow (1-d)\mathbf{e} + d\mathbf{A}^\top P_{k-1}$
- 4: $k \leftarrow k + 1$
- 5: **until** $\|P_k - P_{k-1}\|_1 < \varepsilon$
- 6: **return** P_k

Výhody PageRanku. *Boj se spamem:* stránka je důležitá, jsou-li důležité stránky, které na ni odkazují — a protože pro majitele stránky není snadné přidat *vstupní* odkazy z jiných důležitých stránek, není snadné PageRank ovlivnit. *Globální a dotazově nezávislá míra:* hodnoty PageRanku všech stránek se počítají a ukládají *offline*, nikoli v době dotazu. **Kritika:** právě dotazová nezávislost — PageRank neumí rozlišit stránky autoritativní *obecně* od stránek autoritativních *k tématu dotazu*.

Souhrn. V rámci analýzy vazeb jsme viděli principy *centrality* a *prestíže*, *ko-citaci* a *bibliografické párování* a algoritmy *HITS* a *PageRank* (který stojí za Googlem). Yahoo! a MSN mají vlastní nepublikované algoritmy založené na odkazech. **Důležité:** hyperodkazové řazení není jediný algoritmus používaný ve vyhledávačích — kombinuje se s mnoha faktory založenými na obsahu, aby vzniklo finální pořadí prezentované uživateli.

6.15 Tematické komunity a jejich objevování

Odkazy lze použít i k nalezení *tematických komunit* — skupin tvůrců obsahu či lidí sdílejících nějaké společné zájmy (webové komunity, e-mailové komunity, komunity pojmenovaných entit). Souvisejícím pojmem je *cílené procházení* (*focused crawling*): kombinace obsahu a odkazů k procházení webových stránek konkrétního tématu — následují se odkazy a pomocí učení/klasifikace se rozhoduje, zda je stránka k tématu.

Tematická komunita

Pro danou konečnou množinu entit $E = \{e_1, \dots, e_n\}$ *téhož typu* je *tematická komunita* pár $K = (T, M)$, kde T je *téma* komunity a $M \subseteq E$ množina všech entit z E , které téma T sdílí. Je-li $e \in M$, říkáme, že e je *členem* komunity K .

- *Téma definuje komunitu:* pro dané téma T je množina členů určena jednoznačně, takže dvě komunity jsou si rovny, mají-li totéž téma.
- *Téma lze definovat libovolně:* může to být událost (sportovní událost, skandál) nebo koncept (dolování z webu).
- *Entita může být v libovolném počtu komunit:* komunity se tedy mohou *překrývat* a více komunit může sdílet členy.
- *Entity v E jsou téhož typu:* lidé a organizace tedy nemohou být v téže komunitě.

Komunity mohou tvořit *hierarchickou strukturu*: tematická komunita (T, M) může obsahovat množinu tematických *subkomunit* $\{(T_1, M_1), \dots, (T_k, M_k)\}$, kde T_i je podtéma T a $M_i \subseteq M$;

(T, M) je pak *superkomunita*. Každou subkomunitu lze dále dekomponovat.

Cíl objevování: pro danou datovou sadu obsahující entity najít skryté komunity — pro každou komunitu najít *téma* (obvykle reprezentované množinou klíčových slov) a *členy*.

Manifestace komunity v datech: členové komunity jsou nějak propojeni a přidružené texty obsahují slova indikativní pro téma komunity. *Webové stránky:* členové komunity jsou spíše vzájemně propojeni *hyperodkazy*; stránky obsahují *slova* vztažená k tématu. *E-maily:* členové spíše komunikují mezi sebou; e-maily obsahují slova vztažená k tématu. *Textové dokumenty:* členové se spíše objevují *spolu* v téže větě či dokumentu; slova ve větách indikují téma.

Bipartitní jádra komunit

(i, j) -bipartitní jádro je úplný bipartitní podgraf s alespoň i *vějířovými* uzly (*fans*) a alespoň j *středovými* uzly (*centers*), tj. obsahuje všechny možné hrany mezi i vějířovými a j středovými vrcholy. Vějíře reprezentují *specializované huby*, středy odpovídají *autoritám*.

Postup hledání bipartitních jader:

- *Krok 1 — prořezání.* Prořez podle vstupního stupně: odstraň všechny stránky, které jsou velmi silně odkazované (počet vstupních odkazů $> k$, např. $k = 50$). Iterativně prořezávej vějíře a středy (pro nalezení (i, j) -jader): odstraň všechny vějíře s výstupním stupněm menším než j ; odstraň všechny středy se vstupním stupněm menším než i ; odstraň z grafu hrany spojené s odstraněnými uzly.
- *Krok 2 — generování všech (i, j) -jader.* Najdi všechna $(1, j)$ -jádra, tj. množinu všech vrcholů s výstupním stupněm alespoň j ; zkonstruuj všechna $(2, j)$ -jádra kontrolou každého vějíře, který odkazuje také na některý střed v $(1, j)$ -jádrě; pokračuj stejným způsobem až do i .

Omezení: algoritmus najde jen *jádrové* stránky komunit, ne všechny členské stránky, a nenajde ani *témata* komunit, ani jejich hierarchickou strukturu.

Překrývající se komunity pojmenovaných entit

Cíl: najít komunity entit z textového korpusu. **Hlavní idea:** dvě entity jsou propojeny, *vyskytnou-li se v téže větě* (jsou-li dva lidé zmíněni v téže větě, obvykle mezi nimi existuje vztah); jedna entita se může objevit ve více komunitách; nejvýznamnější klíčová slova reprezentují téma komunity. Slibné výsledky při hledání komunit politických osobností a celebrit z webových dokumentů.

Algoritmus:

1. *Postav graf vazeb* — parsuj každý dokument a v každé větě identifikuj obsažené pojmenované entity; má-li věta více než jednu entitu, propoj tyto entity *po párech*; klíčová slova z věty připoj k propojeným párům jako textový obsah; ostatní věty zahod.
2. *Najdi všechny trojúhelníky* — trojúhelník tvoří tři entity svázané dohromady a reprezentuje základní stavební blok komunit; pozorování: komunita má tendenci se rozšiřovat trojúhelníky sdílejícími společnou hranu.
3. *Najdi jádra komunit* — jádro je skupina těsně svázaných trojúhelníků, tedy *uvolněný* úplný podgraf (klika); intuitivně jde o množinu těsně propojených členů komunity.
4. *Klastruj okolo jader* — trojúhelníky a páry entit, které v žádném jádru nejsou, se přiřadí k jádrům podle *podobnosti textového obsahu* s nalezenými jádry.

6.16 Detekce komunit

Komunita a detekce komunit

Komunity jsou tvořeny jednotlivci tak, že ti *uvnitř* skupiny interagují mezi sebou *častěji* než s těmi vně skupiny. V různých kontextech se jim říká také *skupina*, *klastr*, *kohezní podgrupa*, *modul*.

Detekce komunit je objevování skupin v síti, kde příslušnost jednotlivců ke skupinám *není explicitně dána*.

Definice komunity je subjektivní: hustě propojený podgraf může být komunitou, ale i každá komponenta souvislosti může být komunitou.

Dva typy skupin v sociálních médiích: *explicitní* skupiny tvořené předplacením uživatelů a *implicitní* skupiny tvořené sociálními interakcemi. Proč extrahovat skupiny z topologie, i když některé weby umožňují do skupin vstoupit? Ne všechny weby platformu pro komunity poskytují; ne všichni lidé jsou ochotni vynaložit úsilí na vstup do skupiny; skupiny se mohou *dynamicky měnit*. Interakce v síti poskytují informaci o vztazích mezi uživateli, doplňují ostatní druhy informací, pomáhají vizualizaci a navigaci a poskytují základ pro další úlohy.

Metody minimálního řezu

Minimální řez (Ahuja et al., 1993). Řez je rozdělení vrcholů grafu do dvou disjunktních množin; úloha *minimálního řezu* hledá takové rozdělení, které minimalizuje počet hran mezi oběma množinami (nebo součet vah u vážených sítí) — tzv. *velikost řezu*. Motivace: většina interakcí nastává *uvnitř* skupiny, mezi skupinami jsou vzácné $\Rightarrow$ detekce komunit $\approx$ úloha minimálního řezu.

Pro $k > 2$ se implementuje strategie *iterativního bisekce* (ve druhé a dalších iteracích se skupiny postupně dělí na dvě podskupiny).

Nevýhoda: produkuje skupiny nevyvážených velikostí (jedna množina často bývá jedno-prvková).

Složitost: jednoduché řešení pomocí *s-t řezu* založeného na maximálním toku má složitost $O(n^5)$; implementace Kargerovým algoritmem $O(n^4)$, ale nezaručuje vždy nalezení minimálního řezu. Pro jednotkové váhy $w_{ij} = 1$ a bez balančních omezení je úloha 2-cestného řezu polynomiálně řešitelná v $O(n^2 \log n)$ (Kargerův randomizovaný minimální řez). Obecné váhy hran a balanční omezení činí úlohu **NP-těžkou**.

Poměrový a normalizovaný řez

Minimální řez často vrací nevyvážené rozdělení (jedna množina je singleton) $\Rightarrow$ změníme účelovou funkci tak, aby brala v úvahu *velikost komunity*. Pro rozdělení $\pi = \{C_1, \dots, C_k\}$:

$$\text{RatioCut}(\pi) = \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{|C_i|}, \quad \text{NCut}(\pi) = \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{\text{vol}(C_i)},$$

kde $|C_i|$ je počet uzlů v C_i , $\text{vol}(C_i)$ součet stupňů v C_i a $\text{cut}(A, B) = \sum_{i \in A, j \in B} a_{ij}$. **Obě míry preferují vyvážené rozdělení.**

Girvanové–Newmanův algoritmus (2002)

Divizivní hierarchický algoritmus (přístup shora dolů): dekomponuje počáteční úplný graf na postupně menší souvislé části, dokud nezůstanou žádné hrany k odstranění a každý uzel nereprezentuje sám komunitu.

Idea: identifikovat hrany spojující vrcholy z *různých* komunit (tzv. mosty) a iterativně je z grafu odstraňovat. **Kritérium:** *mezilehlost hrany* (*edge betweenness*) — podíl nejkratších cest procházejících danou hranou; hrany s vysokou mezilehlostí leží na velkém počtu nejkratších cest mezi vrcholy, a jsou tedy mosty.

- 1: spočítej mezilehlost všech hran v síti
- 2: odstraň hranu s *nejvyšší* mezilehlostí
- 3: opakuj předchozí kroky, dokud v grafu zbývají hrany k odstranění

Vstup: celý graf. **Výstup:** hierarchická struktura, reprezentovaná např. dendrogramem.

Volba počtu komunit: algoritmus vrátí *množinu* možných řešení, ale neurčí, které je nejlepší. K volbě nejlepšího rozdělení se pro každé dělení sítě spočítá *modularita* a vybere se rozdělení s nejvyšší modularitou.

Nevýhoda: vhodný pouze pro sítě *střední velikosti* kvůli vysokým výpočetním nákladům — $O(m^2n)$ pro graf o m hranách a n vrcholech, resp. $O(n^3)$ pro řídké sítě.

Příklad výpočtu mezilehlosti hrany: mezilehlost hrany $e(1, 2)$ je 4, protože všechny nejkratší cesty z 2 do $\{4, 5, 6, 7, 8, 9\}$ musí procházet buď $e(1, 2)$, nebo $e(2, 3)$, a $e(1, 2)$ je nejkratší cesta mezi 1 a 2. Po odstranění $e(4, 5)$ se mezilehlost $e(4, 6)$ zvýší na 20 (nejvyšší); po odstranění $e(4, 6)$ má nejvyšší mezilehlost 4 hrana $e(7, 9)$.

Modularita Q

Kvantifikuje kvalitu daného rozdělení grafu na komunity. **Základní idea:** síť má smysluplnou komunitní strukturu, je-li počet hran *mezi* komunitami menší, než by se očekávalo při náhodné volbě. Očekávaný počet hran mezi uzly i a j se stupni k_i, k_j je $k_i k_j / 2m$.

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

kde A_{ij} je prvek matice sousednosti (počet hran mezi i a j), m počet hran, k_i stupeň vrcholu i , c_i skupina, do níž i patří, a δ Kroneckerovo delta.

Hodnoty: $Q \in [-1, 1]$, může být pozitivní i negativní. Je-li pozitivní, existuje možnost nalézt v síti komunitní strukturu; jsou-li hodnoty velké a pozitivní, může odpovídající rozdělení odrážet skutečnou komunitní strukturu. Pro smysluplné komunity by mělo být $Q \geq 0,3$ (Clauset et al., 2004).

Louvainova metoda (Blondel et al., 2008)

Hladová optimalizační metoda pro *aglomerativní hierarchickou* maximalizaci modularity. Dvě fáze:

Fáze 1 — každý uzel se považuje za samostatnou komunitu:

1. spočítej modularitu Q ;
2. přesuň izolovaný uzel z jeho komunity do sousední komunity;
3. spočítej zisk/ztrátu modularity způsobenou změnou přiřazení;
4. zvýší-li se modularita (je-li zisk), ponech uzel v „nové“ komunitě;
5. opakuj, dokud není možné žádné další zlepšení.

Výstup: rozdělení k -té úrovně (k je číslo iterace).

Fáze 2 — vytvoř novou síť (*supergraf*) odvozenou z původní, kde každý nový uzel (*supervrchol*) je agregací uzlů přiřazených dané komunitě nalezené ve fázi 1; dva supervrcholy jsou spojeny hranou, existuje-li alespoň jedna hrana spojující dva vrcholy uvnitř odpovídajících komunit.

Iterace: opakuj fáze 1 a 2, dokud není dosaženo maximální modularity.

Výhody: dobrý výkon i na velkých síťových grafech při nízkých výpočetních nákladech — $O(n \log n)$, kde n je počet vrcholů. **Nevýhoda:** výsledky jsou *citlivé na pořadí* zpracování.

Další nevýhody: *nesouvislé komunity* — přesune-li se uzel do jiné komunity, může se původní komunita stát *vnitřně nesouvislou*, a přesto zůstanou její uzly lokálně optimálně přiřazeny, takže v ní zůstanou; *uváznutí v lokálních optimech*; *selhání v upřesnění komunit* po počátečním slučování.

Leidenův algoritmus (Traag, Waltman, van Eck, 2019)

Upřesnění Louvainovy metody pro *aglomerativní hierarchickou* maximalizaci modularity. Tři fáze: (1) lokální přesuny uzlů (obdobně jako Louvain); (2) **upřesnění rozdělení**; (3) agregace sítě (obdobně jako Louvain).

Výhody: efektivnější než Louvainova metoda, ale pro obrovské grafy může vést k delším dobám zpracování; paralelní vícejádrové implementace mohou rychlost zvýšit. **Nevýhoda:** *tvrdé rozdělení* — uzly mohou patřit jen do jedné komunity.

Výzvy detekce komunit. Přes velké množství algoritmů zůstává detekce komunit náročným problémem, a to ze dvou hlavních důvodů: (1) počet komunit obvykle není znám a je třeba jej určit; (2) komunity jsou obvykle *heterogenní* co do velikosti a hustoty.

6.16.1 Taxonomie metod detekce komunit

Metody lze rozdělit do čtyř kategorií (kritéria se mohou podle úlohy lišit): *uzlově orientované* (každý uzel ve skupině splňuje určité vlastnosti), *skupinově orientované* (spojení uvnitř skupiny jako celku splňují určité vlastnosti, bez zoomu na úroveň uzlů), *síťově orientované* (rozdělení celé sítě do několika disjunktních množin) a *hierarchicky orientované* (konstrukce hierarchické struktury komunit).

Uzlově orientovaná detekce komunit

Uzly mohou být požadovány, aby splňovaly různé vlastnosti používané v tradiční SNA:

Úplná vzájemnost — kliky. *Klika* je maximální úplný podgraf tří a více uzlů, z nichž všechny jsou navzájem sousední. Nalezení maximální kliky je **NP-těžké**; přímočará implementace je časově velmi drahá. *Idea rekurzivního prořezávání:* v klice velikosti k má každý uzel stupeň $\geq k - 1$, takže uzly se stupněm $< k - 1$ v maximální klice nebudou — opakovaně: vzorkuj podsít a najdi v ní kliku (např. hladově); je-li velikosti k , odstraň pro nalezení větší kliky všechny uzly se stupněm $\leq k - 1$; opakuj, dokud není síť dost malá. V sociálních médiích se prořeže mnoho uzlů, protože stupně uzlů následují mocninné rozdělení. Kliky se normálně používají jako *jádro* či *semeno* pro prozkoumání větších komunit.

Metoda perkolace klik (*clique percolation method*, CPM) — klika je striktní definice, v praxi nestabilní; CPM je příkladem metody pro nalezení *překrývajících se* komunit. *Vstup:* parametr k a síť. *Postup:* najdi všechny kliky velikosti k ; zkonstruuji *graf klik* — dvě kliky jsou sousední, sdílejí-li $k - 1$ uzlů; každá komponenta souvislosti grafu klik tvoří komunitu. *Příklad:* kliky velikosti 3 jsou $\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$, $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\} \Rightarrow$ komunity $\{1, 2, 3, 4\}$ a $\{4, 5, 6, 7, 8\}$ (uzel 4 je v obou).

Dosažitelnost — k -klika, k -klub, k -klan. Hlavní idea: každý uzel ve skupině má být dosažitelný v k skocích. k -klika je maximální podgraf, v němž je největší geodetická vzdálenost mezi libovolnými dvěma uzly $\leq k$ (pozor: k -klika může mít *uvnitř podgrafu* průměr $\geq k$ — např. v jiném příkladovém grafu ze slidů má 2-klika $\{12, 4, 10, 1, 6\}$ $d(1, 6) = 3$). k -klub je podstruktura s průměrem $\leq k$. k -klan je k -klika s průměrem $\leq k$. *Příklad:* kliky $\{1, 2, 3\}$; 2-kliky $\{1, 2, 3, 4, 5\}$, $\{2, 3, 4, 5, 6\}$; 2-kluby $\{1, 2, 3, 4\}$, $\{1, 2, 3, 5\}$, $\{2, 3, 4, 5, 6\}$.

Stupně uzlů — k -jádro, k -plex. Každý uzel má mít určitý počet spojení na uzly *uvnitř* skupiny. k -jádro (k -core) je podstruktura, kde je každý uzel spojen s alespoň k dalšími členy skupiny. k -plex: pro skupinu o n_s uzlech má být každý uzel sousední s ne méně než $n_s - k$ uzly ve skupině. Definice jsou komplementární: k -jádro je $(n_s - k)$ -plex; množina všech k -jader pro dané k však *není* identická s množinou všech $(n_s - k)$ -plexů, protože velikost skupiny n_s se mezi k -jádry může lišit.

Vazby uvnitř vs. vně — LS množiny. *LS množina* (Luccio–Sami): libovolná vlastní podmnožina LS množiny má více vazeb na ostatní uzly *ve* skupině než *mimo* ni. Cílem bylo najít kohezni skupiny uzlů pro návrh obvodů na čipech; pro síťovou analýzu je definice *příliš striktní*. Uvolněnou definicí je k -komponenta — maximální podgraf, který zůstává souvislý i po odstranění méně než k vrcholů či hran; vyžaduje výpočet hranové souvislosti mezi libovolným párem uzlů algoritmem minimálního řezu / maximálního toku.

Omezení uzlově orientovaných definic: jsou příliš striktní (lze je však použít jako jádro komunity); nejsou škálovatelné, běžně se používají pro sítě malé velikosti; někdy nejsou konzistentní s vlastnostmi rozsáhlých sítí (např. stupně uzlů v bezškálových sítích).

Skupinově orientovaná detekce: kvazikliky

Skupinově orientovaná kritéria vyžadují, aby *celá skupina* splňovala určitou podmínku — např. aby hustota skupiny byla dostatečně velká ($\geq$ daný prah). Podgraf $G_S = (V_S, E_S)$ je γ -hustá

kvaziklika, je-li

$$|E_S| \geq \gamma \cdot \frac{|V_S|(|V_S| - 1)}{2}.$$

Lze použít strategii podobnou klikám: *lokální prohledávání* — vzorkuj podgraf a najdi maximální γ -hustou kvazikliku (velikosti k); *heuristické prořezávání* — odstraň uzly se stupněm menším než průměrný stupeň ($< \gamma(k - 1)$).

Síťově orientovaná detekce komunit

Kritéria uvažují spojení v síti *globálně*; cílem je rozdělit uzly sítě do *disjunktních* množin. Přístupy: shlukování podle podobnosti vrcholů; modely latentního prostoru; aproximace blokovým modelem; spektrální shlukování; maximalizace modularity.

Shlukování podle podobnosti vrcholů. Aplikuje se k -means nebo shlukování založené na podobnosti na uzly; podobnost vrcholů je definována podobností jejich *sousedství*. *Strukturní ekvivalence*: dva uzly jsou strukturně ekvivalentní právě tehdy, jsou-li spojeny s *touž* množinou aktérů — pro praktické použití je to příliš restriktivní. Praktické míry (spojení se chápou jako příznaky):

$$\text{Jaccard}(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}, \quad \cos(v_i, v_j) = \frac{|N_i \cap N_j|}{\sqrt{|N_i| |N_j|}}.$$

Pro rozsáhlé sítě se spočítá podobnost vrcholů kosinovou nebo Jaccardovou mírou a aplikuje se klasický k -means (každý klastr je asociován s centroidem, každý uzel je přiřazen ke klastru s nejbližším centroidem).

Modely latentního prostoru. Zobrazí uzly do *nízkorozměrného* prostoru tak, aby byla zachována blízkost uzlů podle propojenosti sítě, a pak aplikují k -means. *Vícerozměrné škálování* (MDS): pro danou síť se zkonstruuje matice blízkosti P reprezentující párové vzdálenosti uzlů (např. geodetické); nechť $S \in \mathbb{R}^{n \times l}$ jsou souřadnice uzlů v nízkorozměrném prostoru — řešením je $S = V\Lambda^{1/2}$, kde V je l hlavních vlastních vektorů (a Λ diagonální matice odpovídajících vlastních čísel) dvojité centrované matice blízkosti.

Blokové modely. Matice sousednosti se aproximuje jako $A \approx S\Sigma S^\top$, kde Σ je *míchací matice* a S *indikátorová matice komunit*; uvolní-li se S na reálné hodnoty, odpovídá optimální řešení hlavním vlastním vektorům A .

Spektrální shlukování. Poměrový i normalizovaný řez lze přeformulovat jako minimalizaci $\text{tr}(S^\top LS)$, kde $L = D - A$ je *grafový laplacián* (pro poměrový řez), resp. $L = D^{-1/2}(D - A)D^{-1/2}$ *normalizovaný grafový laplacián* (pro normalizovaný řez) a D diagonální matice stupňů. Po *spektrální relaxaci* je optimálním řešením matice tvořená vlastními vektory s *nejmenšími* vlastními čísly. (První vlastní vektor říká, že všechny uzly patří do téhož klastru — je nepoužitelný.)

Maximalizace modularity. Modularita měří sílu rozdělení tím, že bere v úvahu rozdělení stupňů; *síla komunity* je $\sum_{i,j \in C} (A_{ij} - \frac{d_i d_j}{2m})$ a modularita $Q = \frac{1}{2m} \sum_C \sum_{i,j \in C} (A_{ij} - \frac{d_i d_j}{2m})$. Analogicky ke spektrálnímu shlukování lze maximalizaci modularity přeformulovat pomocí *matice modularity* $B = A - \frac{dd^\top}{2m}$; optimálním řešením jsou *hlavní* vlastní vektory matice modularity, na které se jako postprocessing aplikuje k -means.

Jednotný pohled: maticová faktorizace. Modely latentního prostoru, blokové modely, spektrální shlukování a maximalizace modularity lze všechny zapsat jako faktorizaci $X \approx S\Sigma S^\top$, kde

$$X = \begin{cases} \text{(dvojitě centrovaná) matice blízkosti} & \text{modely latentního prostoru,} \\ \text{sociomatrice } A & \text{aproximace blokovým modelem,} \\ \text{grafový laplacián} & \text{minimalizace řezu,} \\ \text{matice modularity} & \text{maximalizace modularity.} \end{cases}$$

Omezení: všechny vyžadují, aby uživatel zadal *počet komunit* předem.

Hierarchicky orientovaná detekce komunit

Cíl: vybudovat hierarchickou strukturu komunit na základě topologie sítě; umožňuje analýzu sítě v různých *rozlišeních*. Dva reprezentativní přístupy:

Divizivní hierarchické shlukování — např. Girvanové–Newman ($O(m^2n)$, resp. $O(n^3)$ pro řídké sítě): rozděl uzly do několika množin; každou množinu dále rozděl na menší; na rozdělení lze aplikovat síťově orientované metody. Jeden konkrétní příklad: rekurzivně odstraňuj „nejslabší“ vazbu — najdi hranu s nejmenší silou, odstraň ji a aktualizuj sílu každé hrany; rekurzivně opakuj, dokud se síť nerozpadne na požadovaný počet komponent souvislosti. Každá komponenta pak tvoří komunitu. Sílu vazby lze měřit *mezilehlostí hrany* — hrana s vyšší mezilehlostí bývá mostem mezi dvěma komunitami.

Aglomerativní hierarchické shlukování — např. Louvain, $O(n \log n)$: inicializuj každý uzel jako komunitu; postupně slučuj komunity do větších podle konkrétního kritéria (např. na základě *přírůstku modularity*).

6.17 Predikce vazeb

Úloha predikce vazeb

Cíl: predikovat *budoucí* vazby mezi páry uzlů v síti. V komerčních sociálních sítích (Facebook, LinkedIn) se uživatelům doporučují potenciální přátelé. Použít lze podobnost obsahu i strukturu:

- **Míry založené na obsahu** využívají princip *homofilie* — uzly s podobným obsahem se spíše propojí. Zlepšují predikci vazeb, ale jsou citlivé na konkrétní síť (např. krátké a šumivé tweety s nestandardními akronymy na Twitteru). *Obsahová homofilie nutně neimplikuje strukturní propojenost!*
- **Strukturní míry** využívají princip *tranzitivního uzávěru* — dva uzly, které sdílejí sousedství, se spíše propojí. Tranzitivní uzávěr je velmi častý napříč doménami, míry jsou účinné v různých typech sítí a přímo aplikovatelné. Patří k nim *sousedské míry*, *Katzova míra*, míry založené na *náhodných procházkách*. *Strukturní propojenost obvykle implikuje i obsahovou homofilii.*

Sousedské míry predikce vazeb

Využívají počet *společných sousedů* mezi párem uzlů i a j ke kvantifikaci pravděpodobnosti budoucí vazby. Nechť S_i je množina sousedů uzlu i .

Míra společných sousedů: $CN(i, j) = |S_i \cap S_j|$. *Slabina:* nezohledňuje *relativní* počet společných sousedů vůči počtu ostatních spojení — kdyby byly Alice a Bob velmi populární veřejné osobnosti spojené s velkým počtem aktérů, mohli by mít mnoho společných sousedů *jen náhodou*.

Jaccardova míra: $Jaccard(i, j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ — měří podobnost sousedství, *normalizovanou* vůči stupňům uzlů, které se kvůli mocninnému rozdělení silně liší. Větší stupeň Alice nebo Boba tedy jejich Jaccardův koeficient snižuje. *Slabina:* nepřizpůsobuje se dobře *prostředním* sousedům — jsou-li společní sousedé Alice a Boba velmi populární (mají vysoký stupeň), pak se vyskytují jako společní sousedé i u mnoha jiných párů uzlů, a jsou proto pro predikci *méně důležití*.

Adamicova–Adarova míra — vážená verze míry společných sousedů, navržená tak, aby zohlednila různou důležitost různých společných sousedů: váha společného souseda je *klesající* funkcí jeho stupně, např. $1/\log |S_k|$:

$$AA(i, j) = \sum_{k \in S_i \cap S_j} \frac{1}{\log |S_k|}.$$

Příklad: pro čtyři společné sousedy se stupni 4, 2, 2, 4 je $\frac{1}{\log 4} + \frac{1}{\log 2} + \frac{1}{\log 2} + \frac{1}{\log 4} = \frac{3}{\log 2}$.

Katzova míra

Sousedské míry *nejdou účinné*, je-li počet sousedů sdílených párem uzlů malý. Sdílejí-li Alice a Bob jediného společného souseda a Alice a Jim také jediného, sousedské míry mezi těmito případy nerozliší — existuje však *nepřímá* propojenost přes delší cesty. V takových případech jsou vhodnější míry založené na *procházkách*, jako Katzova míra.

Nechť $W_{ij}^{(t)}$ je počet procházek délky t mezi uzly i a j . Pro předem zvolený parametr $\beta < 1$:

$$\text{Katz}(i, j) = \sum_{t=1}^{\infty} \beta^t W_{ij}^{(t)},$$

kde β je *diskontní faktor*, který potlačuje význam delších procházek; pro malá β nekonečná suma konverguje. Je-li A symetrická matice sousednosti neorientované sítě, lze matici Katzových koeficientů K rozměru $n \times n$ spočítat jako

$$K = \sum_{t=1}^{\infty} \beta^t A^t = (I - \beta A)^{-1} - I$$

(vlastní čísla A^t jsou t -té mocniny vlastních čísel A). Silně centrální uzly mají tendenci tvořit vazby s mnoha uzly $\Rightarrow$ součet Katzových koeficientů uzlu i vůči ostatním uzlům se nazývá jeho **Katzova centralita**.

Predikce vazeb jako klasifikační úloha

Přítomnost či absence vazby mezi párem uzlů se chápe jako *binární* indikátor třídy; pro každý pár uzlů se zkonstruuje vícerozměrný datový záznam:

- *příznaky* zahrnují všechny různé podobnosti mezi uzly — sousedské, Katzovy, založené na procházkách — plus řadu příznaků preferenčního připojování, např. stupně obou uzlů v páru;
- výsledkem je *pozitivně-neoznačkováná* klasifikační úloha: páry uzlů s hranou jsou pozitivní příklady a zbývající páry jsou neoznačkovány;
- neoznačkové příklady lze při trénování přibližně považovat za negativní;
- protože je negativních párů v rozsáhlých a řídkých sítích příliš mnoho, použije se jen *vzorek* negativních příkladů.

Algoritmus. *Trénovací fáze:* vygeneruj data s jedním záznamem pro každý pár uzlů s hranou a vzorek záznamů z párů bez hrany; příznaky odpovídají extrahovaným podobnostním a strukturním rysům; označení třídy je přítomnost či absence hrany; postav a natrénuj model klasifikátoru. *Testovací fáze:* převed' každý testovaný pár uzlů na vícerozměrný záznam a použij natrénovaný klasifikátor k predikci označení. Běžnou volbou bazového klasifikátoru je *logistická regrese*; kvůli nevyváženosti dat se často používají *cenově citlivé* verze klasifikátorů.

Výhody supervizovaného přístupu: obsahové příznaky lze extrahovat a použít snadno; klasifikátor se *sám* naučí relevanci příznaků; pro orientované sítě lze extrahovat příznaky i *asymetricky* (např. vstupní a výstupní stupně místo obyčejných stupňů, nebo různé míry založené na náhodných procházkách jako personalizovaný PageRank); vyšší flexibilita díky schopnosti učit se vztahy mezi vazbami a příznaky různých typů.

Souhrn predikce vazeb. Různé míry se v účinnosti liší podle datové sady: *sousedské* míry lze efektivně spočítat i pro rozsáhlá data a fungují téměř tak dobře jako ostatní nesupervizované míry; míry založené na *náhodných procházkách* a *Katzovy* jsou užitečné pro *velmi řídké* sítě, v nichž nelze počet společných sousedů robustně změřit; *supervize* poskytuje lepší přesnost a adaptabilitu napříč doménami sociálních sítí, ale je výpočetně drahá; v posledních letech se k vylepšení predikce používá i obsah, ale strukturní míry jsou *výrazně silnější* (kvůli tranzitivním vlastnostem reálných sítí); obsahové míry jsou založeny na „obrácené homofilii“, kdy se pro predikci vazeb využívá podobný nebo s vazbami korelovaný obsah.

6.18 Shrnutí ke zkoušce

- **SNA** = sociální vědy + analýza sítí + teorie grafů; soustředí se na *vztahy*, ne na entity a jejich atributy. Reprezentace: obrázek, seznam hran, matice sousednosti (u orientovaných grafů nesymetrická). *Ego síť* vs. „celá“ síť — *problém specifikace hranic*: žádná studovaná síť není celá.
- **Síla vazeb**: silné vs. slabé; *síla slabých vazeb* (Granovetter 1973). Homofilie, tranzitivita a přemostění; homofilie + tranzitivita → *kliky*. Most = hrana, jejíž odstranění rozpojí koncové uzly; *zkratkový most* = vzdálenost po odstranění vzroste nad 2. **Překryv sousedství** $O(i, j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2}$ — čím větší, tím silnější vazba.
- **Čtyři míry centrality**: *stupňová* (kolik lidí přímo dosáhne), *mezilehlá* (na kolika nejkratších cestách leží; kde se síť rozpadne), *blízkostní* (převrácená průměrná délka nejkratších cest; rychlost šíření), *vlastní* ($Ax = \lambda x$, jen *největší* vlastní číslo dává nezáporný vektor; jak dobře je spojen s dobře propojenými). Umět interpretační tabulku a vysvětlit, proč nestačí jedna míra (uzly 3 a 5 *společně* > uzel 10). **Strukturní ekvivalence**: pozice lze vyměnit bez změny struktury.
- **Koheze**: reciprocita (jen orientované grafy), hustota ($|E|/|V|^2$); klika = 1; orientované mají poloviční, **klastrovací koeficient** $C_i = \frac{2|e_{jk}|}{k_i(k_i-1)}$ a síť jako průměr, průměr sítě (*diameter*) = nejdelší nejkratší cesta, průměrná vzdálenost, excentricita. **Malý svět** = vysoký C + krátká průměrná cesta.
- **Šest vlastností reálných komplexních sítí**: efekt malého světa (Milgram, medián 6), tranzitivita/klastrování, mocinné rozdělení stupňů, odolnost (robustní k náhodnému, zranitelná cíleným odstraněním hubů), vzorce míchání (asortativní míchání = homofilie), komunitní struktura. **Bezškálové sítě**: růst + *preferenční připojování* (rich-get-richer, Matoušův efekt, kumulativní výhoda); důvody — popularita, kvalita, smíšený model (halo efekt). **Centralizace** a struktura *jádro-periferie* (bow-tie analýza).
- **Modelování vlivu**: společné rysy (orientovaný graf, aktivní/neaktivní, *aktivovaný uzel nelze deaktivovat*). **LTM** — prah θ_v uniformně z $[0, 1]$, aktivace při $\sum_{\text{aktivní}} w_{u,v} \geq \theta_v \sum w_{u,v}$; soustředí se na *příjemce*. **ICM** — *jediná* příležitost aktivovat každého souseda s pravděpodobností $p_{v,u}$; soustředí se na *vysílající*. **Maximalizace vlivu** je NP-těžká, hladový přístup dává **63 %** optima.
- **Vliv vs. korelace**: *test korelace* (podíl hran mezi různými hodnotami atributu vs. $2p(1-p)$; příklad kuřáků 14 % < 49 %); tři procesy — homofilie, *zmatení*, vliv; logistický model $\ln \frac{p(a)}{1-p(a)} = \alpha \ln(a+1) + \beta$, α = koeficient sociální korelace; **shuffle test** — zpřeházej časové značky; liší-li se α' významně od α , jde o důkaz *vlivu*.
- **Test homofilie**: heterogenních hran je významně méně než $2pq \Rightarrow$ homofilie; významně *více* $\Rightarrow$ *inverzní* homofilie (partnerské vztahy). Umět příklad: 5 z 18 vs. $2pq = 8$ z 18.
- **Výběr vs. sociální vliv**: výběr = charakteristiky řídí tvorbu vazeb; vliv = vazby formují *mutabilní* charakteristiky. *Imutabilní* (pohlaví, rasa, věk) vs. *mutabilní* (chování, názory). Rozlišit lze jen *longitudinálními* studiemi (obezita: Christakis & Fowler, 12 000 lidí / 32 let → důkaz pro *sociální vliv*).
- **Afiliční sítě**: bipartitní (osoby × fokusy), sociálně-afiliční má dva typy hran. **Tři uzávěry**: tranzitivní (společný přítel), *fokální* (společný fokus; *výběr*), *členský* (přítel už ve fokusu; *sociální vliv*). Metodika měření $T(k)$ ze dvou snímků; báze model $T_{\text{base}}(k) = 1 - (1-p)^k$; empiricky velký nárůst z 1 na 2 a poté *klesající výnosy*.
- **Schellingův model**: agenti dvou typů v mřížce, spokojeni při $\geq t$ sousedech téhož typu, nespokojení se přesouvají $\Rightarrow$ *samovolná segregace*. Ukazuje, jak se *imutabilní* charakteristika stane silně korelovanou s *mutabilní* (volbou místa) a jak lokální homofilie vytvoří globální vzorec.
- **Strukturní balance**: balancované trojúhelníky jsou +++ a +-- (tj. všechny tři pozitivní, nebo právě jedna pozitivní). **Věta o balanci**: úplný balancovaný graf $\Rightarrow$ všichni přátelé, nebo rozdělení na dvě skupiny vnitřních přátel a vzájemných nepřátel — umět důkaz (X = přátelé A , Y = nepřátelé A , tři kroky). **Slabá balance**: zakázána jen konfigurace *dvě pozitivní a jedna negativní* $\Rightarrow$ rozdělení na *libovolný počet* skupin. Aplikace: evropské aliance 1872–1907, Bangladéš 1972.
- **HITS**: dotazově *závislý*; kořenová množina S ($t = 200$) $\rightarrow$ báze množina B ; $a = L^\top h$, $h = La$,

mocninná iterace s **nutnou normalizací**; $a_k = (L^\top L)^{k-1} a_0$ konverguje k hlavnímu vlastnímu vektoru $L^\top L = ko-citační$ matice ($LL^\top =$ bibliografické párování). Slabiny: snadné spamování, drift tématu, neefektivnost v době dotazu.

- **PageRank**: dotazově *nezávislý*, $P(i) = \sum_{(j,i) \in E} P(j)/O_j$, *nepotřebuje normalizaci* (celkový PageRank je konstantní — „kapalina“). Problém „pomaleho úniku“ $\Rightarrow$ *škálované* pravidlo s tlumicím faktorem d (0,8–0,9; 0,85 v článku), „vodní cyklus“. Markovský řetězec potřebuje *stochastickou, ireducibilní a aperiodickou* matici — webový graf nesplňuje ani jednu (visící stránky, není silně souvislý, periodicitu); vše se řeší *jedinou* strategií (odkaz z každé stránky na každou s malou pravděpodobností). Finální tvar $P(i) = (1-d) + d \sum_{(j,i) \in E} P(j)/O_j$, výpočet mocninnou iterací. Výhody: odolnost proti spamu (vstupní odkazy nelze snadno získat), globální offline míra; kritika: dotazová nezávislost.
- **Tematické komunity**: $K = (T, M)$, téma určuje komunitu, komunity se mohou *překrývat*, hierarchie subkomunit. *Bipartitní jádra* — (i, j) -jádro, vějíře = huby, středy = autority; prořezání + generování jader; najde jen jádra, ne témata ani hierarchii. *Komunity pojmenovaných entit*: graf ze společných výskytů ve větě $\rightarrow$ trojúhelníky $\rightarrow$ jádra $\rightarrow$ klastrování podle textové podobnosti.
- **Detekce komunit** — **definice je subjektivní**; explicitní vs. implicitní skupiny. *Minimální řez*: nevyvážené skupiny, $O(n^5)$ / Karger $O(n^4)$, s obecnými vahami a balančními omezeními **NP-těžké** $\Rightarrow$ *poměrový a normalizovaný řez* (dělení $|C_i|$, resp. $\text{vol}(C_i)$).
- **Girvanové–Newman**: divizivní, odstraňuje hrany s nejvyšší *mezilehlostí*, $O(m^2 n) / O(n^3)$; počet komunit se vybere podle nejvyšší **modularity** $Q = \frac{1}{2m} \sum_{i,j} (A_{ij} - \frac{k_i k_j}{2m}) \delta(c_i, c_j)$, $Q \in [-1, 1]$, smysluplné komunity $Q \geq 0,3$.
- **Louvain**: hladová aglomerativní maximalizace modularity ve dvou fázích (lokální přesuny $\rightarrow$ supergraf), $O(n \log n)$; nevýhody: citlivost na pořadí, *nesouvislé komunity*, lokální optima. **Leiden** přidává fázi *upřesnění rozdělení*; jen tvrdé rozdělení. Výzvy: neznámý počet komunit, heterogenní velikosti a hustoty.
- **Čtyři kategorie metod**: *uzlově orientované* (kliky — NP-těžké, rekursivní prořezávání uzlů se stupněm $< k-1$, **CPM** pro *překrývající se* komunity přes graf klik sdílejících $k-1$ uzlů; k -klika / k -klub / k -klan; k -jádro a k -plex jsou komplementární; LS množiny a k -komponenty), *skupinově orientované* (γ -husté kvazikliky), *síťově orientované* (podobnost vrcholů — Jaccard, kosinus + k -means; MDS; blokové modely; spektrální shlukování přes grafový laplacián s *nejmenšími* vlastními čísly; maximalizace modularity přes matici modularity s *největšími* — vše sjednoceno jako maticová faktorizace $X \approx S \Sigma S^\top$, vyžaduje zadat počet komunit), *hierarchicky orientované* (divizivní = Girvanové–Newman, aglomerativní = Louvain).
- **Predikce vazeb**: obsahové míry (homofilie) vs. strukturní (tranzitivní uzávěr; silnější). Sousedské míry: *společní sousedé* $|S_i \cap S_j|$ (nezohledňuje stupně), *Jaccard* $\frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ (normalizuje stupně obou uzlů, ne prostředních sousedů), *Adamic–Adar* $\sum_k \frac{1}{\log |S_k|}$ (váha klesá se stupněm společného souseda). **Katzova míra** $\sum_t \beta^t W_{ij}^{(t)} = (I - \beta A)^{-1} - I$ pro *řídke* sítě s málo společnými sousedy; součet přes ostatní uzly = *Katzova centralita*. **Klasifikační pojetí**: pozitivně-neoznačkováná úloha, vzorek negativ, logistická regrese jako bazový klasifikátor, cenově citlivé varianty; výhoda — klasifikátor se sám naučí relevanci příznaků a lze použít *asymetrické* příznaky u orientovaných sítí.

7 Praktické využití technik

Poslední věta okruhu žádá spojení obou polovin dohromady: k čemu se probrané techniky *skutečně* používají. U zkoušky se hodí umět ke každé aplikaci říct *jakou úlohu* řeší (kap. 1), *jakou metodou* (kap. 3) a *jak se vyhodnocuje* (kap. 5).

7.1 Dobývání znalostí

Aplikace	Typ úlohy / metoda	Poznámka
Analýza nákupního košíku	asociační pravidla, Apriori / FP-Growth	plánování a rozvržení prodejny, kupony a časově omezené slevy, „balíčkování“ produktů; srovnání nových a existujících prodejen pomocí <i>virtuálních položek</i>
Detekce podvodů	klasifikace, detekce odlehlých hodnot	20 mil. výkonů zdravotní pojišťovny → 214 tis. podivných → 14 tis. předaných k ruční kontrole → 8 tis. potvrzených podvodů; stroj <i>zaostřuje</i> pozornost experta
Kreditní skórování	rozhodovací stromy, naivní Bayes, logistická regrese	průběžný příklad v celém textu; požaduje se <i>srozumitelnost</i> (regulace) a zacházení s chybějícími hodnotami
Segmentace zákazníků	klastrová analýza (<i>k</i> -means, hierarchické)	reprezentace segmentu <i>centroidem</i> („typický zákazník“) je pro byznys velmi intuitivní
Predikce odchodu (churn)	klasifikace na nevyvážených datech	accuracy nestačí ⇒ precision/recall, F_1 , ROC/AUC; <i>lift analýza</i> pro rozhodnutí, kolik zákazníků kontaktovat
Cílené marketingové kampaně	skórování a řazení, lift	ekonomická úvaha nad lift křivkou (příklad s hodinkami: 1. příhrádka +\$550, 10. příhrádka −\$475)
Klasifikace textu	SVM s kernely	patří k nejlepším klasifikátorům pro text (vysokorozměrná data); srozumitelnost se neočekává
Predikce časových řad	regrese, neuronové sítě	řada se transformuje posuvným oknem na tabulku vstupů a výstupů
Klasifikace chemických látek	klasifikace strukturních dat	reprezentace řetězci SMILES nebo fakty v Prologu
Web usage mining	sekvenční vzory (GSP)	navigační vzory uživatelů podle posloupností návštěv stránek
Doporučovací systémy [♦ nad rámec slidů]	klastrová analýza, klasifikace, predikce vazeb	kolaborativní filtrování (podobnost uživatelů či položek) vs. obsahové filtrování; studený start řeší obsah

Poznámka k motivaci rozhodovacích stromů. Přednáška uvádí případ banky Barclays, která zavedla telefonní systém rozdělující volající podle úvěrového hodnocení: klienti s vysokými úsporami nebo dostatečným úvěrovým limitem byli spojováni s britským operátorem, ostatní s indickým call centrem. Cílem bylo „odfiltrout“ zákazníky, kteří nemají peníze na nákup produktů banky. Kritika: systém je preferenční vůči bohatším klientům a nemá za cíl poskytnout nejlepší radu. Je to dobrá ilustrace toho, že *technicky správný* klasifikační model může mít problematické důsledky — a že fáze *vyhodnocení* v CRISP-DM musí zahrnovat i pohled mimo přesnost modelu.

7.2 Analýza sociálních sítí

Oblast	Využití
Podniky	analýza a zlepšení komunikačních toků v organizaci, s partnery či zákazníky; marketingové kampaně založené na sociálních sítích (vizualizace komunikačních toků před a po zavedení systému pro správu obsahu)
Bezpečnostní složky a armáda	identifikace kriminálních a teroristických sítí ze zachycené komunikace; identifikace <i>klíčových hráčů</i> v nich (míry centrality, odd. 6)
Sociální sítě (Facebook)	identifikace a doporučení potenciálních přátel na základě „přátel přátel“ — přímo úloha <i>predikce vazeb</i>
Občanská společnost	odhalování konfliktů zájmů ve skrytých vazbách mezi státními orgány, lobbisty a podniky
Síťoví operátoři	optimalizace struktury a kapacity komunikačních sítí
Medicína	nakažlivé nemoci a jejich prevence; sexuálně přenosné nemoci — epidemie syfilis v Rockdale County (Atlanta, 1996): skupina bílých dívek okolo 16 let, skupina bílých chlapců 17–21 let a skupina afroamerických chlapců 17–21 let; syfilis diagnostikována u 6 bílých žen, 2 bílých mužů a 2 afroamerických mužů
Bezškálové sítě	<i>výpočetní technika</i> : sítě s SF architekturou (WWW) jsou extrémně odolné proti náhodným poruchám, ale velmi zranitelné koordinovaným útokem — vymýcení internetových virů je v podstatě nemožné; <i>medicína</i> : vakcinační kampaně cílené na huby (lidé s mnoha kontakty — jejich identifikace je ale obtížná), nové léky mířící na klíčové molekuly (huby) dané nemoci s minimalizací vedlejších účinků pomocí nalezených vazebných sítí v buňkách; <i>business</i> : porozumění vazbám mezi firmami, odvětvím a ekonomikou pro monitoring a eliminaci kaskádových finančních krachů; marketing — zkoumání šíření „nákazy“ v SF sítích a efektivnější reklama nových produktů
Detekce komunit	<i>Belgický mobilní operátor</i> (Louvainova metoda): cca 2 miliony zákazníků; velikost uzlu odpovídá počtu jednotlivců v komunitě, barva hlavnímu jazyku komunity (červená francouzština, zelená nizozemština); zobrazeny jen komunity nad 100 zákazníků. Zoom ve vyšším rozlišení odhalí <i>přechodovou</i> komunitu tvořenou subkomunitami s méně zřetelným jazykovým oddělením

Kdy a proč SNA použít. Studovat sociální síť (offline i online) nebo zvýšit její efektivitu; vizualizovat data tak, aby se odhalily vzorce ve vztazích či interakcích; sledovat cesty, kterými v sociálních sítích proudí informace; v kvantitativním výzkumu (pro kvalitativní je síťová perspektiva také cenná) — rozsah akcí a příležitostí dostupných jednotlivcům je často funkcí jejich *pozice* v sociální síti a odhalení těchto pozic (např. jako otců, matek, učitelů, pracovníků) může přinést zajímavé a někdy překvapivé výsledky; kvantitativní analýza sítě pomáhá identifikovat různé typy aktérů (klíčové hráče); analyzovat sociální média obecně, identifikovat příčiny dysfunkčních komunit či sítí a podporovat sociální kohezi a růst online komunity.

Úvahy o návrhu (*thoughts on design*)

Teorie SNA inspirovala některé z prvních sociálních sítí (např. SixDegrees), stále se však při návrhových rozhodnutích nepoužívá tak často. Otevřené otázky:

- Jak může online platforma (nebo její správci) využít metody a poznatky SNA?
- Jak může platforma povzbudit uživatele, aby jednali (*lokálně*) tak, že tím rozšíří komunitu?
- Jaká opatření může online komunita přijmout k optimalizaci své struktury? Příklad: *kliky* mohou být nežádoucí, protože odhánějí nově přichozí.
- Jaké struktury jsou žádoucí pro různé typy online platform?
- Jak mohou online komunity identifikovat a využít klíčové hráče ke svému prospěchu?

7.3 Provozní a etické aspekty nasazení

◆ nad rámec slidů] Nasazení modelu (fáze *Deployment* v CRISP-DM, odd. 1.5) není konec projektu:

- **Monitoring a drift.** Rozdělení dat se v čase mění (*data drift*), stejně jako vztah mezi vstupy a cílem (*concept drift*) — model je nutné pravidelně přeučovat a sledovat pokles kvality na produkčních datech.
- **Zpětná vazba modelu do dat.** Nasazený model ovlivňuje realitu, kterou měří: skórovací model pro schvalování úvěrů vidí jen ty žadatele, kterým úvěr schválil, takže další trénovací data jsou jím samým zkreslená.
- **Soukromí.** Analýza sociálních sítí pracuje s daty o vztazích, které uživatel často nemohl ovlivnit; i anonymizovaný graf lze deanonymizovat podle struktury. Predikce vazeb a modelování vlivu jsou v podstatě nástroje pro předpovídání a ovlivňování chování konkrétních lidí.
- **Spravedlnost a diskriminace.** Model naučený na historických datech reprodukuje historickou nerovnost (viz případ Barclays výše); chráněný atribut lze často *rekonstruovat* z ostatních atributů, takže jeho vypuštění samo nestačí.
- **Právní rámec.** GDPR čl. 22 (právo na vysvětlení automatizovaného rozhodnutí) a AI Act pro vysoce rizikové systémy — v regulovaných doménách je interpretovatelnost právním požadavkem, nikoli pohodlím.

7.4 Shrnutí ke zkoušce

- Ke každé aplikaci umět říct **trojici**: typ úlohy (deskriptivní / prediktivní, s učitelem / bez učitele), použitá metoda, způsob vyhodnocení.
- **Vlajkové aplikace dobývání znalostí**: analýza nákupního košíku (asociační pravidla), detekce podvodů (příklad zdravotní pojišťovny — stroj *zaostřuje* pozornost experta, nenahrazuje jej), kreditní skórování (stromy a Bayes; požadavek srozumitelnosti), segmentace zákazníků (klastrování, centroid = typický zákazník), churn a cílený marketing (nevyvážená data → ROC/AUC a *lift*), klasifikace textu (SVM), predikce časových řad (posuvné okno), web usage mining (sekvenční vzory).
- **Vlajkové aplikace SNA**: identifikace klíčových hráčů v kriminálních a teroristických sítích (centralita), doporučování přátel (predikce vazeb), optimalizace komunikačních sítí, epidemiologie (Rockdale County 1996), vakcinace cílená na *huby*, marketing a kaskádové finanční krachy v bezškálových sítích, segmentace zákazníků operátora podle jazykových komunit (Louvain, 2 mil. zákazníků).
- **Klíčová vlastnost SF sítí pro praxi**: odolnost proti náhodným poruchám (až 80 % uzlů) vs. zranitelnost cíleným útokem (5–15 % hubů) — z toho plyne jak strategie vakcinace, tak nemožnost vymýtit internetové viry.
- **Nasazení není konec**: monitoring driftu, zpětná vazba modelu do dat, soukromí (i anonymizovaný graf lze deanonymizovat), spravedlnost (chráněný atribut lze rekonstruovat z ostatních), GDPR čl. 22 a AI Act.